Exploring Virtual Reality for Aviation Theory

1.0

Generated by Doxygen 1.15.0

Chapter 1

Todo List

Member ClimbingTurnActivity.HandleSlipStream ()

Slipstream animation/effect needs implementation

Chapter 2

Directory Hierarchy

2.1 Directories

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 5

File Index

5.1 File List

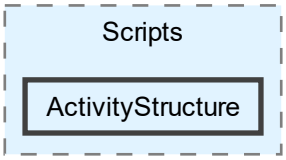
Here is a list of all files with brief descriptions:

Chapter 6

Directory Documentation

6.1 Assets/Scripts/ActivityStructure Directory Reference

Directory dependency graph for ActivityStructure:



Files

- file ActivityHelper.cs
- file IActivityController.cs

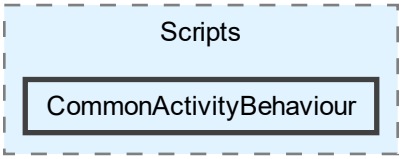
6.2 Assets Directory Reference

Directories

- directory Editor
- directory ModuleExclusiveAssets
- directory Scripts

6.3 Assets/Scripts/CommonActivityBehaviour Directory Reference

Directory dependency graph for CommonActivityBehaviour:

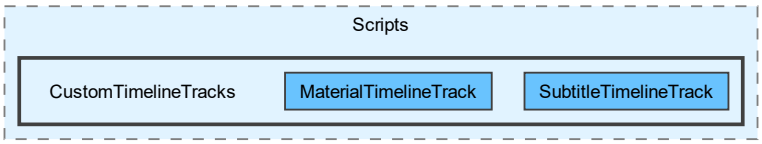


Files

- file AccelerateScript.cs
- file ControllerBackward.cs
- file DetectRotation.cs
- file RotateLeftController.cs

6.4 Assets/Scripts/CustomTimelineTracks Directory Reference

Directory dependency graph for CustomTimelineTracks:

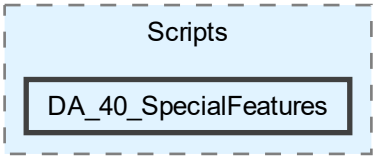


Directories

- directory MaterialTimelineTrack
- directory SubtitleTimelineTrack

6.5 Assets/Scripts/DA_40_SpecialFeatures Directory Reference

Directory dependency graph for DA_40_SpecialFeatures:



Files

- file BankGhostTrailBehaviour.cs
- file DA_40.cs
- file PlaneComponentHighlighter.cs
- file ProceduralAirflow.cs

6.6 Assets/Editor Directory Reference

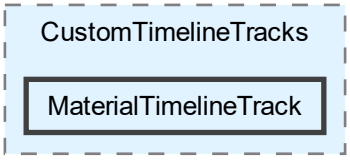
Directory dependency graph for Editor:

Files

- file GhostMeshGenerator.cs

6.7 Assets/Scripts/CustomTimelineTracks/MaterialTimelineTrack Directory Reference

Directory dependency graph for MaterialTimelineTrack:



Files

- file MaterialBehaviour.cs
- file MaterialClip.cs
- file MaterialTrack.cs

6.8 Assets/Scripts/Misc Directory Reference

Directory dependency graph for Misc:

Files

- file BankGhostTrailDemo.cs

6.9 Assets/Scripts/MiscBehaviour Directory Reference

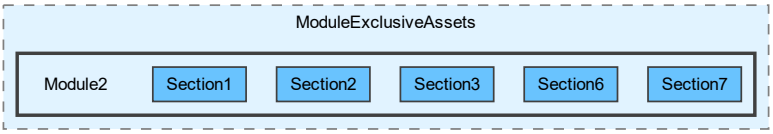
Directory dependency graph for MiscBehaviour:

Files

- file AxisRotationController.cs
- file GhostTrailToggle.cs
- file MovePlaneToPlatform.cs
- file SkyboxRotation.cs

6.10 Assets/ModuleExclusiveAssets/Module2 Directory Reference

Directory dependency graph for Module2:



Directories

- directory Section1
- directory Section2
- directory Section3
- directory Section6
- directory Section7

6.11 Assets/ModuleExclusiveAssets Directory Reference

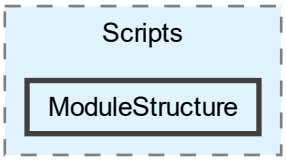
Directory dependency graph for ModuleExclusiveAssets:

Directories

- directory Module2

6.12 Assets/Scripts/ModuleStructure Directory Reference

Directory dependency graph for ModuleStructure:



Files

- file ModuleActivities.cs
- file ModuleActivityScheduler.cs
- file ModuleManager.cs

6.13 Assets/ModuleExclusiveAssets/Module2/Section2/NEW Directory Reference

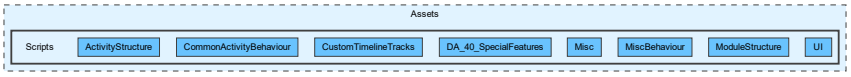
Directory dependency graph for NEW:

Files

- file IncreaseThrottle.cs

6.14 Assets/Scripts Directory Reference

Directory dependency graph for Scripts:

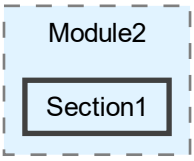


Directories

- directory ActivityStructure
- directory CommonActivityBehaviour
- directory CustomTimelineTracks
- directory DA_40_SpecialFeatures
- directory Misc
- directory MiscBehaviour
- directory ModuleStructure
- directory UI

6.15 Assets/ModuleExclusiveAssets/Module2/Section1 Directory Reference

Directory dependency graph for Section1:

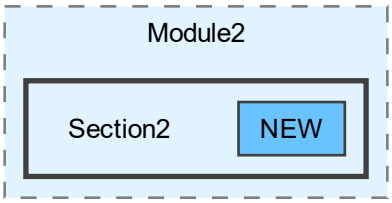


Files

- file Sec1StallActivity.cs

6.16 Assets/ModuleExclusiveAssets/Module2/Section2 Directory Reference

Directory dependency graph for Section2:



Directories

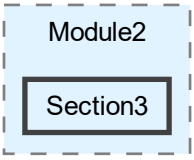
- directory NEW

Files

- file HoldToLowerNose.cs

6.17 Assets/ModuleExclusiveAssets/Module2/Section3 Directory Reference

Directory dependency graph for Section3:

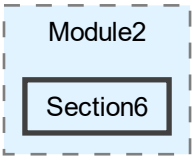


Files

- file ClimbingTurnActivity.cs
- file DescendingTurnActivity.cs
- file LevelTurnActivity.cs

6.18 Assets/ModuleExclusiveAssets/Module2/Section6 Directory Reference

Directory dependency graph for Section6:

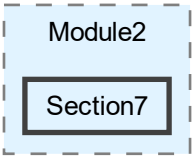


Files

- file Sec6SpinRecoveryActivity.cs

6.19 Assets/ModuleExclusiveAssets/Module2/Section7 Directory Reference

Directory dependency graph for Section7:

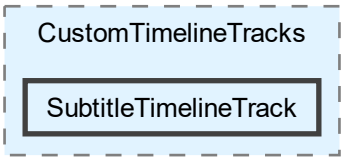


Files

- file TiltController.cs

6.20 Assets/Scripts/CustomTimelineTracks/SubtitleTimelineTrack Directory Reference

Directory dependency graph for SubtitleTimelineTrack:



Files

- file SubtitleBehaviour.cs
- file SubtitleClip.cs
- file SubtitleTrack.cs

6.21 Assets/Scripts/UI Directory Reference

Directory dependency graph for UI:

Files

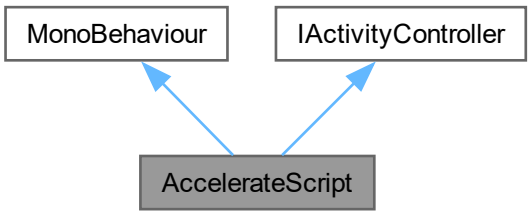
- file AirspeedMeterDisplay.cs
- file AoADisplay.cs
- file CheckboxGroupUI.cs
- file CheckboxManager.cs
- file MenuController.cs
- file ModuleMenuSelection.cs

Chapter 7

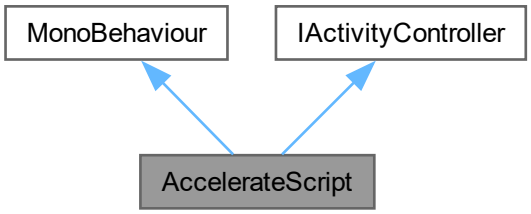
Class Documentation

7.1 AccelerateScript Class Reference

Inheritance diagram for AccelerateScript:



Collaboration diagram for AccelerateScript:



Public Member Functions

- void StartActivity ()

  Each child activity can have a differing implementation of what must be carried out/what inputs must be switched on when the activity starts.
- bool getHasTriggered ()

Public Attributes

- InputActionReference inputAction
- float throttleThreshold = 0.1f

Properties

- ModuleActivityScheduler mas `[get]`

Private Member Functions

- void OnEnable ()
- IEnumerator WaitCoroutine ()
- void OnDisable ()
- void OnPadMoved (InputAction.CallbackContext ctx)
- void OnPadReleased (InputAction.CallbackContext ctx)

Private Attributes

- bool activityEnabled = false
- bool hasTriggered = false

7.1.1 Detailed Description

Definition at line 7 of file AccelerateScript.cs.

7.1.2 Member Function Documentation

7.1.2.1 getHasTriggered()

```
bool AccelerateScript.getHasTriggered ()
```

Definition at line 88 of file AccelerateScript.cs.

7.1.2.2 OnDisable()

```
void AccelerateScript.OnDisable ()  [private]
```

Definition at line 34 of file AccelerateScript.cs.

7.1.2.3 OnEnable()

```
void AccelerateScript.OnEnable ()  [private]
```

Definition at line 18 of file AccelerateScript.cs.

7.1.2.4 OnPadMoved()

```
void AccelerateScript.OnPadMoved (
            InputAction.CallbackContext ctx)  [private]
```

Definition at line 51 of file AccelerateScript.cs.

7.1.2.5 OnPadReleased()

```
void AccelerateScript.OnPadReleased (
            InputAction.CallbackContext ctx)  [private]
```

Definition at line 76 of file AccelerateScript.cs.

7.1.2.6 StartActivity()

```
void AccelerateScript.StartActivity ()
```

Each child activity can have a differing implementation of what must be carried out/what inputs must be switched on when the activity starts.

Override this method for new activity scripts.

Implements IActivityController.

Definition at line 45 of file AccelerateScript.cs.

7.1.2.7 WaitCoroutine()

```
IEnumerator AccelerateScript.WaitCoroutine ()  [private]
```

Definition at line 29 of file AccelerateScript.cs.

7.1.3 Member Data Documentation

7.1.3.1 activityEnabled

```
bool AccelerateScript.activityEnabled = false  [private]
```

Definition at line 13 of file AccelerateScript.cs.

7.1.3.2 hasTriggered

```
bool AccelerateScript.hasTriggered = false  [private]
```

Definition at line 15 of file AccelerateScript.cs.

7.1.3.3 inputAction

```
InputActionReference AccelerateScript.inputAction
```

Definition at line 11 of file AccelerateScript.cs.

7.1.3.4 throttleThreshold

```
float AccelerateScript.throttleThreshold = 0.1f
```

Definition at line 12 of file AccelerateScript.cs.

7.1.4 Property Documentation

7.1.4.1 mas

```
ModuleActivityScheduler AccelerateScript.mas  [get], [private]
```

Definition at line 9 of file AccelerateScript.cs.

The documentation for this class was generated from the following file:

- Assets/Scripts/CommonActivityBehaviour/AccelerateScript.cs

7.2 ActivityHelper Class Reference

Static helper class for activity management and control.

Static Public Member Functions

- static PlayableDirector FindPlayableDirector (TimelineAsset asset)

  Finds the PlayableDirector associated with a given TimelineAsset.
- static PlayableDirector[ ] FindPlayableDirector (TimelineAsset[ ] assets)

  Finds PlayableDirectors for multiple TimelineAssets.
- static ProceduralAirflow getProceduralAirflow ()

  Retrieves the ProceduralAirflow component from the scene.
- static AxisRotationController getAxisRotationController ()

  Retrieves the AxisRotationController component from the scene.
- static AxisRotationController getAxisRotationController (float aoa=0, float bank=0)

  Retrieves and configures the AxisRotationController with initial rotation values.
- static BankGhostTrailBehaviour getBankGhostTrail (bool enable=true)

  Retrieves and configures the BankGhostTrailBehaviour component.
- static float normalizedControllerZRotation (InputActionReference input, float initial=0)

  Returns normalized Z-axis rotation from VR controller input.
- static float normalizedControllerXRotation (InputActionReference input, float initial=0)

  Returns normalized X-axis rotation from VR controller input.
- static void DebugMouseControlRoll (float speed, AxisRotationController manip)

  Debug function for mouse-based roll control.
- static void DebugMouseControlPitch (float speed, AxisRotationController manip)

  Debug function for mouse-based pitch control.
- static void DebugMouseControlRollLeft (float speed, AxisRotationController manip)

  Debug function for mouse-based left roll control.
- static void DebugMouseControlPitchUp (float speed, AxisRotationController manip)

  Debug function for mouse-based pitch up control.
- static void controllerRollControl (float speed, AxisRotationController manip, InputActionReference input, float deadZone, float initial=0, Vector2 vector=default)

  Processes VR controller input for aircraft roll control.
- static void controllerPitchControl (float speed, AxisRotationController manip, InputActionReference input, float deadZone, float initial=0, Vector2 vector=default)

  Processes VR controller input for aircraft pitch control.
- static float clampRange (float rotation, Vector2 range)

  Clamps a rotation value within a specified range.
- static float getRotationSpeed (float rotation, float speed, float deadZone)

  Calculates rotation speed with deadzone compensation.
- static bool checkRotationTargetAchieved (float currentRotation, float targetRotation, float tolerance)

  Checks if current rotation is within tolerance of target (absolute value comparison).
- static bool checkExactRotationTargetAchieved (float currentRotation, float targetRotation, float tolerance)

  Checks if current rotation exactly matches target within tolerance.
- static float getNormalisedPlaneRoll (AxisRotationController manip)

  Gets the aircraft's normalized roll angle.
- static float getNormalisedPlanePitch (AxisRotationController manip)

  Gets the aircraft's normalized pitch angle.
- static IEnumerator PlayTimeLine (PlayableDirector director, System.Action onComplete, float fallBack↩ Wait=1f)

  Coroutine to play a Unity Timeline and invoke callback on completion.

7.2.1 Detailed Description

Static helper class for activity management and control.

Provides utility functions for:

- Timeline and PlayableDirector management
- VR controller input processing and normalization
- Aircraft rotation control (pitch and roll)
- Debug mouse controls for testing
- Rotation clamping and target verification

Author

Connor Freebairn | frecd002@mymail.unisa.edu.au

Definition at line 21 of file ActivityHelper.cs.

7.2.2 Member Function Documentation

7.2.2.1 checkExactRotationTargetAchieved()

```
bool ActivityHelper.checkExactRotationTargetAchieved (
            float currentRotation,
            float targetRotation,
            float tolerance)  [static]
```

Checks if current rotation exactly matches target within tolerance.

Compares current and target rotation directly, respecting sign. Useful for checking if at a specific signed rotation (e.g., exactly at +30 or -30 degrees).

Parameters

currentRotation	Current rotation value
targetRotation	Target rotation value
tolerance	Acceptable difference threshold

Returns

True if within tolerance, false otherwise

Definition at line 472 of file ActivityHelper.cs.

7.2.2.2 checkRotationTargetAchieved()

```
bool ActivityHelper.checkRotationTargetAchieved (
            float currentRotation,
            float targetRotation,
            float tolerance)  [static]
```

Checks if current rotation is within tolerance of target (absolute value comparison).

Compares the absolute values of current and target rotation. Useful when direction doesn't matter, only magnitude (e.g., checking if at 30 degrees regardless of sign).

Parameters

currentRotation	Current rotation value
targetRotation	Target rotation value
tolerance	Acceptable difference threshold

Returns

True if within tolerance, false otherwise

Definition at line 452 of file ActivityHelper.cs.

7.2.2.3 clampRange()

```
float ActivityHelper.clampRange (
            float rotation,
            Vector2 range)  [static]
```

Clamps a rotation value within a specified range.

Parameters

rotation	Rotation value to clamp
range	Vector2 containing min (x) and max (y) clamp values

Returns

Clamped rotation value

Definition at line 416 of file ActivityHelper.cs.

7.2.2.4 controllerPitchControl()

```
void ActivityHelper.controllerPitchControl (
            float speed,
            AxisRotationController manip,
            InputActionReference input,
            float deadZone,
            float initial = 0,
            Vector2 vector = default)  [static]
```

Processes VR controller input for aircraft pitch control.

Reads controller X-axis rotation and applies it to the aircraft's angle of attack with deadzone filtering and speed scaling. Inverts the input so pulling back increases AOA. Clamps the resulting pitch to the specified range.

Parameters

speed	Maximum pitch speed multiplier
manip	AxisRotationController to manipulate
input	InputActionReference for reading controller rotation
deadZone	Deadzone threshold below which input is ignored
initial	Initial controller X rotation offset (default: 0)
vector	Clamp range for pitch as Vector2(min, max) (default: -40 to 40 degrees)

Definition at line 388 of file ActivityHelper.cs.

7.2.2.5 controllerRollControl()

```
void ActivityHelper.controllerRollControl (
            float speed,
            AxisRotationController manip,
            InputActionReference input,
            float deadZone,
            float initial = 0,
            Vector2 vector = default)  [static]
```

Processes VR controller input for aircraft roll control.

Reads controller Z-axis rotation and applies it to the aircraft's bank angle with deadzone filtering and speed scaling. Clamps the resulting roll to the specified range.

Parameters

speed	Maximum roll speed multiplier
manip	AxisRotationController to manipulate
input	InputActionReference for reading controller rotation
deadZone	Deadzone threshold below which input is ignored
initial	Initial controller Z rotation offset (default: 0)
vector	Clamp range for roll as Vector2(min, max) (default: -70 to 70 degrees)

Definition at line 354 of file ActivityHelper.cs.

7.2.2.6 DebugMouseControlPitch()

```
void ActivityHelper.DebugMouseControlPitch (
            float speed,
            AxisRotationController manip)  [static]
```

Debug function for mouse-based pitch control.

Allows testing pitch control using left and right mouse buttons. Left button pitches down (negative), right button pitches up (positive).

Parameters

speed	Pitch speed multiplier
manip	AxisRotationController to manipulate

Definition at line 296 of file ActivityHelper.cs.

7.2.2.7 DebugMouseControlPitchUp()

```
void ActivityHelper.DebugMouseControlPitchUp (
            float speed,
            AxisRotationController manip)  [static]
```

Debug function for mouse-based pitch up control.

Allows testing pitch up using only the left mouse button.

Parameters

speed	Pitch speed multiplier
manip	AxisRotationController to manipulate

Definition at line 333 of file ActivityHelper.cs.

7.2.2.8 DebugMouseControlRoll()

```
void ActivityHelper.DebugMouseControlRoll (
            float speed,
            AxisRotationController manip)  [static]
```

Debug function for mouse-based roll control.

Allows testing roll control using left and right mouse buttons. Left button rolls left (negative), right button rolls right (positive).

Parameters

speed	Roll speed multiplier
manip	AxisRotationController to manipulate

Definition at line 275 of file ActivityHelper.cs.

7.2.2.9 DebugMouseControlRollLeft()

```
void ActivityHelper.DebugMouseControlRollLeft (
            float speed,
            AxisRotationController manip)  [static]
```

Debug function for mouse-based left roll control.

Allows testing roll left using only the right mouse button.

Parameters

speed	Roll speed multiplier
manip	AxisRotationController to manipulate

Definition at line 316 of file ActivityHelper.cs.

7.2.2.10 FindPlayableDirector() [1/2]

```
PlayableDirector ActivityHelper.FindPlayableDirector (
            TimelineAsset asset)  [static]
```

Finds the PlayableDirector associated with a given TimelineAsset.

Searches all PlayableDirectors in the scene to find the one that uses the specified TimelineAsset. Logs a warning if the asset is null or no matching director is found.

Parameters

asset	The TimelineAsset to search for

Returns

The PlayableDirector using the asset, or null if not found

Definition at line 33 of file ActivityHelper.cs.

7.2.2.11 FindPlayableDirector() [2/2]

```
PlayableDirector[] ActivityHelper.FindPlayableDirector (
            TimelineAsset[] assets)  [static]
```

Finds PlayableDirectors for multiple TimelineAssets.

Searches all PlayableDirectors in the scene to find those that use the specified TimelineAssets. Returns an array with matching directors in corresponding indices. Null entries indicate no director was found for that asset.

Parameters

assets	Array of TimelineAssets to search for

Returns

Array of PlayableDirectors matching the input assets, or null if input is invalid

Definition at line 69 of file ActivityHelper.cs.

7.2.2.12 getAxisRotationController() [1/2]

```
AxisRotationController ActivityHelper.getAxisRotationController ()  [static]
```

Retrieves the AxisRotationController component from the scene.

Searches for and returns the first AxisRotationController component found in the scene. Logs an error if the component cannot be found.

Returns

The AxisRotationController component, or null if not found or error occurs

Definition at line 146 of file ActivityHelper.cs.

7.2.2.13 getAxisRotationController() [2/2]

```
AxisRotationController ActivityHelper.getAxisRotationController (
            float aoa = 0,
            float bank = 0)  [static]
```

Retrieves and configures the AxisRotationController with initial rotation values.

Searches for the AxisRotationController and sets its angle of attack and bank angle to the specified values using lerp functions.

Parameters

aoa	Angle of attack to set (default: 0)
bank	Bank angle to set (default: 0)

Returns

The configured AxisRotationController, or null if not found or error occurs

Definition at line 169 of file ActivityHelper.cs.

7.2.2.14 getBankGhostTrail()

```
BankGhostTrailBehaviour ActivityHelper.getBankGhostTrail (
            bool enable = true)  [static]
```

Retrieves and configures the BankGhostTrailBehaviour component.

Searches for the BankGhostTrailBehaviour component and sets its enable state.

Parameters

enable	Whether to enable the ghost trail (default: true)

Returns

The BankGhostTrailBehaviour component, or null if not found or error occurs

Definition at line 193 of file ActivityHelper.cs.

7.2.2.15 getNormalisedPlanePitch()

```
float ActivityHelper.getNormalisedPlanePitch (
            AxisRotationController manip)  [static]
```

Gets the aircraft's normalized pitch angle.

Extracts and normalizes the Z-axis rotation (pitch) from the AxisRotationController to a range of -180 to 180 degrees.

Parameters

manip	AxisRotationController to query

Returns

Normalized pitch angle in degrees, range [-180, 180]

Definition at line 506 of file ActivityHelper.cs.

7.2.2.16 getNormalisedPlaneRoll()

```
float ActivityHelper.getNormalisedPlaneRoll (
            AxisRotationController manip)  [static]
```

Gets the aircraft's normalized roll angle.

Extracts and normalizes the X-axis rotation (roll) from the AxisRotationController to a range of -180 to 180 degrees.

Parameters

manip	AxisRotationController to query

Returns

Normalized roll angle in degrees, range [-180, 180]

Definition at line 491 of file ActivityHelper.cs.

7.2.2.17 getProceduralAirflow()

```
ProceduralAirflow ActivityHelper.getProceduralAirflow ()  [static]
```

Retrieves the ProceduralAirflow component from the scene.

Searches for and returns the first ProceduralAirflow component found in the scene. Logs an error if the component cannot be found.

Returns

The ProceduralAirflow component, or null if not found or error occurs

Definition at line 125 of file ActivityHelper.cs.

7.2.2.18 getRotationSpeed()

```
float ActivityHelper.getRotationSpeed (
            float rotation,
            float speed,
            float deadZone)  [static]
```

Calculates rotation speed with deadzone compensation.

Scales the rotation speed proportionally to how far the input exceeds the deadzone. Preserves the sign of the original rotation.

Parameters

rotation	Input rotation value
speed	Speed multiplier
deadZone	Deadzone threshold

Returns

Scaled rotation speed with sign preserved

Definition at line 433 of file ActivityHelper.cs.

7.2.2.19 normalizedControllerXRotation()

```
float ActivityHelper.normalizedControllerXRotation (
            InputActionReference input,
            float initial = 0)  [static]
```

Returns normalized X-axis rotation from VR controller input.

Reads the controller rotation quaternion and calculates the X-axis rotation relative to an initial rotation value. The result is normalized between -180 and 180 degrees using Mathf.DeltaAngle.

Parameters

input	InputActionReference for reading controller rotation
initial	Initial X rotation offset in degrees (default: 0)

Returns

Normalized X rotation in degrees, range [-180, 180]

Definition at line 248 of file ActivityHelper.cs.

7.2.2.20 normalizedControllerZRotation()

```
float ActivityHelper.normalizedControllerZRotation (
            InputActionReference input,
            float initial = 0)  [static]
```

Returns normalized Z-axis rotation from VR controller input.

Reads the controller rotation quaternion and calculates the Z-axis rotation relative to an initial rotation value. The result is normalized between -180 and 180 degrees using Mathf.DeltaAngle.

Parameters

input	InputActionReference for reading controller rotation
initial	Initial Z rotation offset in degrees (default: 0)

Returns

Normalized Z rotation in degrees, range [-180, 180]

Definition at line 219 of file ActivityHelper.cs.

7.2.2.21 PlayTimeLine()

```
IEnumerator ActivityHelper.PlayTimeLine (
            PlayableDirector director,
            System.Action onComplete,
            float fallBackWait = 1f)  [static]
```

Coroutine to play a Unity Timeline and invoke callback on completion.

Plays the specified PlayableDirector and waits until the timeline completes before invoking the provided callback action. If the director or asset is null, waits for the fallback duration before invoking the callback.

Parameters

director	PlayableDirector containing the timeline to play

onComplete	Callback action to invoke when timeline completes
fallBackWait	Fallback wait time in seconds if director is invalid (default: 1s)

Returns

IEnumerator for coroutine execution

Definition at line 524 of file ActivityHelper.cs.

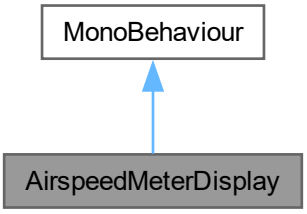
The documentation for this class was generated from the following file:

- Assets/Scripts/ActivityStructure/ActivityHelper.cs

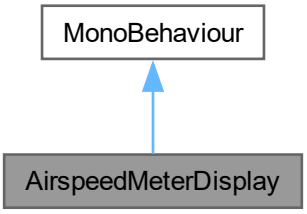
7.3 AirspeedMeterDisplay Class Reference

A dynamic airspeed meter that is displayed on the UI.

Inheritance diagram for AirspeedMeterDisplay:



Collaboration diagram for AirspeedMeterDisplay:



Public Attributes

- TextMeshProUGUI AirspeedMeter

  The airspeed meter GameObject.
- float knots = 0

  The current airspeed value.
- float targetValue

  The target airspeed, used in interpolation.
- bool isMoving

  Flag that determines if airspeed interpolation is currently occurring.

Private Member Functions

- void Start ()
- void Update ()
- void setKnots (float newKnots)

  Sets the airspeed to a new value immediately, without any smoothing/interpolation.
- void moveTowardsKnotsTarget (float target, float speed)

  Special mutator for the airspeed that makes use of Update() to gradually interpolate towards a target value at a set speed.

Private Attributes

- float moveSpeed

  The speed of airspeed interpolation.

7.3.1 Detailed Description

A dynamic airspeed meter that is displayed on the UI.

Author

Caleb Martin | marcy066@mymail.unisa.edu.au

Definition at line 10 of file AirspeedMeterDisplay.cs.

7.3.2 Member Function Documentation

7.3.2.1 moveTowardsKnotsTarget()

```
void AirspeedMeterDisplay.moveTowardsKnotsTarget (
            float target,
            float speed)  [private]
```

Special mutator for the airspeed that makes use of Update() to gradually interpolate towards a target value at a set speed.

Parameters

target	The new value for the airspeed to be set to.
speed	The speed at which airspeed interpolation should occur.

Returns

void

Definition at line 59 of file AirspeedMeterDisplay.cs.

7.3.2.2 setKnots()

```
void AirspeedMeterDisplay.setKnots (
            float newKnots)  [private]
```

Sets the airspeed to a new value immediately, without any smoothing/interpolation.

Parameters

newKnots	The new value for the airspeed to be set to.

Returns

void

Definition at line 46 of file AirspeedMeterDisplay.cs.

7.3.2.3 Start()

```
void AirspeedMeterDisplay.Start ()  [private]
```

Definition at line 18 of file AirspeedMeterDisplay.cs.

7.3.2.4 Update()

```
void AirspeedMeterDisplay.Update ()  [private]
```

Definition at line 24 of file AirspeedMeterDisplay.cs.

7.3.3 Member Data Documentation

7.3.3.1 AirspeedMeter

```
TextMeshProUGUI AirspeedMeterDisplay.AirspeedMeter
```

The airspeed meter GameObject.

Definition at line 12 of file AirspeedMeterDisplay.cs.

7.3.3.2 isMoving

```
bool AirspeedMeterDisplay.isMoving
```

Flag that determines if airspeed interpolation is currently occurring.

Definition at line 16 of file AirspeedMeterDisplay.cs.

7.3.3.3 knots

```
float AirspeedMeterDisplay.knots = 0
```

The current airspeed value.

Definition at line 13 of file AirspeedMeterDisplay.cs.

7.3.3.4 moveSpeed

```
float AirspeedMeterDisplay.moveSpeed  [private]
```

The speed of airspeed interpolation.

Definition at line 15 of file AirspeedMeterDisplay.cs.

7.3.3.5 targetValue

```
float AirspeedMeterDisplay.targetValue
```

The target airspeed, used in interpolation.

Definition at line 14 of file AirspeedMeterDisplay.cs.

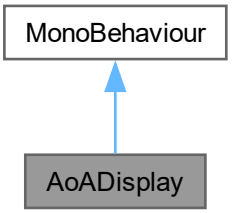
The documentation for this class was generated from the following file:

- Assets/Scripts/UI/AirspeedMeterDisplay.cs

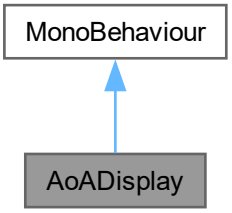
7.4 AoADisplay Class Reference

A dynamic Angle of Attack meter that is displayed on the UI, changing colour dynamically based on how close to critical angle it is.

Inheritance diagram for AoADisplay:



Collaboration diagram for AoADisplay:



Public Attributes

- TextMeshProUGUI AoAMeter

  The Angle of Attack meter GameObject.
- GameObject aircraft

  Reference to the DA-40 aircraft GameObject.

Private Member Functions

- void Start ()
- void Update ()

7.4.1 Detailed Description

A dynamic Angle of Attack meter that is displayed on the UI, changing colour dynamically based on how close to critical angle it is.

Author

Caleb Martin | marcy066@mymail.unisa.edu.au

Definition at line 10 of file AoADisplay.cs.

7.4.2 Member Function Documentation

7.4.2.1 Start()

```
void AoADisplay.Start ()  [private]
```

Definition at line 15 of file AoADisplay.cs.

7.4.2.2 Update()

```
void AoADisplay.Update ()  [private]
```

Definition at line 20 of file AoADisplay.cs.

7.4.3 Member Data Documentation

7.4.3.1 aircraft

```
GameObject AoADisplay.aircraft
```

Reference to the DA-40 aircraft GameObject.

Definition at line 13 of file AoADisplay.cs.

7.4.3.2 AoAMeter

```
TextMeshProUGUI AoADisplay.AoAMeter
```

The Angle of Attack meter GameObject.

Definition at line 12 of file AoADisplay.cs.

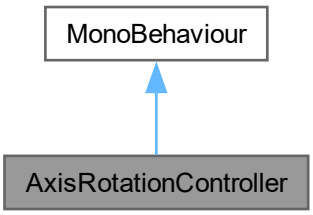
The documentation for this class was generated from the following file:

- Assets/Scripts/UI/AoADisplay.cs

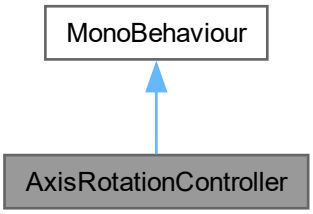
7.5 AxisRotationController Class Reference

Helper class to perform rotation actions.

Inheritance diagram for AxisRotationController:



Collaboration diagram for AxisRotationController:



Public Member Functions

- void LerpAOA (float angle, float lerpDuration=1.5f)

  Lerp between initial and target angle.
- void LerpBank (float angle, float lerpDuration=1.5f)

  Lerp between initial and target angle.
- void IncrementBank (float delta)

  Increment x rotaiton.
- void IncrementAOA (float delta)

  Increment z rotaiton.
- void IncrementYaw (float delta)

  Increment y rotation (yaw).
- Quaternion getRotation ()

  Returns parent Quaternion rotation.
- void setRoll (float roll)

Set x rotation of parent GameObject.

- void setPitch (float pitch)

  Set x rotation of parent GameObject.
- bool IsLerpingRoll ()

  Check if Plane GameObject is lerping roll.
- bool IsLerpingPitch ()

  Check if Plane GameObject is lerping pitch.

Private Member Functions

- void Update ()

  Main update loop.
- void UpdateRoll ()

  Updates Roll according to lerp parameters.
- void UpdatePitch ()

  Updates Pitch according to lerp parameters.

Private Attributes

- bool isLerpingPitch

  Current state of pitch lerp.
- bool isLerpingRoll

  Current state of roll lerp.
- float startRotationZ

  Start for pitch lerp.
- float targetRotationZ

  Target for for pitch lerp.
- float startRotationX

  Start for roll lerp.
- float targetRotationX

  Target for roll lerp.
- float durationRoll

  Duration of lerp in seconds.
- float durationPitch

  Duration of lerp in seconds.
- float timeElapsedRoll

  Current time elapsed during roll lerp.
- float timeElapsedPitch

  Current time elapsed during pitch lerp.

7.5.1 Detailed Description

Helper class to perform rotation actions.

Provides functions for performing different rotation actions on a game object. Only performs roll and pitch rotations for a game object with a +x forward axis and +z left side axis.

Author

Connor Freebairn | frecd002@mymail.unisa.edu.au

Definition at line 13 of file AxisRotationController.cs.

7.5.2 Member Function Documentation

7.5.2.1 getRotation()

```
Quaternion AxisRotationController.getRotation ()
```

Returns parent Quaternion rotation.

Returns

Quaternion

Definition at line 126 of file AxisRotationController.cs.

7.5.2.2 IncrementAOA()

```
void AxisRotationController.IncrementAOA (
            float delta)
```

Increment z rotaiton.

Adds parameter delta to parent GameObjects z rotation. Usually scaled according to deltatime.

Parameters

delta	small -/+ value for rotation

Definition at line 85 of file AxisRotationController.cs.

7.5.2.3 IncrementBank()

```
void AxisRotationController.IncrementBank (
            float delta)
```

Increment x rotaiton.

Adds parameter delta to parent GameObjects x rotation. Usually scaled according to deltatime.

Parameters

delta	small -/+ value for rotation

Definition at line 72 of file AxisRotationController.cs.

7.5.2.4 IncrementYaw()

```
void AxisRotationController.IncrementYaw (
            float delta)
```

Increment y rotation (yaw).

Adds parameter delta to the parent GameObjects y rotation. Usually scaled according to deltaTime.

Parameters

delta	small -/+ value for rotation

Definition at line 98 of file AxisRotationController.cs.

7.5.2.5 IsLerpingPitch()

```
bool AxisRotationController.IsLerpingPitch ()
```

Check if Plane GameObject is lerping pitch.

Returns

boolean for lerping state

Definition at line 166 of file AxisRotationController.cs.

7.5.2.6 IsLerpingRoll()

```
bool AxisRotationController.IsLerpingRoll ()
```

Check if Plane GameObject is lerping roll.

Returns

boolean for lerping state

Definition at line 156 of file AxisRotationController.cs.

7.5.2.7 LerpAOA()

```
void AxisRotationController.LerpAOA (
            float angle,
            float lerpDuration = 1::5f)
```

Lerp between initial and target angle.

Lerp from the current rotation of the parent GameObject to target angle around z axis.

Parameters

angle	target angle for lerp
lerpDuration	duration of time to lerp to target

Definition at line 37 of file AxisRotationController.cs.

7.5.2.8 LerpBank()

```
void AxisRotationController.LerpBank (
            float angle,
            float lerpDuration = 1::5f)
```

Lerp between initial and target angle.

Lerp from the current rotation of the parent GameObject to target angle around x axis.

Parameters

angle	target angle for lerp
lerpDuration	duration of time to lerp to target

Definition at line 55 of file AxisRotationController.cs.

7.5.2.9 setPitch()

```
void AxisRotationController.setPitch (
            float pitch)
```

Set x rotation of parent GameObject.

Parameters

pitch	rotation value as a float

Definition at line 146 of file AxisRotationController.cs.

7.5.2.10 setRoll()

```
void AxisRotationController.setRoll (
            float roll)
```

Set x rotation of parent GameObject.

Parameters

roll	rotation value as a float

Definition at line 136 of file AxisRotationController.cs.

7.5.2.11 Update()

```
void AxisRotationController.Update ()  [private]
```

Main update loop.

Checks if pitch or roll needs to continue lerping and calls UpdatePitch() and UpdateRoll() respectivley

Definition at line 108 of file AxisRotationController.cs.

7.5.2.12 UpdatePitch()

```
void AxisRotationController.UpdatePitch ()  [private]
```

Updates Pitch according to lerp parameters.

Called by main Update() loop to perform pitch lerp

Definition at line 197 of file AxisRotationController.cs.

7.5.2.13 UpdateRoll()

```
void AxisRotationController.UpdateRoll ()  [private]
```

Updates Roll according to lerp parameters.

Called by main Update() loop to perform roll lerp

Definition at line 176 of file AxisRotationController.cs.

7.5.3 Member Data Documentation

7.5.3.1 durationPitch

```
float AxisRotationController.durationPitch  [private]
```

Duration of lerp in seconds.

Definition at line 25 of file AxisRotationController.cs.

7.5.3.2 durationRoll

```
float AxisRotationController.durationRoll  [private]
```

Duration of lerp in seconds.

Definition at line 24 of file AxisRotationController.cs.

7.5.3.3 isLerpingPitch

```
bool AxisRotationController.isLerpingPitch  [private]
```

Current state of pitch lerp.

Definition at line 15 of file AxisRotationController.cs.

7.5.3.4 isLerpingRoll

```
bool AxisRotationController.isLerpingRoll  [private]
```

Current state of roll lerp.

Definition at line 16 of file AxisRotationController.cs.

7.5.3.5 startRotationX

```
float AxisRotationController.startRotationX  [private]
```

Start for roll lerp.

Definition at line 21 of file AxisRotationController.cs.

7.5.3.6 startRotationZ

```
float AxisRotationController.startRotationZ  [private]
```

Start for pitch lerp.

Definition at line 18 of file AxisRotationController.cs.

7.5.3.7 targetRotationX

```
float AxisRotationController.targetRotationX  [private]
```

Target for roll lerp.

Definition at line 22 of file AxisRotationController.cs.

7.5.3.8 targetRotationZ

```
float AxisRotationController.targetRotationZ  [private]
```

Target for for pitch lerp.

Definition at line 19 of file AxisRotationController.cs.

7.5.3.9 timeElapsedPitch

```
float AxisRotationController.timeElapsedPitch  [private]
```

Current time elapsed during pitch lerp.

Definition at line 27 of file AxisRotationController.cs.

7.5.3.10 timeElapsedRoll

```
float AxisRotationController.timeElapsedRoll  [private]
```

Current time elapsed during roll lerp.

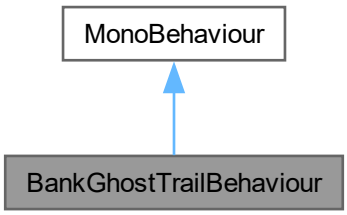
Definition at line 26 of file AxisRotationController.cs.

The documentation for this class was generated from the following file:

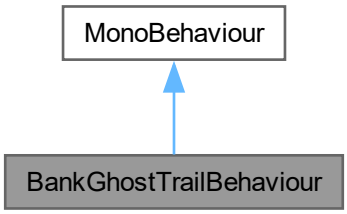
- Assets/Scripts/MiscBehaviour/AxisRotationController.cs

7.6 BankGhostTrailBehaviour Class Reference

Inheritance diagram for BankGhostTrailBehaviour:



Collaboration diagram for BankGhostTrailBehaviour:



Public Attributes

- bool enable = true

  Enable banking behaviour.
- ParticleSystem ghostTrail

  Ghost trail particle system.
- float velocity = 50f

  Velocity of the particles.

Private Member Functions

- void Update ()

  Main update loop.
- void UpdateGhostTrailMotion ()

  Determines and updates behaviour of particles.
- void OnDrawGizmosSelected ()

  Draw Gizmo Sphere representing turn radius.

Private Attributes

- ParticleSystem.Particle[ ] ghostParticles

  List of particles emitted by the particle system.

7.6.1 Detailed Description

Definition at line 3 of file BankGhostTrailBehaviour.cs.

7.6.2 Member Function Documentation

7.6.2.1 OnDrawGizmosSelected()

```
void BankGhostTrailBehaviour.OnDrawGizmosSelected ()  [private]
```

Draw Gizmo Sphere representing turn radius.

Sphere represents the calculated radius of the plane and is rendered when GameObject is selected

Definition at line 137 of file BankGhostTrailBehaviour.cs.

7.6.2.2 Update()

```
void BankGhostTrailBehaviour.Update ()  [private]
```

Main update loop.

Calls UpdateGhostTrailMotion() if enable flag set to true

Definition at line 18 of file BankGhostTrailBehaviour.cs.

7.6.2.3 UpdateGhostTrailMotion()

```
void BankGhostTrailBehaviour.UpdateGhostTrailMotion ()  [private]
```

Determines and updates behaviour of particles.

Creates a local copy of the assigned Particle System's particle buffer and modifies the tranform of particles. Particles follow the calculated radius and the logic is functional during level, ascending and descending flight.

Formula for calculating turning radius

$$R = \frac{V^2}{g \times \tan(\theta)}$$

where:

- R = Radius (m)
- V = Velocity (m/s)
- g = Gravity (m/s^2)
- θ = Bank angle (degrees)

Particles transforms are modified and buffer is returned to the Particle system.

Definition at line 44 of file BankGhostTrailBehaviour.cs.

7.6.3 Member Data Documentation

7.6.3.1 enable

```
bool BankGhostTrailBehaviour.enable = true
```

Enable banking behaviour.

Definition at line 6 of file BankGhostTrailBehaviour.cs.

7.6.3.2 ghostParticles

```
ParticleSystem.Particle [] BankGhostTrailBehaviour.ghostParticles  [private]
```

List of particles emitted by the particle system.

Definition at line 11 of file BankGhostTrailBehaviour.cs.

7.6.3.3 ghostTrail

```
ParticleSystem BankGhostTrailBehaviour.ghostTrail
```

Ghost trail particle system.

Definition at line 8 of file BankGhostTrailBehaviour.cs.

7.6.3.4 velocity

```
float BankGhostTrailBehaviour.velocity = 50f
```

Velocity of the particles.

Definition at line 10 of file BankGhostTrailBehaviour.cs.

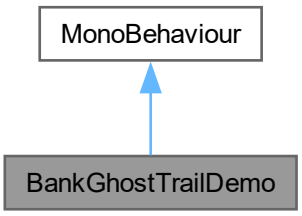
The documentation for this class was generated from the following file:

- Assets/Scripts/DA_40_SpecialFeatures/BankGhostTrailBehaviour.cs

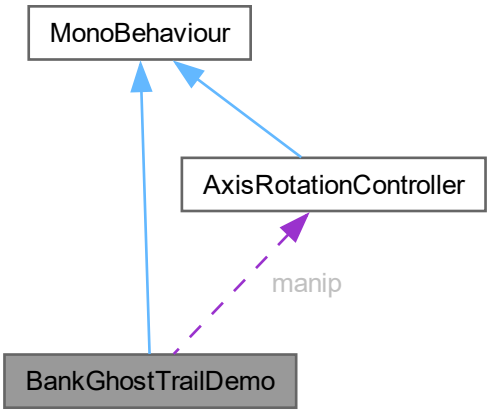
7.7 BankGhostTrailDemo Class Reference

A test comment.

Inheritance diagram for BankGhostTrailDemo:



Collaboration diagram for BankGhostTrailDemo:



Public Attributes

- AxisRotationController manip

  cool little test comment

Private Member Functions

- void Start ()
- void Update ()

  The update loop.

Private Attributes

- float delay = 0f

  the delay.
- float delayTarget = 5f

  The delay target.
- bool started = false

7.7.1 Detailed Description

A test comment.

A more bigger better comment

Author

Author Name (for classes/interfaces).

Definition at line 7 of file BankGhostTrailDemo.cs.

7.7.2 Member Function Documentation

7.7.2.1 Start()

```
void BankGhostTrailDemo.Start ()  [private]
```

Definition at line 15 of file BankGhostTrailDemo.cs.

7.7.2.2 Update()

```
void BankGhostTrailDemo.Update ()  [private]
```

The update loop.

Updates every frame because <3. and this is an extra line

Parameters

nothing	this is literally nothing
this	doesnt exist, literall the param this, does. not. exist.

Returns

no why would you think this returns something bozo

Definition at line 29 of file BankGhostTrailDemo.cs.

7.7.3 Member Data Documentation

7.7.3.1 delay

```
float BankGhostTrailDemo.delay = 0f  [private]
```

the delay.

the full stop breaks to comment in half

Definition at line 12 of file BankGhostTrailDemo.cs.

7.7.3.2 delayTarget

```
float BankGhostTrailDemo.delayTarget = 5f  [private]
```

The delay target.

Definition at line 13 of file BankGhostTrailDemo.cs.

7.7.3.3 manip

```
AxisRotationController BankGhostTrailDemo.manip
```

cool little test comment

Definition at line 10 of file BankGhostTrailDemo.cs.

7.7.3.4 started

```
bool BankGhostTrailDemo.started = false  [private]
```

Definition at line 14 of file BankGhostTrailDemo.cs.

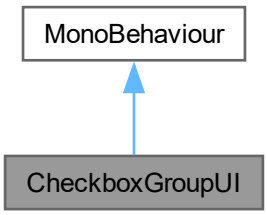
The documentation for this class was generated from the following file:

- Assets/Scripts/Misc/BankGhostTrailDemo.cs

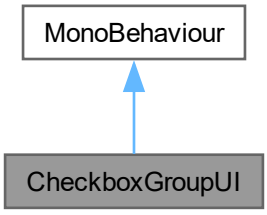
7.8 CheckboxGroupUI Class Reference

A group of UI checkboxes used in activity guidance.

Inheritance diagram for CheckboxGroupUI:



Collaboration diagram for CheckboxGroupUI:



Public Member Functions

- void SetChecked (bool isChecked)

  Sets the checkbox image active or not.
- void SetLabel (string newText)

  Sets the checkbox label text.
- void SetVisible (bool visible)

Public Attributes

- GameObject groupObject

  The checkbox group.
- TextMeshProUGUI labelText

  The label for the group of checkboxes.
- Image checkImage

  The checkmark image.

7.8.1 Detailed Description

A group of UI checkboxes used in activity guidance.

Author

Caleb Martin | marcy066@mymail.unisa.edu.au

Definition at line 11 of file CheckboxGroupUI.cs.

7.8.2 Member Function Documentation

7.8.2.1 SetChecked()

```
void CheckboxGroupUI.SetChecked (
            bool isChecked)
```

Sets the checkbox image active or not.

Parameters

isChecked	The state for the checkbox to be set to.

Returns

void

Definition at line 22 of file CheckboxGroupUI.cs.

7.8.2.2 SetLabel()

```
void CheckboxGroupUI.SetLabel (
            string newText)
```

Sets the checkbox label text.

Parameters

newText	The text for the checkbox label to be set to.

Returns

void

Definition at line 32 of file CheckboxGroupUI.cs.

7.8.2.3 SetVisible()

```
void CheckboxGroupUI.SetVisible (
            bool visible)
```

Definition at line 38 of file CheckboxGroupUI.cs.

7.8.3 Member Data Documentation

7.8.3.1 checkImage

```
Image CheckboxGroupUI.checkImage
```

The checkmark image.

Definition at line 15 of file CheckboxGroupUI.cs.

7.8.3.2 groupObject

```
GameObject CheckboxGroupUI.groupObject
```

The checkbox group.

Definition at line 13 of file CheckboxGroupUI.cs.

7.8.3.3 labelText

```
TextMeshProUGUI CheckboxGroupUI.labelText
```

The label for the group of checkboxes.

Definition at line 14 of file CheckboxGroupUI.cs.

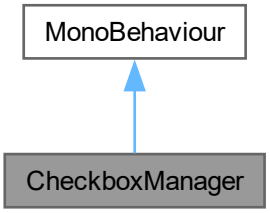
The documentation for this class was generated from the following file:

- Assets/Scripts/UI/CheckboxGroupUI.cs

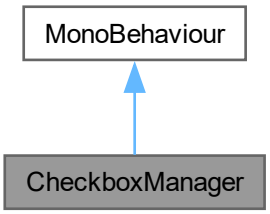
7.9 CheckboxManager Class Reference

A central class to manage checkboxes in tandem with activities to support the user.

Inheritance diagram for CheckboxManager:



Collaboration diagram for CheckboxManager:



Public Member Functions

- void SetupSteps (string[] instructions)

  Called by the ModuleActivityScheduler, sets up a number of checkboxes and their descriptions based on activity content.
- void HighlightStep (int stepIndex)

  Highlights the current step as a different colour, making it clear what the user needs to perform next.
- void CompleteStep (int stepIndex)

  Update the UI to reflect each completed step.

Private Attributes

- List< CheckboxGroupUI > checkboxGroups = new List<CheckboxGroupUI>()

  Collection of all of the CheckboxGroupUI instances.
- int totalSteps = 0

  The number of steps in the current activity.

7.9.1 Detailed Description

A central class to manage checkboxes in tandem with activities to support the user.

Author

Caleb Martin | marcy066@mymail.unisa.edu.au

Definition at line 12 of file CheckboxManager.cs.

7.9.2 Member Function Documentation

7.9.2.1 CompleteStep()

```
void CheckboxManager.CompleteStep (
            int stepIndex)
```

Update the UI to reflect each completed step.

Parameters

stepIndex	The step that needs to be marked as complete.

Returns

void

Definition at line 69 of file CheckboxManager.cs.

7.9.2.2 HighlightStep()

```
void CheckboxManager.HighlightStep (
            int stepIndex)
```

Highlights the current step as a different colour, making it clear what the user needs to perform next.

Parameters

stepIndex	The step that needs to be highlighted.

Returns

void

Definition at line 51 of file CheckboxManager.cs.

7.9.2.3 SetupSteps()

```
void CheckboxManager.SetupSteps (
            string[] instructions)
```

Called by the ModuleActivityScheduler, sets up a number of checkboxes and their descriptions based on activity content.

Parameters

instructions	The series of instructions that each checkbox group's descriptions will be sequentially set to.

Returns

void

Definition at line 23 of file CheckboxManager.cs.

7.9.3 Member Data Documentation

7.9.3.1 checkboxGroups

```
List<CheckboxGroupUI> CheckboxManager.checkboxGroups = new List<CheckboxGroupUI>()  [private]
```

Collection of all of the CheckboxGroupUI instances.

Definition at line 14 of file CheckboxManager.cs.

7.9.3.2 totalSteps

```
int CheckboxManager.totalSteps = 0  [private]
```

The number of steps in the current activity.

Definition at line 16 of file CheckboxManager.cs.

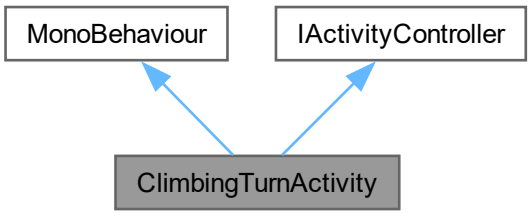
The documentation for this class was generated from the following file:

- Assets/Scripts/UI/CheckboxManager.cs

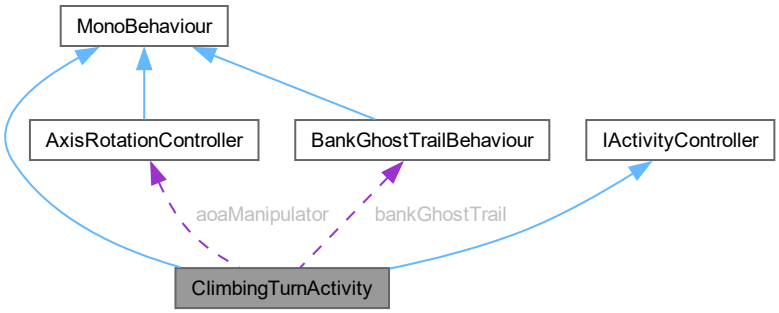
7.10 ClimbingTurnActivity Class Reference

Climbing turn training activity controller.

Inheritance diagram for ClimbingTurnActivity:



Collaboration diagram for ClimbingTurnActivity:



Public Member Functions

- void OnEnable ()

  Initialize activity on component enable.
- void OnDisable ()

  Cleanup activity on component disable.
- void StartActivity ()

  Start the activity when triggered by ModuleActivityScheduler.
- void EnableCurrentStep ()

  Unlock input for the current training step.
- void HandleSlipStream ()

  Handle automated slipstream demonstration step.
- void HandleBanking ()

  Handle banking during climb training step.
- void HandleStall ()

  Handle stall induction training step.

Public Attributes

- TimelineAsset postSlipStream

  Timeline demonstrating slipstream effects during climb.
- TimelineAsset postBank

  Timeline played after achieving 15 bank target.
- TimelineAsset postStall

  Timeline demonstrating stall during climbing turn.
- InputActionReference rotationInput

  VR controller rotation input action for aircraft control.
- float bankSensitivity = 45f

  Roll/pitch sensitivity multiplier for controller input.
- float deadZone = 0.1f

  Controller deadzone threshold in degrees to filter noise.
- float stepAdvanceTolerance = 3f

  Tolerance in degrees for achieving target angles before advancing.
- bool useMouseClicks = true

  Enable mouse button control for testing without VR.
- float mouseRollSpeed = 10f

  Speed multiplier for mouse-based roll/pitch control.

Properties

- ModuleActivityScheduler mas `[get]`

  Reference to singleton activity scheduler.

Private Types

- enum TurnStep { SlipStream , Bank , Stall , Complete }

  Training progression states.

Private Member Functions

- void Update ()

  Main update loop.

Private Attributes

- bool rollInputLocked = false

  Flag to disable roll input during transitions and timelines.
- bool pitchInputLocked = false

  Flag to disable pitch input during transitions and timelines.
- TurnStep currentStep

  Current training step in the activity sequence.
- float currentBank = 0f

  Current aircraft bank angle in degrees, updated each frame.
- Quaternion controllerOrigin = Quaternion.identity

  Initial controller orientation captured at step start.
- bool hasControllerOrigin = false

  Flag indicating if controller origin has been captured this step.
- bool hasNotifiedMAS = false

  Failsafe flag to prevent multiple completion notifications to scheduler.
- AxisRotationController aoaManipulator

  Controller for aircraft pitch and roll manipulation.
- BankGhostTrailBehaviour bankGhostTrail

  Visual trail effect showing aircraft bank history.
- PlayableDirector postSlipStreamDirector

  PlayableDirector for slipstream demonstration timeline.
- PlayableDirector postBankDirector

  PlayableDirector for post-bank timeline.
- PlayableDirector postStallDirector

  PlayableDirector for stall demonstration timeline.

7.10.1 Detailed Description

Climbing turn training activity controller.

Manages a climbing turn exercise where users practice banking while maintaining a climbing attitude, followed by an aggressive pitch to induce stall. Demonstrates the relationship between climbing turns, slipstream effects, and stall conditions. Implements the IActivityController interface for integration with the ModuleActivityScheduler system.

Training sequence:

1. SlipStream - Automated demonstration of slipstream effects during climb
2. Bank - User achieves 15° bank while maintaining 10° climb attitude
3. Stall - User increases pitch to 25° to induce stall condition
4. Complete - Activity finished, notifies scheduler

Author

Connor Freebairn | frecd002@mymail.unisa.edu.au

Definition at line 22 of file ClimbingTurnActivity.cs.

7.10.2 Member Enumeration Documentation

7.10.2.1 TurnStep

```
enum ClimbingTurnActivity.TurnStep  [private]
```

Training progression states.

Defines the sequence of training steps in the climbing turn activity.

Enumerator

SlipStream	Automated slipstream demonstration during initial climb.
Bank	User performs banking while maintaining climb.
Stall	User increases pitch to induce stall condition.
Complete	Activity finished, awaiting cleanup.

Definition at line 48 of file ClimbingTurnActivity.cs.

7.10.3 Member Function Documentation

7.10.3.1 EnableCurrentStep()

```
void ClimbingTurnActivity.EnableCurrentStep ()
```

Unlock input for the current training step.

Resets roll and pitch input locks and controller origin flag to allow user control. Called after transitions and timeline completions.

Definition at line 136 of file ClimbingTurnActivity.cs.

7.10.3.2 HandleBanking()

```
void ClimbingTurnActivity.HandleBanking ()
```

Handle banking during climb training step.

Captures controller origin on first frame, reads current bank angle, and checks if user has achieved 15 bank target while maintaining the 10° climb attitude. When target is reached, locks input, plays confirmation timeline, and advances to stall demonstration. Allows controller or debug mouse input when target not yet achieved.

Definition at line 204 of file ClimbingTurnActivity.cs.

7.10.3.3 HandleSlipStream()

```
void ClimbingTurnActivity.HandleSlipStream ()
```

Handle automated slipstream demonstration step.

Locks input and immediately plays the slipstream demonstration timeline on first call. Advances to banking step after timeline completes. This is a passive demonstration step with no user input required.

Todo Slipstream animation/effect needs implementation

Definition at line 182 of file ClimbingTurnActivity.cs.

7.10.3.4 HandleStall()

```
void ClimbingTurnActivity.HandleStall ()
```

Handle stall induction training step.

Captures controller origin on first frame, reads current pitch angle, and checks if user has achieved 25 pitch target to induce stall during the climbing turn. Uses absolute value comparison (checkRotationTargetAchieved) allowing either positive or negative 25 pitch. When target is reached, locks input, plays stall demonstration timeline, and advances to completion. Allows controller or debug mouse input when target not yet achieved.

Definition at line 255 of file ClimbingTurnActivity.cs.

7.10.3.5 OnDisable()

```
void ClimbingTurnActivity.OnDisable ()
```

Cleanup activity on component disable.

Disables input actions and turns off visual trail effects.

Definition at line 110 of file ClimbingTurnActivity.cs.

7.10.3.6 OnEnable()

```
void ClimbingTurnActivity.OnEnable ()
```

Initialize activity on component enable.

Enables input actions, resets progression state to initial values, retrieves references to aircraft controllers and visual effects, and finds PlayableDirectors for all timeline assets. Initializes aircraft to 10 pitch (climbing attitude).

Definition at line 82 of file ClimbingTurnActivity.cs.

7.10.3.7 StartActivity()

```
void ClimbingTurnActivity.StartActivity ()
```

Start the activity when triggered by ModuleActivityScheduler.

Called by the activity scheduler to begin the training sequence. Unlocks input for the first step.

Implements IActivityController.

Definition at line 125 of file ClimbingTurnActivity.cs.

7.10.3.8 Update()

```
void ClimbingTurnActivity.Update ()  [private]
```

Main update loop.

Routes execution to appropriate handler based on current training step. Handles completion notification with failsafe to prevent duplicate calls.

Definition at line 149 of file ClimbingTurnActivity.cs.

7.10.4 Member Data Documentation

7.10.4.1 aoaManipulator

```
AxisRotationController ClimbingTurnActivity.aoaManipulator  [private]
```

Controller for aircraft pitch and roll manipulation.

Definition at line 67 of file ClimbingTurnActivity.cs.

7.10.4.2 bankGhostTrail

```
BankGhostTrailBehaviour ClimbingTurnActivity.bankGhostTrail  [private]
```

Visual trail effect showing aircraft bank history.

Definition at line 68 of file ClimbingTurnActivity.cs.

7.10.4.3 bankSensitivity

```
float ClimbingTurnActivity.bankSensitivity = 45f
```

Roll/pitch sensitivity multiplier for controller input.

Definition at line 35 of file ClimbingTurnActivity.cs.

7.10.4.4 controllerOrigin

```
Quaternion ClimbingTurnActivity.controllerOrigin = Quaternion.identity  [private]
```

Initial controller orientation captured at step start.

Definition at line 61 of file ClimbingTurnActivity.cs.

7.10.4.5 currentBank

```
float ClimbingTurnActivity.currentBank = 0f  [private]
```

Current aircraft bank angle in degrees, updated each frame.

Definition at line 59 of file ClimbingTurnActivity.cs.

7.10.4.6 currentStep

```
TurnStep ClimbingTurnActivity.currentStep  [private]
```

Current training step in the activity sequence.

Definition at line 58 of file ClimbingTurnActivity.cs.

7.10.4.7 deadZone

```
float ClimbingTurnActivity.deadZone = 0.1f
```

Controller deadzone threshold in degrees to filter noise.

Definition at line 36 of file ClimbingTurnActivity.cs.

7.10.4.8 hasControllerOrigin

```
bool ClimbingTurnActivity.hasControllerOrigin = false  [private]
```

Flag indicating if controller origin has been captured this step.

Definition at line 62 of file ClimbingTurnActivity.cs.

7.10.4.9 hasNotifiedMAS

```
bool ClimbingTurnActivity.hasNotifiedMAS = false  [private]
```

Failsafe flag to prevent multiple completion notifications to scheduler.

Definition at line 64 of file ClimbingTurnActivity.cs.

7.10.4.10 mouseRollSpeed

```
float ClimbingTurnActivity.mouseRollSpeed = 10f
```

Speed multiplier for mouse-based roll/pitch control.

Definition at line 41 of file ClimbingTurnActivity.cs.

7.10.4.11 pitchInputLocked

```
bool ClimbingTurnActivity.pitchInputLocked = false  [private]
```

Flag to disable pitch input during transitions and timelines.

Definition at line 57 of file ClimbingTurnActivity.cs.

7.10.4.12 postBank

```
TimelineAsset ClimbingTurnActivity.postBank
```

Timeline played after achieving 15 bank target.

Definition at line 28 of file ClimbingTurnActivity.cs.

7.10.4.13 postBankDirector

```
PlayableDirector ClimbingTurnActivity.postBankDirector  [private]
```

PlayableDirector for post-bank timeline.

Definition at line 72 of file ClimbingTurnActivity.cs.

7.10.4.14 postSlipStream

```
TimelineAsset ClimbingTurnActivity.postSlipStream
```

Timeline demonstrating slipstream effects during climb.

Definition at line 27 of file ClimbingTurnActivity.cs.

7.10.4.15 postSlipStreamDirector

```
PlayableDirector ClimbingTurnActivity.postSlipStreamDirector  [private]
```

PlayableDirector for slipstream demonstration timeline.

Definition at line 71 of file ClimbingTurnActivity.cs.

7.10.4.16 postStall

```
TimelineAsset ClimbingTurnActivity.postStall
```

Timeline demonstrating stall during climbing turn.

Definition at line 29 of file ClimbingTurnActivity.cs.

7.10.4.17 postStallDirector

```
PlayableDirector ClimbingTurnActivity.postStallDirector  [private]
```

PlayableDirector for stall demonstration timeline.

Definition at line 73 of file ClimbingTurnActivity.cs.

7.10.4.18 rollInputLocked

```
bool ClimbingTurnActivity.rollInputLocked = false  [private]
```

Flag to disable roll input during transitions and timelines.

Definition at line 56 of file ClimbingTurnActivity.cs.

7.10.4.19 rotationInput

```
InputActionReference ClimbingTurnActivity.rotationInput
```

VR controller rotation input action for aircraft control.

Definition at line 32 of file ClimbingTurnActivity.cs.

7.10.4.20 stepAdvanceTolerance

```
float ClimbingTurnActivity.stepAdvanceTolerance = 3f
```

Tolerance in degrees for achieving target angles before advancing.

Definition at line 37 of file ClimbingTurnActivity.cs.

7.10.4.21 useMouseClicks

```
bool ClimbingTurnActivity.useMouseClicks = true
```

Enable mouse button control for testing without VR.

Definition at line 40 of file ClimbingTurnActivity.cs.

7.10.5 Property Documentation

7.10.5.1 mas

```
ModuleActivityScheduler ClimbingTurnActivity.mas  [get], [private]
```

Reference to singleton activity scheduler.

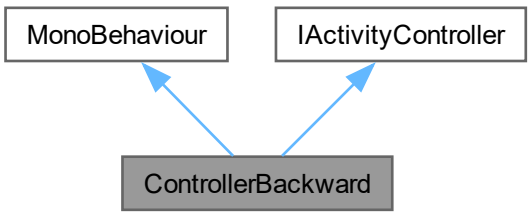
Definition at line 24 of file ClimbingTurnActivity.cs.

The documentation for this class was generated from the following file:

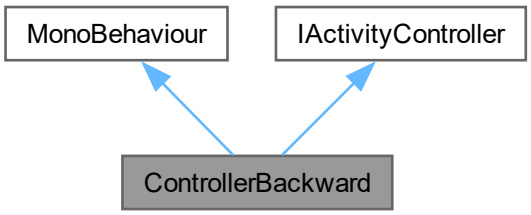
- Assets/ModuleExclusiveAssets/Module2/Section3/ClimbingTurnActivity.cs

7.11 ControllerBackward Class Reference

Inheritance diagram for ControllerBackward:



Collaboration diagram for ControllerBackward:



Public Member Functions

- void Start ()
- void StartActivity ()

  Each child activity can have a differing implementation of what must be carried out/what inputs must be switched on when the activity starts.
- void TriggerHaptic (BaseInteractionEventArgs e)
- void TriggerHaptic (XRBaseController controller)

Public Attributes

- InputActionReference rotateAction
- float intensity = 1f
- float duration = 5f
- float minZPositionThreshold = 0f
- float maxZPositionThreshold = 90f

Properties

- ModuleActivityScheduler mas `[get]`

Private Member Functions

- void OnEnable ()
- void OnDisable ()
- void OnPositionChange (InputAction.CallbackContext ctx)

Private Attributes

- bool activityEnabled = false

7.11.1 Detailed Description

Author

Chintana Luu | luucy003@mymail.unisa.edu.au

Definition at line 9 of file ControllerBackward.cs.

7.11.2 Member Function Documentation

7.11.2.1 OnDisable()

```
void ControllerBackward.OnDisable ()  [private]
```

Definition at line 43 of file ControllerBackward.cs.

7.11.2.2 OnEnable()

```
void ControllerBackward.OnEnable ()  [private]
```

Definition at line 32 of file ControllerBackward.cs.

7.11.2.3 OnPositionChange()

```
void ControllerBackward.OnPositionChange (
            InputAction.CallbackContext ctx)  [private]
```

Definition at line 75 of file ControllerBackward.cs.

7.11.2.4 Start()

```
void ControllerBackward.Start ()
```

Definition at line 25 of file ControllerBackward.cs.

7.11.2.5 StartActivity()

```
void ControllerBackward.StartActivity ()
```

Each child activity can have a differing implementation of what must be carried out/what inputs must be switched on when the activity starts.

Override this method for new activity scripts.

Implements IActivityController.

Definition at line 54 of file ControllerBackward.cs.

7.11.2.6 TriggerHaptic() [1/2]

```
void ControllerBackward.TriggerHaptic (
            BaseInteractionEventArgs e)
```

Definition at line 61 of file ControllerBackward.cs.

7.11.2.7 TriggerHaptic() [2/2]

```
void ControllerBackward.TriggerHaptic (
            XRBaseController controller)
```

Definition at line 68 of file ControllerBackward.cs.

7.11.3 Member Data Documentation

7.11.3.1 activityEnabled

```
bool ControllerBackward.activityEnabled = false  [private]
```

Definition at line 22 of file ControllerBackward.cs.

7.11.3.2 duration

```
float ControllerBackward.duration = 5f
```

Definition at line 17 of file ControllerBackward.cs.

7.11.3.3 intensity

```
float ControllerBackward.intensity = 1f
```

Definition at line 16 of file ControllerBackward.cs.

7.11.3.4 maxZPositionThreshold

```
float ControllerBackward.maxZPositionThreshold = 90f
```

Definition at line 20 of file ControllerBackward.cs.

7.11.3.5 minZPositionThreshold

```
float ControllerBackward.minZPositionThreshold = 0f
```

Definition at line 19 of file ControllerBackward.cs.

7.11.3.6 rotateAction

```
InputActionReference ControllerBackward.rotateAction
```

Definition at line 13 of file ControllerBackward.cs.

7.11.4 Property Documentation

7.11.4.1 mas

```
ModuleActivityScheduler ControllerBackward.mas  [get], [private]
```

Definition at line 11 of file ControllerBackward.cs.

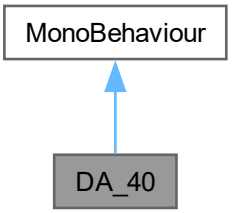
The documentation for this class was generated from the following file:

- Assets/Scripts/CommonActivityBehaviour/ControllerBackward.cs

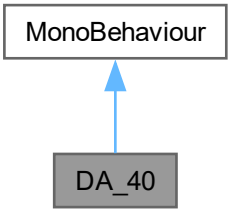
7.12 DA_40 Class Reference

Handles mostly cosmetic elements of the DA-40 aircraft itself, including default appearance, sounds, component access and toggling, and propeller rotation.

Inheritance diagram for DA_40:



Collaboration diagram for DA_40:



Public Member Functions

- void playSFX (AudioSource sound)

  Plays a sound that is passed in as an AudioSource.
- void stopSFX (AudioSource sound)

  Stops a sound that is currently playing, passed in as an AudioSource.
- void TogglePropeller (bool newState)

  Sets a new state for the propeller spin.

Private Member Functions

- void Start ()
- void Update ()

Private Attributes

- Color GlowColour

  The colour in which highlighted components will glow.
- List< GameObject > InitialHighlightedComponents = new List<GameObject>()

  A collection of aircraft components (as GameObjects) that will start glowing at runtime.
- bool CockpitClosed

  Flag determining if the cockpit of the aircraft is open or closed.
- bool PorteClosed

  Flag determining if the porte of the aircraft is open or closed.
- bool PropellerEnabled

  Flag determining if the front propeller is spinning or not.
- bool GhostTrailEnabled

  Flag determining if the ghost trail is active or not.
- ParticleSystem GhostTrail

  Particle system representing the "after-image", or ghost trail, of the aircraft.
- bool AxialPanelsEnabled

  DEPRECATED: Flag determining if the axial panel visualisation system is active or not.
- float PropellerRotationSpeed = 1000f

  The rotation speed of the front propeller.
- bool IsPassthroughEnabled

  DEPRECATED: Flag determining if passthrough mode is enabled or not.
- List< GameObject > AxialScrollingPlanes = new List<GameObject>()
- float AxialScrollSpeed

  DEPRECATED: The sensitivity of the scroll speed at which the textures of the axial panels shift.
- TextMeshProUGUI subtitles

  Reference to the subtitles displayed on the LocalCanvas of the user interface.
- AudioSource FlightLoop

  The whirring sound of the engine.
- AudioSource PropellerStart

  The startup sound of the engine when the plane is turned on.
- AudioSource StallWarning

  The stall horn sound that plays when a wing (or wings) is stalling.
- GameObject Dashboard

  The dashboard of the aircraft.
- GameObject Canopy

  The canopy of the aircraft.
- GameObject Porte

  The porte of the aircraft.
- GameObject Vitres

  The vitres of the aircraft.
- GameObject Interior

  The interior of the aircraft.
- GameObject Seats

  The seats of the aircraft.
- GameObject Yoke_R

  The right yoke of the aircraft.
- GameObject Yoke_L

  The left yoke of the aircraft.
- GameObject FuselageFrame

  The fuselage frame of the aircraft.

- GameObject Step_R

  The right step of the aircraft.
- GameObject Step_L

  The left step of the aircraft.
- GameObject Antennes

  The antennes of the aircraft.
- GameObject PropellerHub

  The propeller hub of the aircraft.
- GameObject Helice

  The helice of the aircraft.
- GameObject Exhaust

  The exhaust of the aircraft.
- GameObject EmpennageFrame

  The empennage frame of the aircraft.
- GameObject Rudder

  The rudder of the aircraft.
- GameObject Stabiliser

  The stabiliser of the aircraft.
- GameObject Elevator

  The elevator of the aircraft.
- GameObject TailGuard

  The tail guard of the aircraft.
- GameObject TailLight

  The tail light of the aircraft.
- GameObject WingFrame_R

  The right wing frame of the aircraft.
- GameObject Aileron_R

  The right aileron of the aircraft.
- GameObject LandingFlap_R

  The right landing flap of the aircraft.
- GameObject WingFrame_L

  The left wing frame of the aircraft.
- GameObject Aileron_L

  The left aileron of the aircraft.
- GameObject LandingFlap_L

  The left landing flap of the aircraft.
- GameObject LandingLightBulb

  The landing light bulb of the aircraft.
- GameObject InnerGuards

  The inner wing guards of the aircraft.
- GameObject Struts

  The struts of the aircraft.
- GameObject LandingStrutBrace_F

  The front landing strut brace of the aircraft.
- GameObject LandingStrut_F

  The front landing strut of the aircraft.
- GameObject OuterGuard_F

  The front outer guard of the aircraft.
- GameObject InnerGuard_F

  The front inner guard of the aircraft.
- GameObject Wheel_F

The front wheel of the aircraft.

- GameObject LandingStrut_R

  The right landing strut of the aircraft.
- GameObject OuterGuard_R

  The right outer guard of the aircraft.
- GameObject InnerGuard_R

  The right inner guard of the aircraft.
- GameObject Wheel_R

  The right wheel of the aircraft.
- GameObject LandingStrut_L

  The left landing strut of the aircraft.
- GameObject OuterGuard_L

  The left outer guard of the aircraft.
- GameObject InnerGuard_L

  The left inner guard of the aircraft.
- GameObject Wheel_L

  The left wheel of the aircraft.
- bool lastTrailState

  The previous state of the ghost trail.
- float propellerAcceleration = 500f

  The rate at which the propeller rotation accelerates to full speed.
- float currentPropellerSpeed = 0f

  The current rotation speed of the propeller.

7.12.1 Detailed Description

Handles mostly cosmetic elements of the DA-40 aircraft itself, including default appearance, sounds, component access and toggling, and propeller rotation.

Author

Caleb Martin | marcy066@mymail.unisa.edu.au

Definition at line 11 of file DA_40.cs.

7.12.2 Member Function Documentation

7.12.2.1 playSFX()

```
void DA_40.playSFX (
            AudioSource sound)
```

Plays a sound that is passed in as an AudioSource.

Parameters

sound	The sound to be played.

Returns

void

Definition at line 199 of file DA_40.cs.

7.12.2.2 Start()

```
void DA_40.Start ()  [private]
```

Definition at line 98 of file DA_40.cs.

7.12.2.3 stopSFX()

```
void DA_40.stopSFX (
            AudioSource sound)
```

Stops a sound that is currently playing, passed in as an AudioSource.

Parameters

sound	The sound to be stopped.

Returns

void

Definition at line 209 of file DA_40.cs.

7.12.2.4 TogglePropeller()

```
void DA_40.TogglePropeller (
            bool newState)
```

Sets a new state for the propeller spin.

Parameters

newState	The new state for the propeller.

Returns

void

Definition at line 219 of file DA_40.cs.

7.12.2.5 Update()

```
void DA_40.Update ()  [private]
```

Definition at line 137 of file DA_40.cs.

7.12.3 Member Data Documentation

7.12.3.1 Aileron_L

```
GameObject DA_40.Aileron_L  [private]
```

The left aileron of the aircraft.

Definition at line 69 of file DA_40.cs.

7.12.3.2 Aileron_R

```
GameObject DA_40.Aileron_R  [private]
```

The right aileron of the aircraft.

Definition at line 64 of file DA_40.cs.

7.12.3.3 Antennes

```
GameObject DA_40.Antennes  [private]
```

The antennes of the aircraft.

Definition at line 49 of file DA_40.cs.

7.12.3.4 AxialPanelsEnabled

```
bool DA_40.AxialPanelsEnabled  [private]
```

DEPRECATED: Flag determining if the axial panel visualisation system is active or not.

Definition at line 21 of file DA_40.cs.

7.12.3.5 AxialScrollingPlanes

```
List<GameObject> DA_40.AxialScrollingPlanes = new List<GameObject>()  [private]
```

Definition at line 24 of file DA_40.cs.

7.12.3.6 AxialScrollSpeed

```
float DA_40.AxialScrollSpeed  [private]
```

DEPRECATED: The sensitivity of the scroll speed at which the textures of the axial panels shift.

Definition at line 25 of file DA_40.cs.

7.12.3.7 Canopy

```
GameObject DA_40.Canopy  [private]
```

The canopy of the aircraft.

Definition at line 37 of file DA_40.cs.

7.12.3.8 CockpitClosed

```
bool DA_40.CockpitClosed  [private]
```

Flag determining if the cockpit of the aircraft is open or closed.

Definition at line 16 of file DA_40.cs.

7.12.3.9 currentPropellerSpeed

```
float DA_40.currentPropellerSpeed = 0f  [private]
```

The current rotation speed of the propeller.

Definition at line 95 of file DA_40.cs.

7.12.3.10 Dashboard

```
GameObject DA_40.Dashboard  [private]
```

The dashboard of the aircraft.

Definition at line 34 of file DA_40.cs.

7.12.3.11 Elevator

```
GameObject DA_40.Elevator  [private]
```

The elevator of the aircraft.

Definition at line 58 of file DA_40.cs.

7.12.3.12 EmpennageFrame

```
GameObject DA_40.EmpennageFrame  [private]
```

The empennage frame of the aircraft.

Definition at line 55 of file DA_40.cs.

7.12.3.13 Exhaust

```
GameObject DA_40.Exhaust  [private]
```

The exhaust of the aircraft.

Definition at line 52 of file DA_40.cs.

7.12.3.14 FlightLoop

```
AudioSource DA_40.FlightLoop  [private]
```

The whirring sound of the engine.

Definition at line 29 of file DA_40.cs.

7.12.3.15 FuselageFrame

```
GameObject DA_40.FuselageFrame  [private]
```

The fuselage frame of the aircraft.

Definition at line 46 of file DA_40.cs.

7.12.3.16 GhostTrail

```
ParticleSystem DA_40.GhostTrail  [private]
```

Particle system representing the "after-image", or ghost trail, of the aircraft.

It's been implemented as a way to visualise the trajectory of the aircraft while it's in-flight.

Definition at line 20 of file DA_40.cs.

7.12.3.17 GhostTrailEnabled

```
bool DA_40.GhostTrailEnabled  [private]
```

Flag determining if the ghost trail is active or not.

Definition at line 19 of file DA_40.cs.

7.12.3.18 GlowColour

```
Color DA_40.GlowColour  [private]
```

The colour in which highlighted components will glow.

Definition at line 14 of file DA_40.cs.

7.12.3.19 Helice

```
GameObject DA_40.Helice  [private]
```

The helice of the aircraft.

Definition at line 51 of file DA_40.cs.

7.12.3.20 InitialHighlightedComponents

```
List<GameObject> DA_40.InitialHighlightedComponents = new List<GameObject>()  [private]
```

A collection of aircraft components (as GameObjects) that will start glowing at runtime.

Definition at line 15 of file DA_40.cs.

7.12.3.21 InnerGuard_F

```
GameObject DA_40.InnerGuard_F  [private]
```

The front inner guard of the aircraft.

Definition at line 81 of file DA_40.cs.

7.12.3.22 InnerGuard_L

```
GameObject DA_40.InnerGuard_L  [private]
```

The left inner guard of the aircraft.

Definition at line 89 of file DA_40.cs.

7.12.3.23 InnerGuard_R

```
GameObject DA_40.InnerGuard_R  [private]
```

The right inner guard of the aircraft.

Definition at line 85 of file DA_40.cs.

7.12.3.24 InnerGuards

```
GameObject DA_40.InnerGuards  [private]
```

The inner wing guards of the aircraft.

Definition at line 74 of file DA_40.cs.

7.12.3.25 Interior

```
GameObject DA_40.Interior  [private]
```

The interior of the aircraft.

Definition at line 40 of file DA_40.cs.

7.12.3.26 IsPassthroughEnabled

```
bool DA_40.IsPassthroughEnabled  [private]
```

DEPRECATED: Flag determining if passthrough mode is enabled or not.

Definition at line 23 of file DA_40.cs.

7.12.3.27 LandingFlap_L

```
GameObject DA_40.LandingFlap_L  [private]
```

The left landing flap of the aircraft.

Definition at line 70 of file DA_40.cs.

7.12.3.28 LandingFlap_R

```
GameObject DA_40.LandingFlap_R  [private]
```

The right landing flap of the aircraft.

Definition at line 65 of file DA_40.cs.

7.12.3.29 LandingLightBulb

```
GameObject DA_40.LandingLightBulb  [private]
```

The landing light bulb of the aircraft.

Definition at line 71 of file DA_40.cs.

7.12.3.30 LandingStrut_F

```
GameObject DA_40.LandingStrut_F  [private]
```

The front landing strut of the aircraft.

Definition at line 79 of file DA_40.cs.

7.12.3.31 LandingStrut_L

```
GameObject DA_40.LandingStrut_L  [private]
```

The left landing strut of the aircraft.

Definition at line 87 of file DA_40.cs.

7.12.3.32 LandingStrut_R

```
GameObject DA_40.LandingStrut_R  [private]
```

The right landing strut of the aircraft.

Definition at line 83 of file DA_40.cs.

7.12.3.33 LandingStrutBrace_F

```
GameObject DA_40.LandingStrutBrace_F  [private]
```

The front landing strut brace of the aircraft.

Definition at line 78 of file DA_40.cs.

7.12.3.34 lastTrailState

```
bool DA_40.lastTrailState  [private]
```

The previous state of the ghost trail.

Definition at line 92 of file DA_40.cs.

7.12.3.35 OuterGuard_F

```
GameObject DA_40.OuterGuard_F  [private]
```

The front outer guard of the aircraft.

Definition at line 80 of file DA_40.cs.

7.12.3.36 OuterGuard_L

```
GameObject DA_40.OuterGuard_L  [private]
```

The left outer guard of the aircraft.

Definition at line 88 of file DA_40.cs.

7.12.3.37 OuterGuard_R

```
GameObject DA_40.OuterGuard_R  [private]
```

The right outer guard of the aircraft.

Definition at line 84 of file DA_40.cs.

7.12.3.38 Porte

```
GameObject DA_40.Porte  [private]
```

The porte of the aircraft.

Definition at line 38 of file DA_40.cs.

7.12.3.39 PorteClosed

```
bool DA_40.PorteClosed  [private]
```

Flag determining if the porte of the aircraft is open or closed.

Definition at line 17 of file DA_40.cs.

7.12.3.40 propellerAcceleration

```
float DA_40.propellerAcceleration = 500f  [private]
```

The rate at which the propeller rotation accelerates to full speed.

Definition at line 94 of file DA_40.cs.

7.12.3.41 PropellerEnabled

```
bool DA_40.PropellerEnabled  [private]
```

Flag determining if the front propeller is spinning or not.

Definition at line 18 of file DA_40.cs.

7.12.3.42 PropellerHub

```
GameObject DA_40.PropellerHub  [private]
```

The propeller hub of the aircraft.

Definition at line 50 of file DA_40.cs.

7.12.3.43 PropellerRotationSpeed

```
float DA_40.PropellerRotationSpeed = 1000f  [private]
```

The rotation speed of the front propeller.

Definition at line 22 of file DA_40.cs.

7.12.3.44 PropellerStart

```
AudioSource DA_40.PropellerStart  [private]
```

The startup sound of the engine when the plane is turned on.

Definition at line 30 of file DA_40.cs.

7.12.3.45 Rudder

```
GameObject DA_40.Rudder  [private]
```

The rudder of the aircraft.

Definition at line 56 of file DA_40.cs.

7.12.3.46 Seats

```
GameObject DA_40.Seats  [private]
```

The seats of the aircraft.

Definition at line 41 of file DA_40.cs.

7.12.3.47 Stabiliser

```
GameObject DA_40.Stabiliser  [private]
```

The stabiliser of the aircraft.

Definition at line 57 of file DA_40.cs.

7.12.3.48 StallWarning

```
AudioSource DA_40.StallWarning  [private]
```

The stall horn sound that plays when a wing (or wings) is stalling.

Definition at line 31 of file DA_40.cs.

7.12.3.49 Step_L

```
GameObject DA_40.Step_L  [private]
```

The left step of the aircraft.

Definition at line 48 of file DA_40.cs.

7.12.3.50 Step_R

```
GameObject DA_40.Step_R  [private]
```

The right step of the aircraft.

Definition at line 47 of file DA_40.cs.

7.12.3.51 Struts

```
GameObject DA_40.Struts  [private]
```

The struts of the aircraft.

Definition at line 75 of file DA_40.cs.

7.12.3.52 subtitles

```
TextMeshProUGUI DA_40.subtitles  [private]
```

Reference to the subtitles displayed on the LocalCanvas of the user interface.

Definition at line 26 of file DA_40.cs.

7.12.3.53 TailGuard

```
GameObject DA_40.TailGuard  [private]
```

The tail guard of the aircraft.

Definition at line 59 of file DA_40.cs.

7.12.3.54 TailLight

```
GameObject DA_40.TailLight  [private]
```

The tail light of the aircraft.

Definition at line 60 of file DA_40.cs.

7.12.3.55 Vitres

```
GameObject DA_40.Vitres  [private]
```

The vitres of the aircraft.

Definition at line 39 of file DA_40.cs.

7.12.3.56 Wheel_F

```
GameObject DA_40.Wheel_F  [private]
```

The front wheel of the aircraft.

Definition at line 82 of file DA_40.cs.

7.12.3.57 Wheel_L

```
GameObject DA_40.Wheel_L  [private]
```

The left wheel of the aircraft.

Definition at line 90 of file DA_40.cs.

7.12.3.58 Wheel_R

```
GameObject DA_40.Wheel_R  [private]
```

The right wheel of the aircraft.

Definition at line 86 of file DA_40.cs.

7.12.3.59 WingFrame_L

```
GameObject DA_40.WingFrame_L  [private]
```

The left wing frame of the aircraft.

Definition at line 68 of file DA_40.cs.

7.12.3.60 WingFrame_R

```
GameObject DA_40.WingFrame_R  [private]
```

The right wing frame of the aircraft.

Definition at line 63 of file DA_40.cs.

7.12.3.61 Yoke_L

```
GameObject DA_40.Yoke_L  [private]
```

The left yoke of the aircraft.

Definition at line 43 of file DA_40.cs.

7.12.3.62 Yoke_R

```
GameObject DA_40.Yoke_R  [private]
```

The right yoke of the aircraft.

Definition at line 42 of file DA_40.cs.

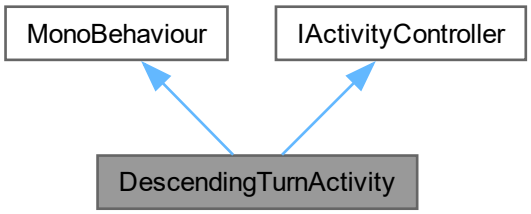
The documentation for this class was generated from the following file:

- Assets/Scripts/DA_40_SpecialFeatures/DA_40.cs

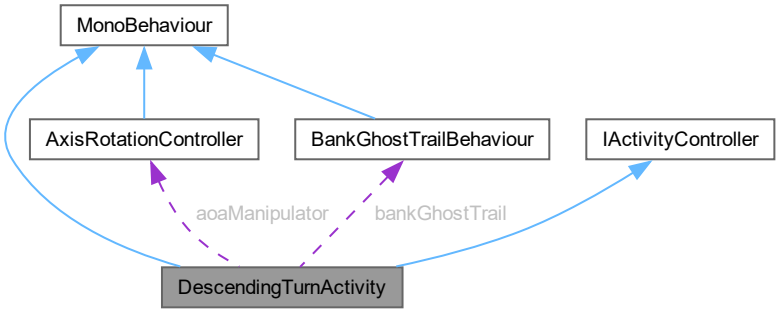
7.13 DescendingTurnActivity Class Reference

Descending turn training activity controller.

Inheritance diagram for DescendingTurnActivity:



Collaboration diagram for DescendingTurnActivity:



Public Member Functions

- void OnEnable ()

  Initialize activity on component enable.
- void OnDisable ()

  Cleanup activity on component disable.
- void StartActivity ()

  Start the activity when triggered by ModuleActivityScheduler.
- void EnableCurrentStep ()

  Unlock input for the current training step.
- void CheckSetupComplete ()

  Check if initial setup positioning is complete.
- void HandleBankAndPitch ()

  Handle simultaneous bank and pitch training step.
- void HandlePitch ()

  Handle pitch recovery training step.
- void HandleStall ()

  Handle automated stall demonstration step.

Public Attributes

- TimelineAsset postBankAndPitch

  Timeline played after achieving bank and pitch targets.
- TimelineAsset postPitchUp

  Timeline played after pitch recovery.
- TimelineAsset postStall

  Timeline demonstrating stall scenario.
- InputActionReference rotationInput

  VR controller rotation input action for aircraft control.
- float bankSensitivity = 45f

  Roll/pitch sensitivity multiplier for controller input.
- float deadZone = 0.1f

  Controller deadzone threshold in degrees to filter noise.
- float stepAdvanceTolerance = 3f

  Tolerance in degrees for achieving target angles before advancing.
- bool useMouseClicks = true

  Enable mouse button control for testing without VR.
- float mouseRollSpeed = 10f

  Speed multiplier for mouse-based roll/pitch control.

Properties

- ModuleActivityScheduler mas `[get]`

  Reference to singleton activity scheduler.

Private Types

- enum TurnStep {
  InitialSetup , BankAndPitch , PitchUp , Stall ,
  Complete , PlayingTimeline }

  Training progression states.

Private Member Functions

- void Update ()

  Main update loop.

Private Attributes

- Quaternion controllerOrigin = Quaternion.identity

  Initial controller orientation captured at step start.
- bool hasControllerOrigin = false

  Flag indicating if controller origin has been captured this step.
- bool hasNotifiedMAS = false

  Failsafe flag to prevent multiple completion notifications to scheduler.
- bool rollInputLocked = false

  Flag to disable roll input during transitions and timelines.
- bool pitchInputLocked = false

  Flag to disable pitch input during transitions and timelines.
- TurnStep currentStep

  Current training step in the activity sequence.
- float currentBank = 0f

  Current aircraft bank angle in degrees, updated each frame.
- AxisRotationController aoaManipulator

  Controller for aircraft pitch and roll manipulation.
- BankGhostTrailBehaviour bankGhostTrail

  Visual trail effect showing aircraft bank history.
- PlayableDirector postBankAndPitchDirector

  PlayableDirector for post bank-and-pitch timeline.
- PlayableDirector postPitchUpDirector

  PlayableDirector for post pitch-up timeline.
- PlayableDirector postStallDirector

  PlayableDirector for stall demonstration timeline.

7.13.1 Detailed Description

Descending turn training activity controller.

Manages a multi-step descending turn exercise where users practice simultaneous banking and pitch control, followed by recovery and stall demonstration. Implements the IActivityController interface for integration with the ModuleActivityScheduler system.

Training sequence:

1. Initial Setup - Automatically positions aircraft to -10 pitch
2. Bank and Pitch - User simultaneously achieves 30 bank and -5 pitch (descending turn)
3. Pitch Up - User recovers from descent by pitching up to +5
4. Stall - Automated stall demonstration timeline
5. Complete - Activity finished, notifies scheduler

Author

Connor Freebairn | frecd002@mymail.unisa.edu.au

Definition at line 22 of file DescendingTurnActivity.cs.

7.13.2 Member Enumeration Documentation

7.13.2.1 TurnStep

```
enum DescendingTurnActivity.TurnStep  [private]
```

Training progression states.

Defines the sequence of training steps in the descending turn activity.

Enumerator

InitialSetup	Automated positioning to starting pitch angle.
BankAndPitch	User performs simultaneous bank and pitch to descend.
PitchUp	User recovers from descent by pitching up.
Stall	Automated stall demonstration.
Complete	Activity finished, awaiting cleanup.
PlayingTimeline	Transitional state while timeline is playing.

Definition at line 53 of file DescendingTurnActivity.cs.

7.13.3 Member Function Documentation

7.13.3.1 CheckSetupComplete()

```
void DescendingTurnActivity.CheckSetupComplete ()
```

Check if initial setup positioning is complete.

Monitors the AxisRotationController lerp operations to detect when the aircraft has finished moving to the initial -10 pitch position. Advances to BankAndPitch step when both pitch and roll lerps are complete.

Definition at line 187 of file DescendingTurnActivity.cs.

7.13.3.2 EnableCurrentStep()

```
void DescendingTurnActivity.EnableCurrentStep ()
```

Unlock input for the current training step.

Resets roll and pitch input locks and controller origin flag to allow user control. Called after transitions and timeline completions.

Definition at line 139 of file DescendingTurnActivity.cs.

7.13.3.3 HandleBankAndPitch()

```
void DescendingTurnActivity.HandleBankAndPitch ()
```

Handle simultaneous bank and pitch training step.

Captures controller origin on first frame, reads current bank and pitch angles, and independently tracks achievement of two targets:

- Roll: 30 bank angle
- Pitch: -5 angle (descending)

Each axis locks independently when its target is achieved. When both are locked, plays confirmation timeline and advances to pitch recovery step. Allows independent controller or debug mouse input for each unlocked axis.

Definition at line 208 of file DescendingTurnActivity.cs.

7.13.3.4 HandlePitch()

```
void DescendingTurnActivity.HandlePitch ()
```

Handle pitch recovery training step.

Captures controller origin on first frame, reads current pitch angle, and checks if user has achieved +5 pitch target to recover from the descent. When target is reached, locks input, plays confirmation timeline, and advances to stall demonstration. Allows controller or debug mouse input when target not yet achieved.

Definition at line 288 of file DescendingTurnActivity.cs.

7.13.3.5 HandleStall()

```
void DescendingTurnActivity.HandleStall ()
```

Handle automated stall demonstration step.

Locks pitch input and immediately plays the stall demonstration timeline. Advances to completion state after timeline finishes. This is a passive demonstration step with no user input required.

Definition at line 335 of file DescendingTurnActivity.cs.

7.13.3.6 OnDisable()

```
void DescendingTurnActivity.OnDisable ()
```

Cleanup activity on component disable.

Disables input actions and turns off visual trail effects.

Definition at line 113 of file DescendingTurnActivity.cs.

7.13.3.7 OnEnable()

```
void DescendingTurnActivity.OnEnable ()
```

Initialize activity on component enable.

Enables input actions, resets progression state to initial values, retrieves references to aircraft controllers and visual effects, and finds PlayableDirectors for all timeline assets. Initializes aircraft to -10 pitch for descending turn start.

Definition at line 84 of file DescendingTurnActivity.cs.

7.13.3.8 StartActivity()

```
void DescendingTurnActivity.StartActivity ()
```

Start the activity when triggered by ModuleActivityScheduler.

Called by the activity scheduler to begin the training sequence. Unlocks input for the first step.

Implements IActivityController.

Definition at line 128 of file DescendingTurnActivity.cs.

7.13.3.9 Update()

```
void DescendingTurnActivity.Update ()  [private]
```

Main update loop.

Routes execution to appropriate handler based on current training step. Logs current step for debugging purposes.

Definition at line 152 of file DescendingTurnActivity.cs.

7.13.4 Member Data Documentation

7.13.4.1 aoaManipulator

```
AxisRotationController DescendingTurnActivity.aoaManipulator  [private]
```

Controller for aircraft pitch and roll manipulation.

Definition at line 69 of file DescendingTurnActivity.cs.

7.13.4.2 bankGhostTrail

```
BankGhostTrailBehaviour DescendingTurnActivity.bankGhostTrail  [private]
```

Visual trail effect showing aircraft bank history.

Definition at line 70 of file DescendingTurnActivity.cs.

7.13.4.3 bankSensitivity

```
float DescendingTurnActivity.bankSensitivity = 45f
```

Roll/pitch sensitivity multiplier for controller input.

Definition at line 35 of file DescendingTurnActivity.cs.

7.13.4.4 controllerOrigin

```
Quaternion DescendingTurnActivity.controllerOrigin = Quaternion.identity  [private]
```

Initial controller orientation captured at step start.

Definition at line 39 of file DescendingTurnActivity.cs.

7.13.4.5 currentBank

```
float DescendingTurnActivity.currentBank = 0f  [private]
```

Current aircraft bank angle in degrees, updated each frame.

Definition at line 66 of file DescendingTurnActivity.cs.

7.13.4.6 currentStep

```
TurnStep DescendingTurnActivity.currentStep  [private]
```

Current training step in the activity sequence.

Definition at line 65 of file DescendingTurnActivity.cs.

7.13.4.7 deadZone

```
float DescendingTurnActivity.deadZone = 0.1f
```

Controller deadzone threshold in degrees to filter noise.

Definition at line 36 of file DescendingTurnActivity.cs.

7.13.4.8 hasControllerOrigin

```
bool DescendingTurnActivity.hasControllerOrigin = false  [private]
```

Flag indicating if controller origin has been captured this step.

Definition at line 40 of file DescendingTurnActivity.cs.

7.13.4.9 hasNotifiedMAS

```
bool DescendingTurnActivity.hasNotifiedMAS = false  [private]
```

Failsafe flag to prevent multiple completion notifications to scheduler.

Definition at line 42 of file DescendingTurnActivity.cs.

7.13.4.10 mouseRollSpeed

```
float DescendingTurnActivity.mouseRollSpeed = 10f
```

Speed multiplier for mouse-based roll/pitch control.

Definition at line 46 of file DescendingTurnActivity.cs.

7.13.4.11 pitchInputLocked

```
bool DescendingTurnActivity.pitchInputLocked = false  [private]
```

Flag to disable pitch input during transitions and timelines.

Definition at line 64 of file DescendingTurnActivity.cs.

7.13.4.12 postBankAndPitch

```
TimelineAsset DescendingTurnActivity.postBankAndPitch
```

Timeline played after achieving bank and pitch targets.

Definition at line 27 of file DescendingTurnActivity.cs.

7.13.4.13 postBankAndPitchDirector

```
PlayableDirector DescendingTurnActivity.postBankAndPitchDirector  [private]
```

PlayableDirector for post bank-and-pitch timeline.

Definition at line 73 of file DescendingTurnActivity.cs.

7.13.4.14 postPitchUp

```
TimelineAsset DescendingTurnActivity.postPitchUp
```

Timeline played after pitch recovery.

Definition at line 28 of file DescendingTurnActivity.cs.

7.13.4.15 postPitchUpDirector

```
PlayableDirector DescendingTurnActivity.postPitchUpDirector  [private]
```

PlayableDirector for post pitch-up timeline.

Definition at line 74 of file DescendingTurnActivity.cs.

7.13.4.16 postStall

```
TimelineAsset DescendingTurnActivity.postStall
```

Timeline demonstrating stall scenario.

Definition at line 29 of file DescendingTurnActivity.cs.

7.13.4.17 postStallDirector

```
PlayableDirector DescendingTurnActivity.postStallDirector  [private]
```

PlayableDirector for stall demonstration timeline.

Definition at line 75 of file DescendingTurnActivity.cs.

7.13.4.18 rollInputLocked

```
bool DescendingTurnActivity.rollInputLocked = false  [private]
```

Flag to disable roll input during transitions and timelines.

Definition at line 63 of file DescendingTurnActivity.cs.

7.13.4.19 rotationInput

```
InputActionReference DescendingTurnActivity.rotationInput
```

VR controller rotation input action for aircraft control.

Definition at line 32 of file DescendingTurnActivity.cs.

7.13.4.20 stepAdvanceTolerance

```
float DescendingTurnActivity.stepAdvanceTolerance = 3f
```

Tolerance in degrees for achieving target angles before advancing.

Definition at line 37 of file DescendingTurnActivity.cs.

7.13.4.21 useMouseClicks

```
bool DescendingTurnActivity.useMouseClicks = true
```

Enable mouse button control for testing without VR.

Definition at line 45 of file DescendingTurnActivity.cs.

7.13.5 Property Documentation

7.13.5.1 mas

```
ModuleActivityScheduler DescendingTurnActivity.mas  [get], [private]
```

Reference to singleton activity scheduler.

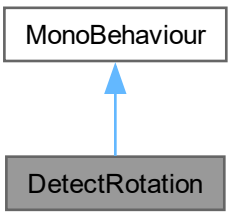
Definition at line 24 of file DescendingTurnActivity.cs.

The documentation for this class was generated from the following file:

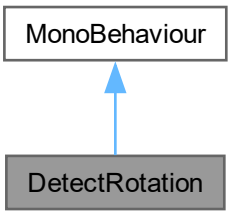
- Assets/ModuleExclusiveAssets/Module2/Section3/DescendingTurnActivity.cs

7.14 DetectRotation Class Reference

Inheritance diagram for DetectRotation:



Collaboration diagram for DetectRotation:



Public Attributes

- GameObject rightController
- GameObject DA_40

  or primary controller.
- InputActionReference locomotionToggle
- Boolean rotationToggle = false

Private Member Functions

- void Update ()

7.14.1 Detailed Description

Definition at line 5 of file DetectRotation.cs.

7.14.2 Member Function Documentation

7.14.2.1 Update()

```
void DetectRotation.Update ()  [private]
```

Definition at line 31 of file DetectRotation.cs.

7.14.3 Member Data Documentation

7.14.3.1 DA_40

```
GameObject DetectRotation.DA_40
```

or primary controller.

Definition at line 10 of file DetectRotation.cs.

7.14.3.2 locomotionToggle

```
InputActionReference DetectRotation.locomotionToggle
```

Definition at line 12 of file DetectRotation.cs.

7.14.3.3 rightController

```
GameObject DetectRotation.rightController
```

Definition at line 9 of file DetectRotation.cs.

7.14.3.4 rotationToggle

```
Boolean DetectRotation.rotationToggle = false
```

Definition at line 15 of file DetectRotation.cs.

The documentation for this class was generated from the following file:

- Assets/Scripts/CommonActivityBehaviour/DetectRotation.cs

7.15 GhostMeshGenerator Class Reference

Static Private Member Functions

- static void GenerateShostMesh ()

7.15.1 Detailed Description

Definition at line 5 of file GhostMeshGenerator.cs.

7.15.2 Member Function Documentation

7.15.2.1 GenerateShostMesh()

```
void GhostMeshGenerator.GenerateShostMesh ()  [static], [private]
```

Definition at line 8 of file GhostMeshGenerator.cs.

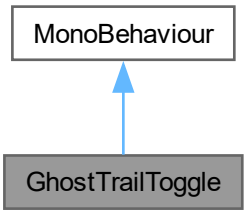
The documentation for this class was generated from the following file:

- Assets/Editor/GhostMeshGenerator.cs

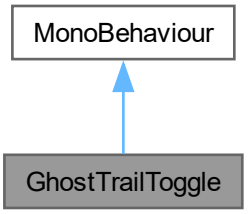
7.16 GhostTrailToggle Class Reference

Handles the toggling of the ghost trail belonging to the DA-40 aircraft on and off.

Inheritance diagram for GhostTrailToggle:



Collaboration diagram for GhostTrailToggle:



Public Attributes

- ParticleSystem GhostTrail
  The ghost trail particle system.
- InputActionReference toggleAction
  Reference to the InputAction that dictates the toggling of the ghost trail.

Private Member Functions

- void OnEnable ()
  Code to be executed at runtime, subscribes to input action so that ghost trail can be toggled on and off.
- void OnDisable ()
  Code to be executed at termination of application, unsubscribes to input action.
- void OnTogglePressed (InputAction.CallbackContext ctx)
  Handles the toggling of the ghost trail on and off.

Private Attributes

- bool ghostTrailEnabled = false

  Flag determining if the ghost trail is currently enabled or not.

7.16.1 Detailed Description

Handles the toggling of the ghost trail belonging to the DA-40 aircraft on and off.

Author

Caleb Martin | marcy066@mymail.unisa.edu.au

Definition at line 10 of file GhostTrailToggle.cs.

7.16.2 Member Function Documentation

7.16.2.1 OnDisable()

```
void GhostTrailToggle.OnDisable ()  [private]
```

Code to be executed at termination of application, unsubscribes to input action.

Returns

void

Definition at line 32 of file GhostTrailToggle.cs.

7.16.2.2 OnEnable()

```
void GhostTrailToggle.OnEnable ()  [private]
```

Code to be executed at runtime, subscribes to input action so that ghost trail can be toggled on and off.

Returns

void

Definition at line 22 of file GhostTrailToggle.cs.

7.16.2.3 OnTogglePressed()

```
void GhostTrailToggle.OnTogglePressed (
            InputAction.CallbackContext ctx)  [private]
```

Handles the toggling of the ghost trail on and off.

Parameters

ctx	The callback context provided by the InputAction to interpret input.

Returns

void

Definition at line 43 of file GhostTrailToggle.cs.

7.16.3 Member Data Documentation

7.16.3.1 GhostTrail

```
ParticleSystem GhostTrailToggle.GhostTrail
```

The ghost trail particle system.

Definition at line 12 of file GhostTrailToggle.cs.

7.16.3.2 ghostTrailEnabled

```
bool GhostTrailToggle.ghostTrailEnabled = false  [private]
```

Flag determining if the ghost trail is currently enabled or not.

Definition at line 14 of file GhostTrailToggle.cs.

7.16.3.3 toggleAction

```
InputActionReference GhostTrailToggle.toggleAction
```

Reference to the InputAction that dictates the toggling of the ghost trail.

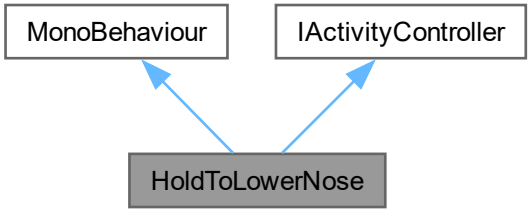
Definition at line 16 of file GhostTrailToggle.cs.

The documentation for this class was generated from the following file:

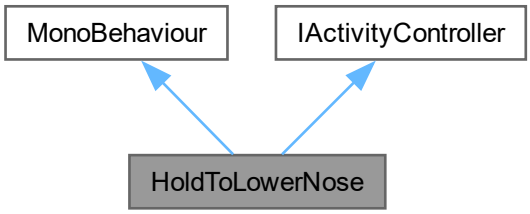
- Assets/Scripts/MiscBehaviour/GhostTrailToggle.cs

7.17 HoldToLowerNose Class Reference

Inheritance diagram for HoldToLowerNose:



Collaboration diagram for HoldToLowerNose:



Public Member Functions

- void StartActivity ()

  Each child activity can have a differing implementation of what must be carried out/what inputs must be switched on when the activity starts.
- void Update ()

Public Attributes

- InputActionReference rotateAction
- float minRotationThreshold = 0.1f
- float maxRotationThreshold = 90f

Properties

- ModuleActivityScheduler mas `[get]`

Private Member Functions

- void OnEnable ()
- void OnDisable ()
- IEnumerator WaitPeriod ()
- void OnPositionChange (InputAction.CallbackContext ctx)

  This script checks if the nose is lowered in the correct range and then waits the for number of seconds specified in the Wait coroutine.

Private Attributes

- Quaternion initialControllerRotation
- Quaternion controllerRotation
- Vector3 initialControllerRot
- Vector3 controllerRot
- bool waitPeriodOver = false
- bool activityEnabled = false

7.17.1 Detailed Description

Author

Chintana Luu | luucy003@mymail.unisa.edu.au

Definition at line 12 of file HoldToLowerNose.cs.

7.17.2 Member Function Documentation

7.17.2.1 OnDisable()

```
void HoldToLowerNose.OnDisable ()  [private]
```

Definition at line 49 of file HoldToLowerNose.cs.

7.17.2.2 OnEnable()

```
void HoldToLowerNose.OnEnable ()  [private]
```

Definition at line 38 of file HoldToLowerNose.cs.

7.17.2.3 OnPositionChange()

```
void HoldToLowerNose.OnPositionChange (
            InputAction.CallbackContext ctx)  [private]
```

This script checks if the nose is lowered in the correct range and then waits the for number of seconds specified in the Wait coroutine.

However, for future developers I suggest adding some haptics and improving the threshold range to make it easier for the user to complete this step.

and this is an extra line

Parameters

no	params in this method lol
none	

Returns

void

Definition at line 88 of file HoldToLowerNose.cs.

7.17.2.4 StartActivity()

```
void HoldToLowerNose.StartActivity ()
```

Each child activity can have a differing implementation of what must be carried out/what inputs must be switched on when the activity starts.

Override this method for new activity scripts.

Implements IActivityController.

Definition at line 60 of file HoldToLowerNose.cs.

7.17.2.5 Update()

```
void HoldToLowerNose.Update ()
```

Definition at line 126 of file HoldToLowerNose.cs.

7.17.2.6 WaitPeriod()

```
IEnumerator HoldToLowerNose.WaitPeriod ()  [private]
```

Definition at line 67 of file HoldToLowerNose.cs.

7.17.3 Member Data Documentation

7.17.3.1 activityEnabled

```
bool HoldToLowerNose.activityEnabled = false  [private]
```

Definition at line 35 of file HoldToLowerNose.cs.

7.17.3.2 controllerRot

```
Vector3 HoldToLowerNose.controllerRot  [private]
```

Definition at line 27 of file HoldToLowerNose.cs.

7.17.3.3 controllerRotation

```
Quaternion HoldToLowerNose.controllerRotation  [private]
```

Definition at line 25 of file HoldToLowerNose.cs.

7.17.3.4 initialControllerRot

```
Vector3 HoldToLowerNose.initialControllerRot  [private]
```

Definition at line 26 of file HoldToLowerNose.cs.

7.17.3.5 initialControllerRotation

```
Quaternion HoldToLowerNose.initialControllerRotation  [private]
```

Definition at line 24 of file HoldToLowerNose.cs.

7.17.3.6 maxRotationThreshold

```
float HoldToLowerNose.maxRotationThreshold = 90f
```

Definition at line 19 of file HoldToLowerNose.cs.

7.17.3.7 minRotationThreshold

```
float HoldToLowerNose.minRotationThreshold = 0.1f
```

Definition at line 18 of file HoldToLowerNose.cs.

7.17.3.8 rotateAction

```
InputActionReference HoldToLowerNose.rotateAction
```

Definition at line 16 of file HoldToLowerNose.cs.

7.17.3.9 waitPeriodOver

```
bool HoldToLowerNose.waitPeriodOver = false  [private]
```

Definition at line 31 of file HoldToLowerNose.cs.

7.17.4 Property Documentation

7.17.4.1 mas

```
ModuleActivityScheduler HoldToLowerNose.mas  [get], [private]
```

Definition at line 14 of file HoldToLowerNose.cs.

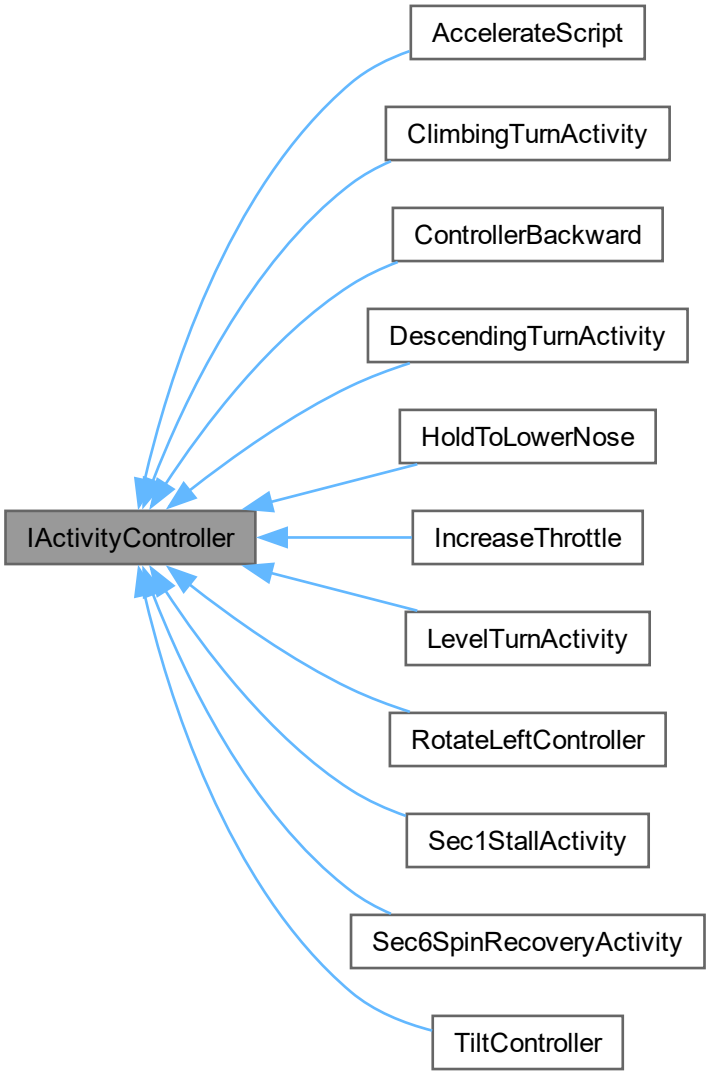
The documentation for this class was generated from the following file:

- Assets/ModuleExclusiveAssets/Module2/Section2/HoldToLowerNose.cs

7.18 IActivityController Interface Reference

Interface class.

Inheritance diagram for IActivityController:



Public Member Functions

- void StartActivity ()

  Each child activity can have a differing implementation of what must be carried out/what inputs must be switched on when the activity starts.

7.18.1 Detailed Description

Interface class.

All activities implement this interface to standardize access from ModuleActivityScheduler's controller prefab instantiation.

Author

Caleb Martin | marcy066@mymail.unisa.edu.au

Definition at line 9 of file IActivityController.cs.

7.18.2 Member Function Documentation

7.18.2.1 StartActivity()

```
void IActivityController.StartActivity ()
```

Each child activity can have a differing implementation of what must be carried out/what inputs must be switched on when the activity starts.

Override this method for new activity scripts.

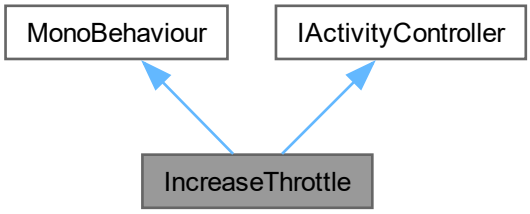
Implemented in AccelerateScript, ClimbingTurnActivity, ControllerBackward, DescendingTurnActivity, HoldToLowerNose, IncreaseThrottle, LevelTurnActivity, RotateLeftController, Sec1StallActivity, Sec6SpinRecoveryActivity, and TiltController.

The documentation for this interface was generated from the following file:

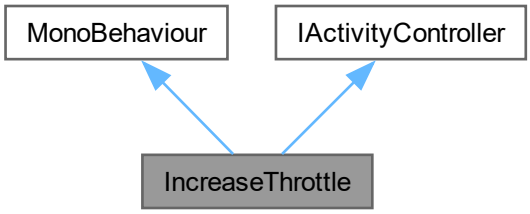
- Assets/Scripts/ActivityStructure/IActivityController.cs

7.19 IncreaseThrottle Class Reference

Inheritance diagram for IncreaseThrottle:



Collaboration diagram for IncreaseThrottle:



Public Member Functions

- void StartActivity ()

  Each child activity can have a differing implementation of what must be carried out/what inputs must be switched on when the activity starts.

Public Attributes

- InputActionReference inputAction
- float throttleThreshold = 0.1f

Properties

- ModuleActivityScheduler mas `[get]`

Private Member Functions

- void OnEnable ()
- void OnDisable ()
- void OnPadMoved (InputAction.CallbackContext ctx)
- void OnPadReleased (InputAction.CallbackContext ctx)

Private Attributes

- bool activityEnabled = false
- bool hasTriggered = false

7.19.1 Detailed Description

Definition at line 4 of file IncreaseThrottle.cs.

7.19.2 Member Function Documentation

7.19.2.1 OnDisable()

```
void IncreaseThrottle.OnDisable ()  [private]
```

Definition at line 26 of file IncreaseThrottle.cs.

7.19.2.2 OnEnable()

```
void IncreaseThrottle.OnEnable ()  [private]
```

Definition at line 15 of file IncreaseThrottle.cs.

7.19.2.3 OnPadMoved()

```
void IncreaseThrottle.OnPadMoved (
            InputAction.CallbackContext ctx)  [private]
```

Definition at line 43 of file IncreaseThrottle.cs.

7.19.2.4 OnPadReleased()

```
void IncreaseThrottle.OnPadReleased (
            InputAction.CallbackContext ctx)  [private]
```

Definition at line 74 of file IncreaseThrottle.cs.

7.19.2.5 StartActivity()

```
void IncreaseThrottle.StartActivity ()
```

Each child activity can have a differing implementation of what must be carried out/what inputs must be switched on when the activity starts.

Override this method for new activity scripts.

Implements IActivityController.

Definition at line 37 of file IncreaseThrottle.cs.

7.19.3 Member Data Documentation

7.19.3.1 activityEnabled

```
bool IncreaseThrottle.activityEnabled = false  [private]
```

Definition at line 10 of file IncreaseThrottle.cs.

7.19.3.2 hasTriggered

```
bool IncreaseThrottle.hasTriggered = false  [private]
```

Definition at line 12 of file IncreaseThrottle.cs.

7.19.3.3 inputAction

```
InputActionReference IncreaseThrottle.inputAction
```

Definition at line 8 of file IncreaseThrottle.cs.

7.19.3.4 throttleThreshold

```
float IncreaseThrottle.throttleThreshold = 0.1f
```

Definition at line 9 of file IncreaseThrottle.cs.

7.19.4 Property Documentation

7.19.4.1 mas

```
ModuleActivityScheduler IncreaseThrottle.mas  [get], [private]
```

Definition at line 6 of file IncreaseThrottle.cs.

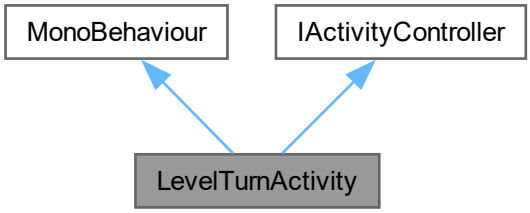
The documentation for this class was generated from the following file:

- Assets/ModuleExclusiveAssets/Module2/Section2/NEW/IncreaseThrottle.cs

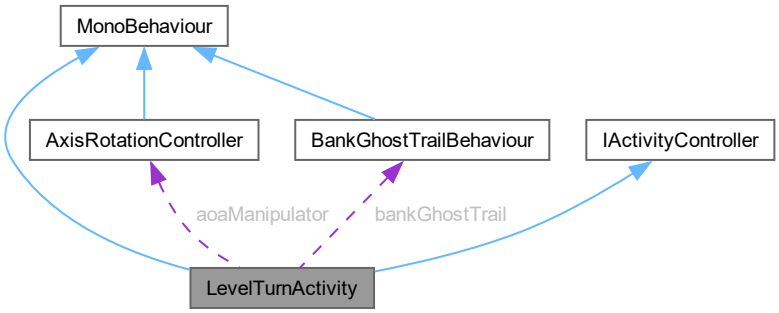
7.20 LevelTurnActivity Class Reference

Level turn training activity controller.

Inheritance diagram for LevelTurnActivity:



Collaboration diagram for LevelTurnActivity:



Public Member Functions

- void OnEnable ()
  Initialize activity on component enable.
- void OnDisable ()
  Cleanup activity on component disable.
- void StartActivity ()
  Start the activity when triggered by ModuleActivityScheduler.
- void EnableCurrentStep ()
  Unlock input for the current training step.
- void HandleBanking ()
  Handle progressive banking training step.
- void StallSetup ()
  Automated setup for stall demonstration.
- void HandleStall ()
  Handle stall demonstration training step.

Public Attributes

- TimelineAsset[ ] bankTimelineAssets

  Timeline assets for each bank angle milestone (15, 30, 45, 60).
- TimelineAsset stallTimelineAsset

  Timeline asset for stall demonstration sequence.
- InputActionReference rotationInput

  VR controller rotation input action for aircraft control.
- float bankSensitivity = 45f

  Roll sensitivity multiplier for controller input.
- float deadZone = 0.1f

  Controller deadzone threshold in degrees to filter noise.
- float stepAdvanceTolerance = 3f

  Tolerance in degrees for achieving target angles before advancing.
- bool useMouseClicks = false

  Enable mouse button control for testing without VR.
- float mouseRollSpeed = 10000f

  Speed multiplier for mouse-based roll/pitch control.

Properties

- ModuleActivityScheduler mas `[get]`

  Reference to singleton activity scheduler.

Private Types

- enum TurnStep { Bank , stallSetup , Stall , Complete }

  Training progression states.

Private Member Functions

- void Update ()

  Main update loop.

Private Attributes

- Quaternion controllerOrigin = Quaternion.identity

  Initial controller orientation captured at step start.
- bool hasControllerOrigin = false

  Flag indicating if controller origin has been captured this step.
- float[ ] bankTargets = { 15f, 30f, 45f, 60f }

  Progressive bank angle targets in degrees.
- int currentBankTarget = 0

  Index of current bank target in bankTargets array.
- TurnStep currentStep

  Current training step in the activity sequence.
- bool inputLocked = false

  Flag to disable user input during transitions and timelines.
- float currentBank = 0f

Current aircraft bank angle in degrees, updated each frame.

- PlayableDirector[ ] bankTimelines

  PlayableDirectors corresponding to bankTimelineAssets.

- PlayableDirector stallTimeline

  PlayableDirector for stall demonstration timeline.

- AxisRotationController aoaManipulator

  Controller for aircraft pitch and roll manipulation.

- BankGhostTrailBehaviour bankGhostTrail

  Visual trail effect showing aircraft bank history.

7.20.1 Detailed Description

Level turn training activity controller.

Manages a progressive banking exercise where users practice aircraft turns at increasing bank angles (15, 30, 45, 60) followed by a stall recovery demonstration. Implements the IActivityController interface for integration with the ModuleActivityScheduler system.

Training sequence:

1. Bank - Practice banking at progressive angles with VR controller input
2. Stall Setup - Automatically position aircraft to 60 bank
3. Stall - Practice pitch control to induce controlled stall
4. Complete - Wait for activity termination

Author

Connor Freebairn | frecd002@mymail.unisa.edu.au

Definition at line 21 of file LevelTurnActivity.cs.

7.20.2 Member Enumeration Documentation

7.20.2.1 TurnStep

```
enum LevelTurnActivity.TurnStep  [private]
```

Training progression states.

Defines the sequence of training steps in the level turn activity.

Enumerator

Bank	User practices progressive banking angles.
stallSetup	Automated positioning to 60 bank for stall.

Stall	User practices pitch control to induce stall.
Complete	Activity finished, awaiting cleanup.

Definition at line 49 of file LevelTurnActivity.cs.

7.20.3 Member Function Documentation

7.20.3.1 EnableCurrentStep()

```
void LevelTurnActivity.EnableCurrentStep ()
```

Unlock input for the current training step.

Resets input lock and controller origin flags to allow user control. Called after transitions and timeline completions.

Definition at line 126 of file LevelTurnActivity.cs.

7.20.3.2 HandleBanking()

```
void LevelTurnActivity.HandleBanking ()
```

Handle progressive banking training step.

Captures controller origin on first frame, reads current bank angle, and checks if user has achieved the current target angle. When target is reached, locks input, plays confirmation timeline, and advances to next bank target or stall setup. Allows controller or debug mouse input when target not yet achieved.

Definition at line 164 of file LevelTurnActivity.cs.

7.20.3.3 HandleStall()

```
void LevelTurnActivity.HandleStall ()
```

Handle stall demonstration training step.

Captures controller origin on first frame, reads current pitch angle, and checks if user has achieved 10 pitch target to induce stall. When target is reached, locks input, plays stall timeline, and notifies activity scheduler of completion. Allows controller or debug mouse input for pitch control when target not yet achieved.

Definition at line 249 of file LevelTurnActivity.cs.

7.20.3.4 OnDisable()

```
void LevelTurnActivity.OnDisable ()
```

Cleanup activity on component disable.

Disables input actions and turns off visual trail effects.

Definition at line 100 of file LevelTurnActivity.cs.

7.20.3.5 OnEnable()

```
void LevelTurnActivity.OnEnable ()
```

Initialize activity on component enable.

Enables input actions, resets progression state to initial values, retrieves references to aircraft controllers and visual effects, and finds PlayableDirectors for all timeline assets.

Definition at line 76 of file LevelTurnActivity.cs.

7.20.3.6 StallSetup()

```
void LevelTurnActivity.StallSetup ()
```

Automated setup for stall demonstration.

Locks input and automatically positions aircraft to 60 bank angle using lerp. Handles both positive and negative bank angles by detecting current bank sign. Advances to stall step when 60° target is achieved within tolerance.

Definition at line 219 of file LevelTurnActivity.cs.

7.20.3.7 StartActivity()

```
void LevelTurnActivity.StartActivity ()
```

Start the activity when triggered by ModuleActivityScheduler.

Called by the activity scheduler to begin the training sequence. Unlocks input for the first step.

Implements IActivityController.

Definition at line 115 of file LevelTurnActivity.cs.

7.20.3.8 Update()

```
void LevelTurnActivity.Update ()  [private]
```

Main update loop.

Routes execution to appropriate handler based on current training step.

Definition at line 137 of file LevelTurnActivity.cs.

7.20.4 Member Data Documentation

7.20.4.1 aoaManipulator

```
AxisRotationController LevelTurnActivity.aoaManipulator  [private]
```

Controller for aircraft pitch and roll manipulation.

Definition at line 66 of file LevelTurnActivity.cs.

7.20.4.2 bankGhostTrail

```
BankGhostTrailBehaviour LevelTurnActivity.bankGhostTrail  [private]
```

Visual trail effect showing aircraft bank history.

Definition at line 67 of file LevelTurnActivity.cs.

7.20.4.3 bankSensitivity

```
float LevelTurnActivity.bankSensitivity = 45f
```

Roll sensitivity multiplier for controller input.

Definition at line 33 of file LevelTurnActivity.cs.

7.20.4.4 bankTargets

```
float [] LevelTurnActivity.bankTargets = { 15f, 30f, 45f, 60f }  [private]
```

Progressive bank angle targets in degrees.

Definition at line 57 of file LevelTurnActivity.cs.

7.20.4.5 bankTimelineAssets

```
TimelineAsset [] LevelTurnActivity.bankTimelineAssets
```

Timeline assets for each bank angle milestone (15, 30, 45, 60).

Definition at line 26 of file LevelTurnActivity.cs.

7.20.4.6 bankTimelines

```
PlayableDirector [] LevelTurnActivity.bankTimelines  [private]
```

PlayableDirectors corresponding to bankTimelineAssets.

Definition at line 63 of file LevelTurnActivity.cs.

7.20.4.7 controllerOrigin

```
Quaternion LevelTurnActivity.controllerOrigin = Quaternion.identity  [private]
```

Initial controller orientation captured at step start.

Definition at line 37 of file LevelTurnActivity.cs.

7.20.4.8 currentBank

```
float LevelTurnActivity.currentBank = 0f  [private]
```

Current aircraft bank angle in degrees, updated each frame.

Definition at line 61 of file LevelTurnActivity.cs.

7.20.4.9 currentBankTarget

```
int LevelTurnActivity.currentBankTarget = 0  [private]
```

Index of current bank target in bankTargets array.

Definition at line 58 of file LevelTurnActivity.cs.

7.20.4.10 currentStep

```
TurnStep LevelTurnActivity.currentStep  [private]
```

Current training step in the activity sequence.

Definition at line 59 of file LevelTurnActivity.cs.

7.20.4.11 deadZone

```
float LevelTurnActivity.deadZone = 0.1f
```

Controller deadzone threshold in degrees to filter noise.

Definition at line 34 of file LevelTurnActivity.cs.

7.20.4.12 hasControllerOrigin

```
bool LevelTurnActivity.hasControllerOrigin = false  [private]
```

Flag indicating if controller origin has been captured this step.

Definition at line 38 of file LevelTurnActivity.cs.

7.20.4.13 inputLocked

```
bool LevelTurnActivity.inputLocked = false  [private]
```

Flag to disable user input during transitions and timelines.

Definition at line 60 of file LevelTurnActivity.cs.

7.20.4.14 mouseRollSpeed

```
float LevelTurnActivity.mouseRollSpeed = 10000f
```

Speed multiplier for mouse-based roll/pitch control.

Definition at line 42 of file LevelTurnActivity.cs.

7.20.4.15 rotationInput

```
InputActionReference LevelTurnActivity.rotationInput
```

VR controller rotation input action for aircraft control.

Definition at line 30 of file LevelTurnActivity.cs.

7.20.4.16 stallTimeline

```
PlayableDirector LevelTurnActivity.stallTimeline  [private]
```

PlayableDirector for stall demonstration timeline.

Definition at line 64 of file LevelTurnActivity.cs.

7.20.4.17 stallTimelineAsset

```
TimelineAsset LevelTurnActivity.stallTimelineAsset
```

Timeline asset for stall demonstration sequence.

Definition at line 27 of file LevelTurnActivity.cs.

7.20.4.18 stepAdvanceTolerance

```
float LevelTurnActivity.stepAdvanceTolerance = 3f
```

Tolerance in degrees for achieving target angles before advancing.

Definition at line 35 of file LevelTurnActivity.cs.

7.20.4.19 useMouseClicks

```
bool LevelTurnActivity.useMouseClicks = false
```

Enable mouse button control for testing without VR.

Definition at line 41 of file LevelTurnActivity.cs.

7.20.5 Property Documentation

7.20.5.1 mas

```
ModuleActivityScheduler LevelTurnActivity.mas  [get], [private]
```

Reference to singleton activity scheduler.

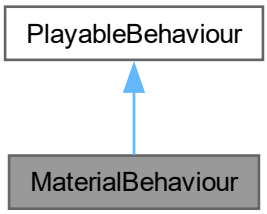
Definition at line 23 of file LevelTurnActivity.cs.

The documentation for this class was generated from the following file:

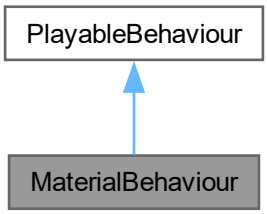
- Assets/ModuleExclusiveAssets/Module2/Section3/LevelTurnActivity.cs

7.21 MaterialBehaviour Class Reference

Inheritance diagram for MaterialBehaviour:



Collaboration diagram for MaterialBehaviour:



Public Member Functions

- override void ProcessFrame (Playable playable, FrameData info, object playerData)

Public Attributes

- Material currentMaterial
- Texture targetTexture

7.21.1 Detailed Description

Definition at line 5 of file MaterialBehaviour.cs.

7.21.2 Member Function Documentation

7.21.2.1 ProcessFrame()

```
override void MaterialBehaviour.ProcessFrame (
            Playable playable,
            FrameData info,
            object playerData)
```

Definition at line 11 of file MaterialBehaviour.cs.

7.21.3 Member Data Documentation

7.21.3.1 currentMaterial

```
Material MaterialBehaviour.currentMaterial
```

Definition at line 8 of file MaterialBehaviour.cs.

7.21.3.2 targetTexture

```
Texture MaterialBehaviour.targetTexture
```

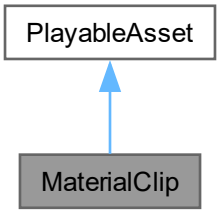
Definition at line 9 of file MaterialBehaviour.cs.

The documentation for this class was generated from the following file:

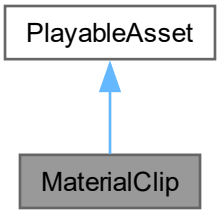
- Assets/Scripts/CustomTimelineTracks/MaterialTimelineTrack/MaterialBehaviour.cs

7.22 MaterialClip Class Reference

Inheritance diagram for MaterialClip:



Collaboration diagram for MaterialClip:



Public Member Functions

- override Playable CreatePlayable (PlayableGraph graph, GameObject owner)

Public Attributes

- Material currentMaterial
- Texture targetTexture

7.22.1 Detailed Description

Definition at line 5 of file MaterialClip.cs.

7.22.2 Member Function Documentation

7.22.2.1 CreatePlayable()

```
override Playable MaterialClip.CreatePlayable (
            PlayableGraph graph,
            GameObject owner)
```

Definition at line 13 of file MaterialClip.cs.

7.22.3 Member Data Documentation

7.22.3.1 currentMaterial

```
Material MaterialClip.currentMaterial
```

Definition at line 9 of file MaterialClip.cs.

7.22.3.2 targetTexture

```
Texture MaterialClip.targetTexture
```

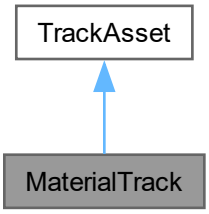
Definition at line 10 of file MaterialClip.cs.

The documentation for this class was generated from the following file:

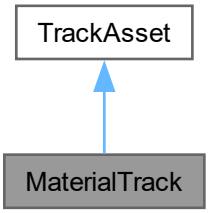
- Assets/Scripts/CustomTimelineTracks/MaterialTimelineTrack/MaterialClip.cs

7.23 MaterialTrack Class Reference

Inheritance diagram for MaterialTrack:



Collaboration diagram for MaterialTrack:



Private Member Functions

- void Start ()
- void Update ()

7.23.1 Detailed Description

Definition at line 13 of file MaterialTrack.cs.

7.23.2 Member Function Documentation

7.23.2.1 Start()

```
void MaterialTrack.Start ()  [private]
```

Definition at line 16 of file MaterialTrack.cs.

7.23.2.2 Update()

```
void MaterialTrack.Update ()  [private]
```

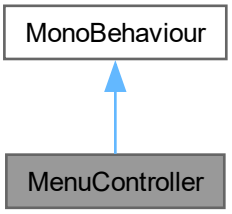
Definition at line 22 of file MaterialTrack.cs.

The documentation for this class was generated from the following file:

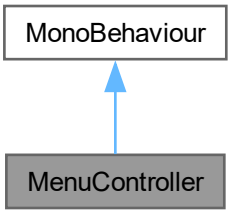
- Assets/Scripts/CustomTimelineTracks/MaterialTimelineTrack/MaterialTrack.cs

7.24 MenuController Class Reference

Inheritance diagram for MenuController:



Collaboration diagram for MenuController:



Public Attributes

- GameObject menuPanel
- InputActionReference openMenuAction

Private Member Functions

- void Awake ()
- void OnDestroy ()
- void ToggleMenu (InputAction.CallbackContext context)
- void OnDeviceChange (InputDevice device, InputDeviceChange change)

7.24.1 Detailed Description

Definition at line 5 of file MenuController.cs.

7.24.2 Member Function Documentation

7.24.2.1 Awake()

```
void MenuController.Awake ()  [private]
```

Definition at line 10 of file MenuController.cs.

7.24.2.2 OnDestroy()

```
void MenuController.OnDestroy ()  [private]
```

Definition at line 17 of file MenuController.cs.

7.24.2.3 OnDeviceChange()

```
void MenuController.OnDeviceChange (
            InputDevice device,
            InputDeviceChange change)  [private]
```

Definition at line 28 of file MenuController.cs.

7.24.2.4 ToggleMenu()

```
void MenuController.ToggleMenu (
            InputAction.CallbackContext context)  [private]
```

Definition at line 24 of file MenuController.cs.

7.24.3 Member Data Documentation

7.24.3.1 menuPanel

```
GameObject MenuController.menuPanel
```

Definition at line 8 of file MenuController.cs.

7.24.3.2 openMenuAction

```
InputActionReference MenuController.openMenuAction
```

Definition at line 9 of file MenuController.cs.

The documentation for this class was generated from the following file:

- Assets/Scripts/UI/MenuController.cs

7.25 ModuleManager.Module Class Reference

Modules are the collections of sections that form learning content within the application.

Public Attributes

- string name

  The name of the Module.
- List< Section > sections

  A collection of Sections that make up a Module.

7.25.1 Detailed Description

Modules are the collections of sections that form learning content within the application.

They possess a name, and are made up of a collection of Sections.

Definition at line 19 of file ModuleManager.cs.

7.25.2 Member Data Documentation

7.25.2.1 name

```
string ModuleManager.Module.name
```

The name of the Module.

Definition at line 21 of file ModuleManager.cs.

7.25.2.2 sections

```
List<Section> ModuleManager.Module.sections
```

A collection of Sections that make up a Module.

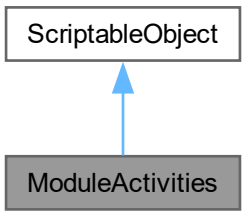
Definition at line 22 of file ModuleManager.cs.

The documentation for this class was generated from the following file:

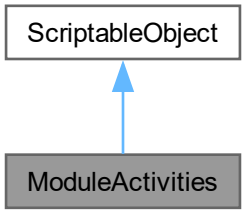
- Assets/Scripts/ModuleStructure/ModuleManager.cs

7.26 ModuleActivities Class Reference

Inheritance diagram for ModuleActivities:



Collaboration diagram for ModuleActivities:



Public Member Functions

- GameObject PrefabSetup (Transform parent)
- bool AdvanceStep ()

Public Attributes

- string activityName
- string[ ] instructions
- bool usesCustomScript = false
- GameObject customScript
- bool playerInputRequired = true
- int totalSteps = 1
- int currentStep = 0

7.26.1 Detailed Description

Definition at line 6 of file ModuleActivities.cs.

7.26.2 Member Function Documentation

7.26.2.1 AdvanceStep()

```
bool ModuleActivities.AdvanceStep ()
```

Definition at line 35 of file ModuleActivities.cs.

7.26.2.2 PrefabSetup()

```
GameObject ModuleActivities.PrefabSetup (
            Transform parent)
```

Definition at line 20 of file ModuleActivities.cs.

7.26.3 Member Data Documentation

7.26.3.1 activityName

```
string ModuleActivities.activityName
```

Definition at line 8 of file ModuleActivities.cs.

7.26.3.2 currentStep

```
int ModuleActivities.currentStep = 0
```

Definition at line 17 of file ModuleActivities.cs.

7.26.3.3 customScript

```
GameObject ModuleActivities.customScript
```

Definition at line 12 of file ModuleActivities.cs.

7.26.3.4 instructions

```
string [] ModuleActivities.instructions
```

Definition at line 9 of file ModuleActivities.cs.

7.26.3.5 playerInputRequired

```
bool ModuleActivities.playerInputRequired = true
```

Definition at line 14 of file ModuleActivities.cs.

7.26.3.6 totalSteps

```
int ModuleActivities.totalSteps = 1
```

Definition at line 16 of file ModuleActivities.cs.

7.26.3.7 usesCustomScript

```
bool ModuleActivities.usesCustomScript = false
```

Definition at line 11 of file ModuleActivities.cs.

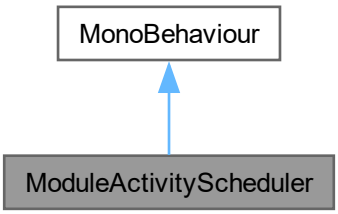
The documentation for this class was generated from the following file:

- Assets/Scripts/ModuleStructure/ModuleActivities.cs

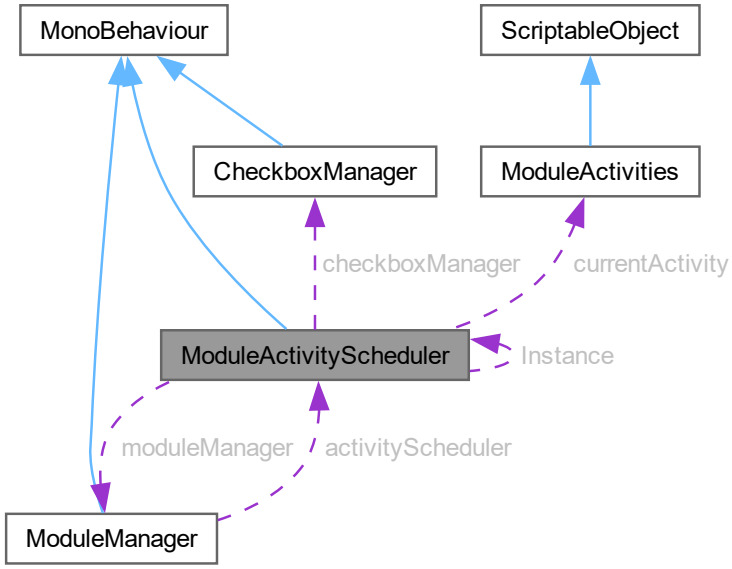
7.27 ModuleActivityScheduler Class Reference

Singleton management class that talks to the ModuleManager to schedule activities ahead of Timeline endings, forming the interactive components of each section that make up a Module.

Inheritance diagram for ModuleActivityScheduler:



Collaboration diagram for ModuleActivityScheduler:



Public Member Functions

- void StartActivity (ModuleActivities newActivity, ModuleManager manager)

  Called by ModuleManager when a new activity starts, to setup UI and controller inputs/activity scripts.
- void OnExternalStepCompleted ()

  Public-facing method to complete a step, to be called by activities like RotateLeftController.

Static Public Attributes

- static ModuleActivityScheduler Instance

  Singleton accessor reference.

Private Member Functions

- void Awake ()

  The setup for the Singleton design pattern implementation occurs when the GameObject this script is attached to is instantiated in the scene at runtime.
- void SetupActivityUI ()

  Calls the checkboxManager to setup the special checbox UI for each activity, passing the relevant instructions.
- void DisplayCurrentStep ()

  Displays the activity text as the header of the checkbox panel in the UI.
- void OnStepCompleted ()

  When an activity's current step is completed, the MAS moves onto the next step if applicable.
- void CompleteActivity ()

  When an activity is completed, turn the UI checkbox panel off, cleanup flags, and call the moduleManager to move onto the next timeline.

Private Attributes

- GameObject checkboxPanel

  The UI panel that contains all of the checkboxes/instructions/title.
- CheckboxManager checkboxManager

  Manages toggling checkboxes and descriptions.
- TextMeshProUGUI activityTitle

  Instruction title (e.g.
- ModuleManager moduleManager

  Reference to the ModuleManager.
- ModuleActivities currentActivity

  The activity that is currently running.
- int currentStepIndex = 0

  The current step index in the activity.
- bool activityInProgress = false

  Flag to determine if an activity is currently in progress or not.
- GameObject activeControllerObject

  The spawned ModuleActivities prefab for the current activity (if applicable).

7.27.1 Detailed Description

Singleton management class that talks to the ModuleManager to schedule activities ahead of Timeline endings, forming the interactive components of each section that make up a Module.

Author

Caleb Martin | marcy066@mymail.unisa.edu.au

Definition at line 11 of file ModuleActivityScheduler.cs.

7.27.2 Member Function Documentation

7.27.2.1 Awake()

```
void ModuleActivityScheduler.Awake ()  [private]
```

The setup for the Singleton design pattern implementation occurs when the GameObject this script is attached to is instantiated in the scene at runtime.

Returns

void

Definition at line 30 of file ModuleActivityScheduler.cs.

7.27.2.2 CompleteActivity()

```
void ModuleActivityScheduler.CompleteActivity ()  [private]
```

When an activity is completed, turn the UI checkbox panel off, cleanup flags, and call the moduleManager to move onto the next timeline.

Returns

void

Definition at line 145 of file ModuleActivityScheduler.cs.

7.27.2.3 DisplayCurrentStep()

```
void ModuleActivityScheduler.DisplayCurrentStep ()  [private]
```

Displays the activity text as the header of the checkbox panel in the UI.

Returns

void

Definition at line 95 of file ModuleActivityScheduler.cs.

7.27.2.4 OnExternalStepCompleted()

```
void ModuleActivityScheduler.OnExternalStepCompleted ()
```

Public-facing method to complete a step, to be called by activities like RotateLeftController.

Returns

void

Definition at line 135 of file ModuleActivityScheduler.cs.

7.27.2.5 OnStepCompleted()

```
void ModuleActivityScheduler.OnStepCompleted ()  [private]
```

When an activity's current step is completed, the MAS moves onto the next step if applicable.

If there are no more steps, we move onto the following timeline instead by calling the moduleManager to proceed with relevant logic.

Returns

void

Definition at line 112 of file ModuleActivityScheduler.cs.

7.27.2.6 SetupActivityUI()

```
void ModuleActivityScheduler.SetupActivityUI ()  [private]
```

Calls the checkboxManager to setup the special checbox UI for each activity, passing the relevant instructions.

Returns

void

Definition at line 86 of file ModuleActivityScheduler.cs.

7.27.2.7 StartActivity()

```
void ModuleActivityScheduler.StartActivity (
            ModuleActivities newActivity,
            ModuleManager manager)
```

Called by ModuleManager when a new activity starts, to setup UI and controller inputs/activity scripts.

Parameters

newActivity	The activity to commence.
manager	A reference to the ModuleManager.

Returns

void

Definition at line 48 of file ModuleActivityScheduler.cs.

7.27.3 Member Data Documentation

7.27.3.1 activeControllerObject

```
GameObject ModuleActivityScheduler.activeControllerObject  [private]
```

The spawned ModuleActivities prefab for the current activity (if applicable).

Definition at line 22 of file ModuleActivityScheduler.cs.

7.27.3.2 activityInProgress

```
bool ModuleActivityScheduler.activityInProgress = false  [private]
```

Flag to determine if an activity is currently in progress or not.

Definition at line 21 of file ModuleActivityScheduler.cs.

7.27.3.3 activityTitle

```
TextMeshProUGUI ModuleActivityScheduler.activityTitle  [private]
```

Instruction title (e.g.

"Increase Pitch", "Increase Power", etc.).

Definition at line 15 of file ModuleActivityScheduler.cs.

7.27.3.4 checkboxManager

```
CheckboxManager ModuleActivityScheduler.checkboxManager  [private]
```

Manages toggling checkboxes and descriptions.

Definition at line 14 of file ModuleActivityScheduler.cs.

7.27.3.5 checkboxPanel

```
GameObject ModuleActivityScheduler.checkboxPanel  [private]
```

The UI panel that contains all of the checkboxes/instructions/title.

Definition at line 13 of file ModuleActivityScheduler.cs.

7.27.3.6 currentActivity

```
ModuleActivities ModuleActivityScheduler.currentActivity  [private]
```

The activity that is currently running.

Definition at line 19 of file ModuleActivityScheduler.cs.

7.27.3.7 currentStepIndex

```
int ModuleActivityScheduler.currentStepIndex = 0  [private]
```

The current step index in the activity.

Definition at line 20 of file ModuleActivityScheduler.cs.

7.27.3.8 Instance

```
ModuleActivityScheduler ModuleActivityScheduler.Instance  [static]
```

Singleton accessor reference.

Definition at line 24 of file ModuleActivityScheduler.cs.

7.27.3.9 moduleManager

```
ModuleManager ModuleActivityScheduler.moduleManager  [private]
```

Reference to the ModuleManager.

Definition at line 17 of file ModuleActivityScheduler.cs.

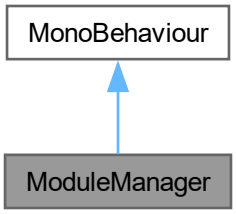
The documentation for this class was generated from the following file:

- Assets/Scripts/ModuleStructure/ModuleActivityScheduler.cs

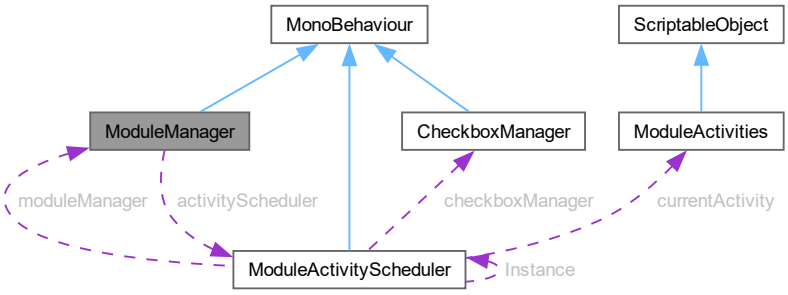
7.28 ModuleManager Class Reference

Central driver class to control the flow of logic on which Modules/sections/timelines/activities should play, and in sequence.

Inheritance diagram for ModuleManager:



Collaboration diagram for ModuleManager:



Classes

- class Module

  Modules are the collections of sections that form learning content within the application.
- class Section

  Sections are combinations of timelines and activities that form a Module.

Public Member Functions

- void OnActivityComplete ()

  Called by the ModuleActivityScheduler when the current activity is completed, starting the next timeline.
- void PlayModule (int moduleIndex)

  Play a specific Module instead of beginning from the first one.

Public Attributes

- List< Module > modules = new List<Module>()

  A list of Modules that form the application as a whole.

Private Member Functions

- void Start ()
- void PlayCurrentTimeline ()

  Plays the timeline in the currently-selected section.
- void OnTimelineFinished (PlayableDirector director)

  When a timeline finishes, this method is called to check for activities to determine what to play next.
- void StartActivity (ModuleActivities activity)

  Communicate with the ModuleActivityScheduler to initiate a passed-in activity.
- void NextTimeline ()

  Progress to the next timeline, skipping instead to the next Section if there are no more timelines in this Section.
- void NextSection ()

  Progress to the next Section, skipping instead to the next Module if there are no more Sections in this Module.
- void NextModule ()

  Progress to the next Module.

Private Attributes

- ModuleActivityScheduler activityScheduler

  Reference to the ModuleActivityScheduler singleton.
- int currentModuleIndex = 0

  The current Module index.
- int currentSectionIndex = 0

  The current Section index.
- int currentTimelineIndex = 0

  The current timeline index.
- PlayableDirector activeDirector

  Reference to the current timeline.
- bool waitingForActivity = false

  Flag that determines if an activity is currently playing or not, used to yield moving to the next timeline in the Section.

7.28.1 Detailed Description

Central driver class to control the flow of logic on which Modules/sections/timelines/activities should play, and in sequence.

Author

Caleb Martin | marcy066@mymail.unisa.edu.au

Definition at line 13 of file ModuleManager.cs.

7.28.2 Member Function Documentation

7.28.2.1 NextModule()

```
void ModuleManager.NextModule () [private]
```

Progress to the next Module.

If there are no more Modules, we have reached the end of the learning content.

Returns

void

Definition at line 180 of file ModuleManager.cs.

7.28.2.2 NextSection()

```
void ModuleManager.NextSection () [private]
```

Progress to the next Section, skipping instead to the next Module if there are no more Sections in this Module.

Returns

void

Definition at line 161 of file ModuleManager.cs.

7.28.2.3 NextTimeline()

```
void ModuleManager.NextTimeline () [private]
```

Progress to the next timeline, skipping instead to the next Section if there are no more timelines in this Section.

Returns

void

Definition at line 143 of file ModuleManager.cs.

7.28.2.4 OnActivityComplete()

```
void ModuleManager.OnActivityComplete ()
```

Called by the ModuleActivityScheduler when the current activity is completed, starting the next timeline.

Returns

void

Definition at line 133 of file ModuleManager.cs.

7.28.2.5 OnTimelineFinished()

```
void ModuleManager.OnTimelineFinished (
            PlayableDirector director)  [private]
```

When a timeline finishes, this method is called to check for activities to determine what to play next.

Parameters

director	Passed here to ensure that an unsubscription to the event is performed, mitigating event stacking.

Returns

void

Definition at line 78 of file ModuleManager.cs.

7.28.2.6 PlayCurrentTimeline()

```
void ModuleManager.PlayCurrentTimeline ()  [private]
```

Plays the timeline in the currently-selected section.

Returns

void

Definition at line 55 of file ModuleManager.cs.

7.28.2.7 PlayModule()

```
void ModuleManager.PlayModule (
            int moduleIndex)
```

Play a specific Module instead of beginning from the first one.

Parameters

moduleIndex	The selected Module to play.

Returns

void

Definition at line 201 of file ModuleManager.cs.

7.28.2.8 Start()

```
void ModuleManager.Start ()  [private]
```

Definition at line 46 of file ModuleManager.cs.

7.28.2.9 StartActivity()

```
void ModuleManager.StartActivity (
            ModuleActivities activity)  [private]
```

Communicate with the ModuleActivityScheduler to initiate a passed-in activity.

Parameters

activity	The activity to be commenced.

Returns

void

Definition at line 116 of file ModuleManager.cs.

7.28.3 Member Data Documentation

7.28.3.1 activeDirector

```
PlayableDirector ModuleManager.activeDirector  [private]
```

Reference to the current timeline.

Definition at line 42 of file ModuleManager.cs.

7.28.3.2 activityScheduler

```
ModuleActivityScheduler ModuleManager.activityScheduler  [private]
```

Reference to the ModuleActivityScheduler singleton.

Definition at line 36 of file ModuleManager.cs.

7.28.3.3 currentModuleIndex

```
int ModuleManager.currentModuleIndex = 0  [private]
```

The current Module index.

Definition at line 39 of file ModuleManager.cs.

7.28.3.4 currentSectionIndex

```
int ModuleManager.currentSectionIndex = 0  [private]
```

The current Section index.

Definition at line 40 of file ModuleManager.cs.

7.28.3.5 currentTimelineIndex

```
int ModuleManager.currentTimelineIndex = 0  [private]
```

The current timeline index.

Definition at line 41 of file ModuleManager.cs.

7.28.3.6 modules

```
List<Module> ModuleManager.modules = new List<Module>()
```

A list of Modules that form the application as a whole.

Definition at line 37 of file ModuleManager.cs.

7.28.3.7 waitingForActivity

```
bool ModuleManager.waitingForActivity = false  [private]
```

Flag that determines if an activity is currently playing or not, used to yield moving to the next timeline in the Section.

Definition at line 44 of file ModuleManager.cs.

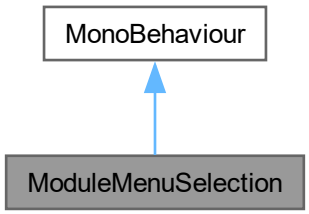
The documentation for this class was generated from the following file:

- Assets/Scripts/ModuleStructure/ModuleManager.cs

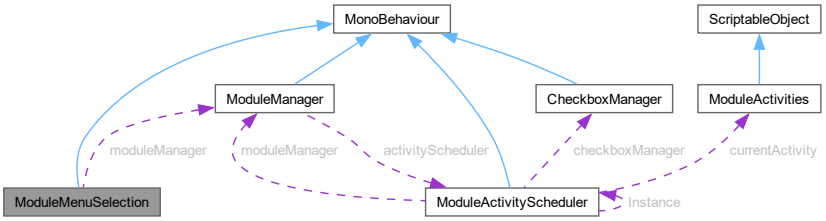
7.29 ModuleMenuSelection Class Reference

Handles menu selection of each Module alongside a Play button to trigger Modules via the ModuleManager.

Inheritance diagram for ModuleMenuSelection:



Collaboration diagram for ModuleMenuSelection:



Public Attributes

- Button playButton

  The interactible Play button GameObject.
- Button module1Button

  The button for the first Module.
- Button module2Button

  The button for the second Module.
- Button module3Button

  The button for the third Module.
- Button backButton

  The interactible Back button GameObject.

Private Member Functions

- void Start ()
- void SelectModule (int moduleIndex)

  Changes the currently selected Module to a new one.
- void OnPlayPressed ()

  Called by the Play button, calls the ModuleManager to initiate the selected Module.
- void OnBackPressed ()

  When the Back button is pressed, reset the selected Module back to -1 as default.

Private Attributes

- ModuleManager moduleManager

  Reference to the ModuleManager.
- int selectedModule = -1

  Index for the currently selected Module, defaulting to -1 where no Module is selected.

7.29.1 Detailed Description

Handles menu selection of each Module alongside a Play button to trigger Modules via the ModuleManager.

Author

Caleb Martin | marcy066@mymail.unisa.edu.au

Definition at line 11 of file ModuleMenuSelection.cs.

7.29.2 Member Function Documentation

7.29.2.1 OnBackPressed()

```
void ModuleMenuSelection.OnBackPressed ()  [private]
```

When the Back button is pressed, reset the selected Module back to -1 as default.

Returns

void

Definition at line 76 of file ModuleMenuSelection.cs.

7.29.2.2 OnPlayPressed()

```
void ModuleMenuSelection.OnPlayPressed ()  [private]
```

Called by the Play button, calls the ModuleManager to initiate the selected Module.

Returns

void

Definition at line 58 of file ModuleMenuSelection.cs.

7.29.2.3 SelectModule()

```
void ModuleMenuSelection.SelectModule (
            int moduleIndex)  [private]
```

Changes the currently selected Module to a new one.

Parameters

moduleIndex	The Module to be selected.

Returns

void

Definition at line 45 of file ModuleMenuSelection.cs.

7.29.2.4 Start()

```
void ModuleMenuSelection.Start ()  [private]
```

Definition at line 25 of file ModuleMenuSelection.cs.

7.29.3 Member Data Documentation

7.29.3.1 backButton

```
Button ModuleMenuSelection.backButton
```

The interactible Back button GameObject.

Definition at line 18 of file ModuleMenuSelection.cs.

7.29.3.2 module1Button

```
Button ModuleMenuSelection.module1Button
```

The button for the first Module.

Definition at line 15 of file ModuleMenuSelection.cs.

7.29.3.3 module2Button

```
Button ModuleMenuSelection.module2Button
```

The button for the second Module.

Definition at line 16 of file ModuleMenuSelection.cs.

7.29.3.4 module3Button

```
Button ModuleMenuSelection.module3Button
```

The button for the third Module.

Definition at line 17 of file ModuleMenuSelection.cs.

7.29.3.5 moduleManager

```
ModuleManager ModuleMenuSelection.moduleManager  [private]
```

Reference to the ModuleManager.

Definition at line 21 of file ModuleMenuSelection.cs.

7.29.3.6 playButton

```
Button ModuleMenuSelection.playButton
```

The interactible Play button GameObject.

Definition at line 14 of file ModuleMenuSelection.cs.

7.29.3.7 selectedModule

```
int ModuleMenuSelection.selectedModule = -1  [private]
```

Index for the currently selected Module, defaulting to -1 where no Module is selected.

Definition at line 23 of file ModuleMenuSelection.cs.

The documentation for this class was generated from the following file:

- Assets/Scripts/UI/ModuleMenuSelection.cs

7.30 PlaneHighlighter Class Reference

Handles highlighting of specific components of the aircraft.

Static Public Member Functions

- static void Highlight (IEnumerable< GameObject > components, Color glowColour)
  Highlights a collection of GameObjects a determined colour using emission.
- static void Unhighlight (IEnumerable< GameObject > components)
  Uh-highlights a collection of GameObjects by disabling their emission and resetting their materials' properties.

7.30.1 Detailed Description

Handles highlighting of specific components of the aircraft.

Works by duplicating, modifying, and then assigning a new emissive material to the selected GameObjects. Typically called from DA_40.cs.

Author

Caleb Martin | marcy066@mymail.unisa.edu.au

Definition at line 10 of file PlaneComponentHighlighter.cs.

7.30.2 Member Function Documentation

7.30.2.1 Highlight()

```
void PlaneHighlighter.Highlight (
            IEnumerable< GameObject > components,
            Color glowColour)  [static]
```

Highlights a collection of GameObjects a determined colour using emission.

Parameters

components	The collection of GameObjects to be highlighted.
glowColour	The the colour that the selected GameObjects should glow.

Returns

void

Definition at line 18 of file PlaneComponentHighlighter.cs.

7.30.2.2 Unhighlight()

```
void PlaneHighlighter.Unhighlight (
            IEnumerable< GameObject > components)  [static]
```

Uh-highlights a collection of GameObjects by disabling their emission and resetting their materials' properties.

Parameters

components	The collection of GameObjects to be reverted to a non-emissive state.

Returns

void

Definition at line 47 of file PlaneComponentHighlighter.cs.

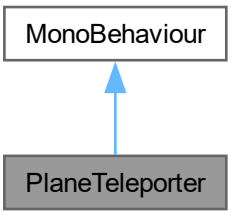
The documentation for this class was generated from the following file:

- Assets/Scripts/DA_40_SpecialFeatures/PlaneComponentHighlighter.cs

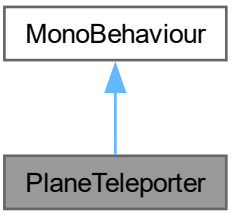
7.31 PlaneTeleporter Class Reference

Helper to handle translating the DA-40 aircraft to and from the viewing platform in a Timeline.

Inheritance diagram for PlaneTeleporter:



Collaboration diagram for PlaneTeleporter:



Public Member Functions

- void TeleportToTarget ()

  Performs teleportation on the GameObject this script is attached to, towards targetPosition.

Public Attributes

- Vector3 targetPosition

  The target position the aircraft should move to.

7.31.1 Detailed Description

Helper to handle translating the DA-40 aircraft to and from the viewing platform in a Timeline.

Author

Caleb Martin | marcy066@mymail.unisa.edu.au

Definition at line 9 of file MovePlaneToPlatform.cs.

7.31.2 Member Function Documentation

7.31.2.1 TeleportToTarget()

```
void PlaneTeleporter.TeleportToTarget ()
```

Performs teleportation on the GameObject this script is attached to, towards targetPosition.

Returns

void

Definition at line 17 of file MovePlaneToPlatform.cs.

7.31.3 Member Data Documentation

7.31.3.1 targetPosition

```
Vector3 PlaneTeleporter.targetPosition
```

The target position the aircraft should move to.

Definition at line 11 of file MovePlaneToPlatform.cs.

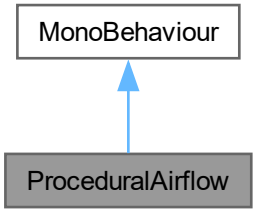
The documentation for this class was generated from the following file:

- Assets/Scripts/MiscBehaviour/MovePlaneToPlatform.cs

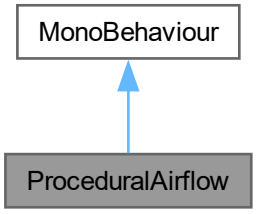
7.32 ProceduralAirflow Class Reference

Airflow, Force Arrows and COP Behaviour.

Inheritance diagram for ProceduralAirflow:



Collaboration diagram for ProceduralAirflow:



Public Attributes

- bool enableParticleSystem = false

  Enable or disable procedural airflow.
- bool enableForceArrows = false

  Enable or disable force arrows.
- bool enableCentreOfPressure = false

  Enable or disable centre of pressure arrow.
- Transform leadingEdge

  Transform located at wings leading edge.
- Transform trailingEdge

  Transform located at wings trailing edge.
- Transform liftAnchor

  Tranfrom for lift arrow generation.
- Transform weightAnchor

  Transform for weight arrow generation.
- Transform thrusAnchor

  Transform for thrust arrow generation.
- Transform dragAnchor

  Transform for drag arrow generation.
- float defaultArrowSize = 1f

  Default size of force arrows.
- Material forceArrowMaterial

  Material for force arrows.
- GameObject arrowHeadModel

  Arrow head game object for force arrows.
- GameObject copModel

  Prefab for centre of pressure indicator.
- ParticleSystem ps

  Airflow particle system, if null script generates new particle system.
- float emissionRate = 20f

  Procedural Airflow, particles per second.
- float forwardSpawnDistance = 0.5f

  Percentage of.
- float particleSize = 0.05f

  Size of particles.
- int maxParticles = 1000

  Max number of particles from particle system.
- float topMaxSpeed = 2f

  Max speed for particles on top of the wing.
- float topMinSpeed = 1.5f

  Min speed for particles on top of the wing.
- float bottomMaxSpeed = 1f

  Max speed for particles below the wing.
- float bottomMinSpeed = 0.5f

  Min speed for particles below the wing.
- float airSeperationStrength = 0.5f

  Multiplies y velocity of top particles to reduce airflow stick to wing chord at higher AOA.
- float trailDownForcePercent = 0.2f

  Percent of particle y direction to be maintained after trailing edge.
- bool useFixedDeltaInEditor = true

  Enable ariflow in the editor.
- float editorDelta = 1f / 60f

  60fps deltatime for editor sim

Private Member Functions

- void Awake ()

  Setup Particle System on awake.
- void InitializeParticleSystem ()

  Setup particle system for procedural airflow.
- void Update ()

  Main update loop.
- void EmitParticles (float dt)

  Manual Airflow Particle Emission.
- void UpdateParticles (float dt)

  Manual Airflow Particle Update.
- void UpdateForceArrows ()

  Updates four force arrows.
- float GetLiftLength (float aoa)

  Returns lift arrow length as a multiplier.
- float GetWeightLength (float aoa)

  Returns weight arrow length as a constant multiplier.
- float GetThrustLength (float aoa)

  Returns thrust arrow length as a multiplier.
- float GetDragLength (float aoa)

  Returns drag arrow length as a multiplier.
- float InterpolateForceValue (float aoa, float[ ] forceValues)

  Interpolates force coefficient values based on angle of attack.
- void SetArrow (GameObject arrow, Vector3 basePos, Vector3 dir, float length)

  Configures arrow transform properties.
- void CreateForceArrows ()

  Creates all four force arrow game objects.
- GameObject CreateArrow (string name, Color colour)

  Creates a single force arrow game object.
- void DestroyPreviousArrows ()

  Destroys all existing force arrow game objects.
- void updateCentreOfPressure ()

  Updates centre of pressure indicator position.
- GameObject createCopIndicator (string name, Color colour)

  Creates centre of pressure indicator game object.
- float getCopPercentage (float aoa)

  Returns centre of pressure position as percentage of chord.
- void destroyCentreOfPressure ()

  Destroys the centre of pressure indicator game object.
- void OnDrawGizmos ()

  Draws editor gizmos for visualization.

Private Attributes

- ParticleSystem.Particle[ ] buffer

  Array of particles from the particle system.
- List< Vector3 > velocities = new List<Vector3>()

  Parallel List to buffer to store individual velocities.
- List< bool > sideFlags = new List<bool>()

Parallel List to buffer to store side flags, true = above wing, falase = below wing.

- float emissionsNextFrame = 0f

  Holds emmisions required per frame.
- float rotation = 0f

  z rotation for arrows update loop
- GameObject liftArrow

  Instance of the lift force arrow game object.
- GameObject weightArrow

  Instance of the weight force arrow game object.
- GameObject thrustArrow

  Instance of the thrust force arrow game object.
- GameObject dragArrow

  Instance of the drag force arrow game object.
- GameObject copGameObject

  cop arrow insatnce

Static Private Attributes

- static float[ ] aoaStages = {3f, 7f, 10f, 14f, 16f, 18f}

  Predefined aoa stages for Force arrow and COP calc.
- static float[ ] liftValues = { 1.0f, 1.2f, 1.5f, 1.7f, 1.6f, 0.8f }

  Parallel array to aoaStages, stores mmultipliers to scale liftArrow.
- static float[ ] dragValues = { 1.0f, 1.2f, 1.5f, 1.7f, 2.0f, 2.5f }

  Parallel array to aoaStages, stores multipliers to scale dragArrow.
- static float[ ] thrustValues = { 1.0f, 1.2f, 1.5f, 1.6f, 1.0f, 0.6f }

  Parallel array to aoaStages, stores multipliers to scale thrustArrow.
- static float[ ] copValues = { 0.25f, 0.24f, 0.22f, 0.20f, 0.20f, 0.45f }

  Parallel array to aoaStages, stores multipliers to move cop arrow along plane chord.

7.32.1 Detailed Description

Airflow, Force Arrows and COP Behaviour.

Class contains definitions and behaviour for

- Procedural Airflow
- Force Arrows
- Centre of Pressure indicator

Current implementaiton of both Fore Arrows and COP indicator utilize hardcoded value that reflect the provided values for Module 2 Section 1 Procedural airflow simulation is more versitile and allows for dynamic modificaitons to behaviour.

Author

Connor Freebairn | frecd002@mymail.unisa.edu.au

Definition at line 18 of file ProceduralAirflow.cs.

7.32.2 Member Function Documentation

7.32.2.1 Awake()

```
void ProceduralAirflow.Awake ()  [private]
```

Setup Particle System on awake.

Calls InitializeParticleSystem() and declares new Particle ::buffer with size of the particle systems internal buffer.

Definition at line 101 of file ProceduralAirflow.cs.

7.32.2.2 CreateArrow()

```
GameObject ProceduralAirflow.CreateArrow (
            string name,
            Color colour)  [private]
```

Creates a single force arrow game object.

Instantiates a cube primitive as the arrow shaft and attaches an arrow head model as a child. Assigns the specified color to both shaft and head materials.

Parameters

name	Base name for the arrow (e.g. "Lift", "Drag")
colour	Color to apply to the arrow material

Returns

The created arrow game object

Definition at line 580 of file ProceduralAirflow.cs.

7.32.2.3 createCopIndicator()

```
GameObject ProceduralAirflow.createCopIndicator (
            string name,
            Color colour)  [private]
```

Creates centre of pressure indicator game object.

Destroys any existing COP indicator and instantiates a new one from the COP model prefab with the specified name and color.

Parameters

name	Name to assign to the COP indicator game object

colour	Color to apply to the indicator (currently unused in implementation)

Returns

The created COP indicator game object

```
Renderer rend = cop.GetComponent<Renderer>();
```

Handle material leaks during edit mode if (Application.isPlaying) rend.material.color = colour; else rend.shared↩ Material.color = colour;

Definition at line 683 of file ProceduralAirflow.cs.

7.32.2.4 CreateForceArrows()

```
void ProceduralAirflow.CreateForceArrows ()  [private]
```

Creates all four force arrow game objects.

Destroys any existing arrows and instantiates new lift, weight, thrust, and drag arrows with their respective colors.

Definition at line 560 of file ProceduralAirflow.cs.

7.32.2.5 destroyCentreOfPressure()

```
void ProceduralAirflow.destroyCentreOfPressure ()  [private]
```

Destroys the centre of pressure indicator game object.

Searches for and destroys the COP indicator if it exists in the scene. Uses DestroyImmediate in edit mode and Destroy in play mode.

Definition at line 733 of file ProceduralAirflow.cs.

7.32.2.6 DestroyPreviousArrows()

```
void ProceduralAirflow.DestroyPreviousArrows ()  [private]
```

Destroys all existing force arrow game objects.

Searches for and destroys any existing lift, weight, thrust, or drag arrows in the scene. Uses DestroyImmediate in edit mode and Destroy in play mode.

Definition at line 621 of file ProceduralAirflow.cs.

7.32.2.7 EmitParticles()

```
void ProceduralAirflow.EmitParticles (
            float dt)  [private]
```

Manual Airflow Particle Emission.

Spawns particles at a fixed rate and assigns initial ::velocities and ::sideFlags values.

Parameters

dt	deltatime, fixed value for editor or actual suring play

Definition at line 213 of file ProceduralAirflow.cs.

7.32.2.8 getCopPercentage()

```
float ProceduralAirflow.getCopPercentage (
            float aoa)  [private]
```

Returns centre of pressure position as percentage of chord.

Calculates the COP position along the wing chord as a percentage from the leading edge based on angle of attack. Uses linear interpolation between predefined stages in ::aoaStages and ::copValues arrays.

Parameters

aoa	Current angle of attack (pitch) of the plane

Returns

Percentage distance from leading edge (0.0 = leading edge, 1.0 = trailing edge)

Definition at line 715 of file ProceduralAirflow.cs.

7.32.2.9 GetDragLength()

```
float ProceduralAirflow.GetDragLength (
            float aoa)  [private]
```

Returns drag arrow length as a multiplier.

Checks the current angle of attack against the values in ::aoaStages. Returns a multiplier for the arrow length via linear interpolation between the upper and lower bounds of the current aoa against the ::aoaStages.

Parameters

aoa	Current angle of attack (pitch) of the plane

Returns

Arrow length multiplier based on interpolated drag values

Definition at line 467 of file ProceduralAirflow.cs.

7.32.2.10 GetLiftLength()

```
float ProceduralAirflow.GetLiftLength (
            float aoa)  [private]
```

Returns lift arrow length as a multiplier.

Checks the current angle of attack against the values in ::aoaStages. Returns a multiplier for the arrow length via linear interpolation between the upper and lower bounds of the current aoa against the ::aoaStages.

Parameters

aoa	Current angle of attack (pitch) of the plane

Returns

Arrow length multiplier based on interpolated lift values

Definition at line 423 of file ProceduralAirflow.cs.

7.32.2.11 GetThrustLength()

```
float ProceduralAirflow.GetThrustLength (
            float aoa)  [private]
```

Returns thrust arrow length as a multiplier.

Checks the current angle of attack against the values in ::aoaStages. Returns a multiplier for the arrow length via linear interpolation between the upper and lower bounds of the current aoa against the ::aoaStages.

Parameters

aoa	Current angle of attack (pitch) of the plane

Returns

Arrow length multiplier based on interpolated thrust values

Definition at line 452 of file ProceduralAirflow.cs.

7.32.2.12 GetWeightLength()

```
float ProceduralAirflow.GetWeightLength (
            float aoa)  [private]
```

Returns weight arrow length as a constant multiplier.

Weight is constant and does not vary with angle of attack, so returns the base arrow size without any scaling factor applied.

Parameters

aoa	Current angle of attack (pitch) of the plane (unused)

Returns

Constant arrow length equal to defaultArrowSize

Definition at line 437 of file ProceduralAirflow.cs.

7.32.2.13 InitializeParticleSystem()

```
void ProceduralAirflow.InitializeParticleSystem ()  [private]
```

Setup particle system for procedural airflow.

If particle system parameter is null generate a new particle system and set relevant variables to ensure compatability with existing functions.

- EmitParticles()
- UpdateParticles()

Definition at line 114 of file ProceduralAirflow.cs.

7.32.2.14 InterpolateForceValue()

```
float ProceduralAirflow.InterpolateForceValue (
            float aoa,
            float[] forceValues)  [private]
```

Interpolates force coefficient values based on angle of attack.

Performs linear interpolation between force coefficient values based on the current angle of attack. The interpolation factor (t) is calculated as: t = (aoa - aoaStages[i]) / (aoaStages[i+1] - aoaStages[i]) where t ranges from 0 (at lower bound) to 1 (at upper bound).

For aoa values outside the defined ::aoaStages range, returns the minimum or maximum force value without interpolation.

Parameters

aoa	Current angle of attack (pitch) of the plane
forceValues	Array of force coefficient values corresponding to ::aoaStages

Returns

Interpolated force coefficient value

Definition at line 487 of file ProceduralAirflow.cs.

7.32.2.15 OnDrawGizmos()

```
void ProceduralAirflow.OnDrawGizmos ()  [private]
```

Draws editor gizmos for visualization.

Renders visual guides in the Unity editor showing the wing chord line (yellow), top airflow guide (cyan), and bottom airflow guide (magenta) at the chord midpoint.

Definition at line 750 of file ProceduralAirflow.cs.

7.32.2.16 SetArrow()

```
void ProceduralAirflow.SetArrow (
            GameObject arrow,
            Vector3 basePos,
            Vector3 dir,
            float length)  [private]
```

Configures arrow transform properties.

Sets the scale, rotation, and position of a force arrow game object. The arrow is oriented to point in the specified direction and positioned so its base sits at the anchor point. Also scales and positions the arrow head child object.

Parameters

arrow	The arrow game object to configure
basePos	World position for the base of the arrow
dir	Direction vector the arrow should point
length	Total length of the arrow in world units

Definition at line 524 of file ProceduralAirflow.cs.

7.32.2.17 Update()

```
void ProceduralAirflow.Update ()  [private]
```

Main update loop.

Handles Airflow particle, Force Arrows and COP indicator rendering. Uses fake delta time for editor rendering.

Definition at line 150 of file ProceduralAirflow.cs.

7.32.2.18 updateCentreOfPressure()

```
void ProceduralAirflow.updateCentreOfPressure ()  [private]
```

Updates centre of pressure indicator position.

Creates the COP indicator if it doesn't exist, then calculates its position along the wing chord based on the current angle of attack. Positions the indicator below the chord at the calculated percentage distance from the leading edge.

Definition at line 647 of file ProceduralAirflow.cs.

7.32.2.19 UpdateForceArrows()

```
void ProceduralAirflow.UpdateForceArrows ()  [private]
```

Updates four force arrows.

Check if arrows exist and recreates if not. Sets each arrows position and scale

Definition at line 391 of file ProceduralAirflow.cs.

7.32.2.20 UpdateParticles()

```
void ProceduralAirflow.UpdateParticles (
            float dt)  [private]
```

Manual Airflow Particle Update.

Determines position of each partiicle in relation to key positions along the chord. Updates particles velocity to coform to the chord of the wing. Particles with the ::sideFlags bool velue of

- False, move along the winf base
- True, move at a reduced y velocity to replicate airflow separation.

Parameters

dt	deltatime, fixed value for editor or actual suring play

Definition at line 278 of file ProceduralAirflow.cs.

7.32.3 Member Data Documentation

7.32.3.1 airSeperationStrength

```
float ProceduralAirflow.airSeperationStrength = 0.5f
```

Multiplies y velocity of top particles to reduce airflow stick to wing chord at higher AOA.

Definition at line 65 of file ProceduralAirflow.cs.

7.32.3.2 aoaStages

```
float [] ProceduralAirflow.aoaStages = {3f, 7f, 10f, 14f, 16f, 18f}  [static], [private]
```

Predefined aoa stages for Force arrow and COP calc.

Definition at line 82 of file ProceduralAirflow.cs.

7.32.3.3 arrowHeadModel

```
GameObject ProceduralAirflow.arrowHeadModel
```

Arrow head game object for force arrows.

Definition at line 43 of file ProceduralAirflow.cs.

7.32.3.4 bottomMaxSpeed

```
float ProceduralAirflow.bottomMaxSpeed = 1f
```

Max speed for particles below the wing.

Definition at line 60 of file ProceduralAirflow.cs.

7.32.3.5 bottomMinSpeed

```
float ProceduralAirflow.bottomMinSpeed = 0.5f
```

Min speed for particles below the wing.

Definition at line 61 of file ProceduralAirflow.cs.

7.32.3.6 buffer

```
ParticleSystem.Particle [] ProceduralAirflow.buffer  [private]
```

Array of particles from the particle system.

Definition at line 71 of file ProceduralAirflow.cs.

7.32.3.7 copGameObject

```
GameObject ProceduralAirflow.copGameObject  [private]
```

cop arrow insatnce

Definition at line 93 of file ProceduralAirflow.cs.

7.32.3.8 copModel

```
GameObject ProceduralAirflow.copModel
```

Prefab for centre of pressure indicator.

Definition at line 46 of file ProceduralAirflow.cs.

7.32.3.9 copValues

```
float [] ProceduralAirflow.copValues = { 0.25f, 0.24f, 0.22f, 0.20f, 0.20f, 0.45f }  [static],
[private]
```

Parallel array to aoaStages, stores multipliers to move cop arrow along plane chord.

Definition at line 90 of file ProceduralAirflow.cs.

7.32.3.10 defaultArrowSize

```
float ProceduralAirflow.defaultArrowSize = 1f
```

Default size of force arrows.

Definition at line 41 of file ProceduralAirflow.cs.

7.32.3.11 dragAnchor

```
Transform ProceduralAirflow.dragAnchor
```

Transform for drag arrow generation.

Definition at line 37 of file ProceduralAirflow.cs.

7.32.3.12 dragArrow

```
GameObject ProceduralAirflow.dragArrow  [private]
```

Instance of the drag force arrow game object.

Definition at line 80 of file ProceduralAirflow.cs.

7.32.3.13 dragValues

```
float [] ProceduralAirflow.dragValues = { 1.0f, 1.2f, 1.5f, 1.7f, 2.0f, 2.5f }  [static],
[private]
```

Parallel array to aoaStages, stores multipliers to scale dragArrow.

Definition at line 86 of file ProceduralAirflow.cs.

7.32.3.14 editorDelta

```
float ProceduralAirflow.editorDelta = 1f / 60f
```

60fps deltatime for editor sim

Definition at line 68 of file ProceduralAirflow.cs.

7.32.3.15 emissionRate

```
float ProceduralAirflow.emissionRate = 20f
```

Procedural Airflow, particles per second.

Definition at line 51 of file ProceduralAirflow.cs.

7.32.3.16 emissionsNextFrame

```
float ProceduralAirflow.emissionsNextFrame = 0f  [private]
```

Holds emmisions required per frame.

Definition at line 74 of file ProceduralAirflow.cs.

7.32.3.17 enableCentreOfPressure

```
bool ProceduralAirflow.enableCentreOfPressure = false
```

Enable or disable centre of pressure arrow.

Definition at line 25 of file ProceduralAirflow.cs.

7.32.3.18 enableForceArrows

```
bool ProceduralAirflow.enableForceArrows = false
```

Enable or disable force arrows.

Definition at line 24 of file ProceduralAirflow.cs.

7.32.3.19 enableParticleSystem

```
bool ProceduralAirflow.enableParticleSystem = false
```

Enable or disable procedural airflow.

Definition at line 23 of file ProceduralAirflow.cs.

7.32.3.20 forceArrowMaterial

```
Material ProceduralAirflow.forceArrowMaterial
```

Material for force arrows.

Definition at line 42 of file ProceduralAirflow.cs.

7.32.3.21 forwardSpawnDistance

```
float ProceduralAirflow.forwardSpawnDistance = 0.5f
```

Percentage of.

Definition at line 52 of file ProceduralAirflow.cs.

7.32.3.22 leadingEdge

```
Transform ProceduralAirflow.leadingEdge
```

Transform located at wings leading edge.

Definition at line 29 of file ProceduralAirflow.cs.

7.32.3.23 liftAnchor

```
Transform ProceduralAirflow.liftAnchor
```

Tranfrom for lift arrow generation.

Definition at line 34 of file ProceduralAirflow.cs.

7.32.3.24 liftArrow

```
GameObject ProceduralAirflow.liftArrow  [private]
```

Instance of the lift force arrow game object.

Definition at line 77 of file ProceduralAirflow.cs.

7.32.3.25 liftValues

```
float [] ProceduralAirflow.liftValues = { 1.0f, 1.2f, 1.5f, 1.7f, 1.6f, 0.8f }  [static],
[private]
```

Parallel array to aoaStages, stores mmultipliers to scale liftArrow.

Definition at line 85 of file ProceduralAirflow.cs.

7.32.3.26 maxParticles

```
int ProceduralAirflow.maxParticles = 1000
```

Max number of particles from particle system.

Definition at line 54 of file ProceduralAirflow.cs.

7.32.3.27 particleSize

```
float ProceduralAirflow.particleSize = 0.05f
```

Size of particles.

Definition at line 53 of file ProceduralAirflow.cs.

7.32.3.28 ps

```
ParticleSystem ProceduralAirflow.ps
```

Airflow particle system, if null script generates new particle system.

Definition at line 50 of file ProceduralAirflow.cs.

7.32.3.29 rotation

```
float ProceduralAirflow.rotation = 0f  [private]
```

z rotation for arrows update loop

Definition at line 75 of file ProceduralAirflow.cs.

7.32.3.30 sideFlags

```
List<bool> ProceduralAirflow.sideFlags = new List<bool>()  [private]
```

Parallel List to buffer to store side flags, true = above wing, falase = below wing.

Definition at line 73 of file ProceduralAirflow.cs.

7.32.3.31 thrusAnchor

```
Transform ProceduralAirflow.thrusAnchor
```

Transform for thrust arrow generation.

Definition at line 36 of file ProceduralAirflow.cs.

7.32.3.32 thrustArrow

```
GameObject ProceduralAirflow.thrustArrow  [private]
```

Instance of the thrust force arrow game object.

Definition at line 79 of file ProceduralAirflow.cs.

7.32.3.33 thrustValues

```
float [] ProceduralAirflow.thrustValues = { 1.0f, 1.2f, 1.5f, 1.6f, 1.0f, 0.6f }  [static],
[private]
```

Parallel array to aoaStages, stores multipliers to scale thrustArrow.

Definition at line 87 of file ProceduralAirflow.cs.

7.32.3.34 topMaxSpeed

```
float ProceduralAirflow.topMaxSpeed = 2f
```

Max speed for particles on top of the wing.

Definition at line 58 of file ProceduralAirflow.cs.

7.32.3.35 topMinSpeed

```
float ProceduralAirflow.topMinSpeed = 1.5f
```

Min speed for particles on top of the wing.

Definition at line 59 of file ProceduralAirflow.cs.

7.32.3.36 trailDownForcePercent

```
float ProceduralAirflow.trailDownForcePercent = 0.2f
```

Percent of particle y direction to be maintained after trailing edge.

Definition at line 66 of file ProceduralAirflow.cs.

7.32.3.37 trailingEdge

```
Transform ProceduralAirflow.trailingEdge
```

Transform located at wings trailing edge.

Definition at line 30 of file ProceduralAirflow.cs.

7.32.3.38 useFixedDeltaInEditor

```
bool ProceduralAirflow.useFixedDeltaInEditor = true
```

Enable ariflow in the editor.

Definition at line 67 of file ProceduralAirflow.cs.

7.32.3.39 velocities

```
List<Vector3> ProceduralAirflow.velocities = new List<Vector3>()  [private]
```

Parallel List to buffer to store individual velocities.

Definition at line 72 of file ProceduralAirflow.cs.

7.32.3.40 weightAnchor

```
Transform ProceduralAirflow.weightAnchor
```

Transform for weight arrow generation.

Definition at line 35 of file ProceduralAirflow.cs.

7.32.3.41 weightArrow

```
GameObject ProceduralAirflow.weightArrow  [private]
```

Instance of the weight force arrow game object.

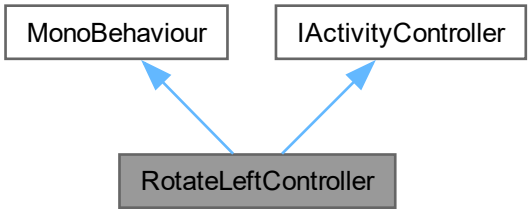
Definition at line 78 of file ProceduralAirflow.cs.

The documentation for this class was generated from the following file:

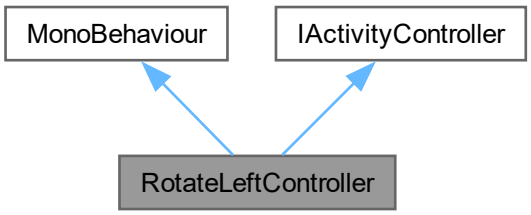
- Assets/Scripts/DA_40_SpecialFeatures/ProceduralAirflow.cs

7.33 RotateLeftController Class Reference

Inheritance diagram for RotateLeftController:



Collaboration diagram for RotateLeftController:



Public Member Functions

- void StartActivity ()

  Each child activity can have a differing implementation of what must be carried out/what inputs must be switched on when the activity starts.

Public Attributes

- InputActionReference rotateAction
- float rotationThreshold = 0.1f

Properties

- ModuleActivityScheduler mas `[get]`

Private Member Functions

- void OnEnable ()
- void OnDisable ()
- void OnRotatePerformed (InputAction.CallbackContext ctx)

  Rotate Method.

Private Attributes

- bool activityEnabled = false

7.33.1 Detailed Description

Author

Chintana Luu | luucy003@mymail.unisa.edu.au

Definition at line 7 of file RotateLeftController.cs.

7.33.2 Member Function Documentation

7.33.2.1 OnDisable()

```
void RotateLeftController.OnDisable ()  [private]
```

Definition at line 26 of file RotateLeftController.cs.

7.33.2.2 OnEnable()

```
void RotateLeftController.OnEnable ()  [private]
```

Definition at line 16 of file RotateLeftController.cs.

7.33.2.3 OnRotatePerformed()

```
void RotateLeftController.OnRotatePerformed (
             InputAction.CallbackContext ctx)  [private]
```

Rotate Method.

This method is subscribed onto the action performed in the OnEnable method and will execute when the controller is rotated.

and this is an extra line

Parameters

nothing	this is literally nothing
this	doesnt exist, literall the param this, does. not. exist.

Returns

no why would you think this returns something bozo

Definition at line 55 of file RotateLeftController.cs.

7.33.2.4 StartActivity()

```
void RotateLeftController.StartActivity ()
```

Each child activity can have a differing implementation of what must be carried out/what inputs must be switched on when the activity starts.

Override this method for new activity scripts.

Implements IActivityController.

Definition at line 36 of file RotateLeftController.cs.

7.33.3 Member Data Documentation

7.33.3.1 activityEnabled

```
bool RotateLeftController.activityEnabled = false  [private]
```

Definition at line 13 of file RotateLeftController.cs.

7.33.3.2 rotateAction

```
InputActionReference RotateLeftController.rotateAction
```

Definition at line 11 of file RotateLeftController.cs.

7.33.3.3 rotationThreshold

```
float RotateLeftController.rotationThreshold = 0.1f
```

Definition at line 12 of file RotateLeftController.cs.

7.33.4 Property Documentation

7.33.4.1 mas

```
ModuleActivityScheduler RotateLeftController.mas  [get], [private]
```

Definition at line 9 of file RotateLeftController.cs.

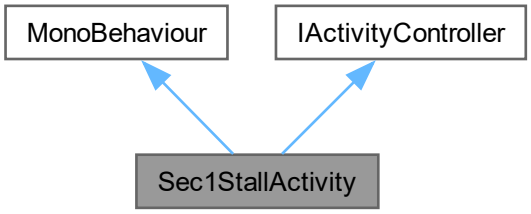
The documentation for this class was generated from the following file:

- Assets/Scripts/CommonActivityBehaviour/RotateLeftController.cs

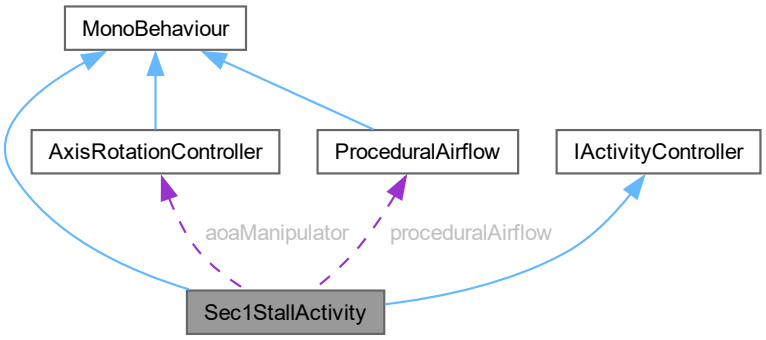
7.34 Sec1StallActivity Class Reference

Section 1 stall training activity controller.

Inheritance diagram for Sec1StallActivity:



Collaboration diagram for Sec1StallActivity:



Public Member Functions

- void OnEnable ()

  Initialize activity on component enable.
- void OnDisable ()

  Cleanup activity on component disable.
- void StartActivity ()

  Start the activity when triggered by ModuleActivityScheduler.
- void EnableCurrentStep ()

  Unlock input for the current training step.
- void HandlePitching ()

  Handle progressive pitch training step.

Public Attributes

- TimelineAsset[ ] pitchTimelineAssets

  Timeline assets for each pitch milestone (3, 7, 10, 14, 16, 18).
- InputActionReference rotationInput

  VR controller rotation input action for aircraft pitch control.
- float pitchSensitivity = 1f

  Pitch sensitivity multiplier for controller input.
- float deadZone = 15f

  Controller deadzone threshold in degrees to filter noise.
- float stepAdvanceTolerance = 3f

  Tolerance in degrees for achieving target pitch before advancing.
- bool useMouseClicks = false

  Enable mouse button control for testing without VR.
- float mouseRollSpeed = 10f

  Speed multiplier for mouse-based pitch control.

Properties

- ModuleActivityScheduler mas `[get]`

  Reference to singleton activity scheduler.

Private Types

- enum TurnStep { Pitch , Complete }

  Training progression states.

Private Member Functions

- void Update ()

  Main update loop.

Private Attributes

- Quaternion controllerOrigin = Quaternion.identity

  Initial controller orientation captured at step start.
- bool hasControllerOrigin = false

  Flag indicating if controller origin has been captured this step.
- float[ ] pitchTargets = { 3f, 7f, 10f, 14f, 16f, 18f }

  Progressive pitch angle targets in degrees leading to stall.
- int currentPitchTarget = 0

  Index of current pitch target in pitchTargets array.
- TurnStep currentStep

  Current training step in the activity sequence.
- bool inputLocked = false

  Flag to disable user input during transitions and timelines.
- float currentPitch = 0f

  Current aircraft pitch angle in degrees, updated each frame.
- PlayableDirector[ ] pitchTimelines

  PlayableDirectors corresponding to pitchTimelineAssets.
- AxisRotationController aoaManipulator

  Controller for aircraft pitch manipulation.
- ProceduralAirflow proceduralAirflow

  Airflow visualization system for force arrows, particles, and COP.

7.34.1 Detailed Description

Section 1 stall training activity controller.

Manages a progressive pitch exercise where users practice increasing angle of attack through six stages (3°, 7°, 10°, 14°, 16°, 18°) while observing aerodynamic effects. Enables real-time visualization of force arrows, airflow particles, and center of pressure to demonstrate the relationship between pitch angle and aerodynamic forces leading to stall. Implements the IActivityController interface for integration with the ModuleActivityScheduler system.

Training sequence:

1. Pitch - Progressive pitch targets with aerodynamic visualization
2. Complete - Activity finished, notifies scheduler

Key features:

- Real-time force arrow visualization (lift, weight, thrust, drag)
- Procedural airflow particle system
- Dynamic center of pressure indicator
- Progressive angle of attack demonstration leading to stall

Author

Connor Freebairn | frecd002@mymail.unisa.edu.au

Definition at line 28 of file Sec1StallActivity.cs.

7.34.2 Member Enumeration Documentation

7.34.2.1 TurnStep

```
enum Sec1StallActivity.TurnStep  [private]
```

Training progression states.

Defines the sequence of training steps in the stall activity.

Enumerator

Pitch	User practices progressive pitch angles with aerodynamic visualization.
Complete	Activity finished, awaiting cleanup.

Definition at line 55 of file Sec1StallActivity.cs.

7.34.3 Member Function Documentation

7.34.3.1 EnableCurrentStep()

```
void Sec1StallActivity.EnableCurrentStep ()
```

Unlock input for the current training step.

Resets input lock and controller origin flag to allow user control. Called after transitions and timeline completions.

Definition at line 135 of file Sec1StallActivity.cs.

7.34.3.2 HandlePitching()

```
void Sec1StallActivity.HandlePitching ()
```

Handle progressive pitch training step.

Captures controller origin on first frame, reads current pitch angle, and checks if user has achieved the current target angle. When target is reached, locks input, plays confirmation timeline, and advances to next pitch target or completion. The progression through six pitch angles (3, 7, 10, 14, 16, 18) demonstrates increasing aerodynamic forces and changing center of pressure as the aircraft approaches stall conditions. Allows controller or debug mouse input when target not yet achieved.

Aerodynamic visualizations update in real-time:

- Force arrows show changing lift, drag, thrust, and weight magnitudes
- Particle system shows airflow separation at higher angles
- Center of pressure indicator moves along chord as angle increases

Definition at line 177 of file Sec1StallActivity.cs.

7.34.3.3 OnDisable()

```
void Sec1StallActivity.OnDisable ()
```

Cleanup activity on component disable.

Disables input actions and deactivates all aerodynamic visualization features (force arrows, particle system, center of pressure indicator).

Definition at line 107 of file Sec1StallActivity.cs.

7.34.3.4 OnEnable()

```
void Sec1StallActivity.OnEnable ()
```

Initialize activity on component enable.

Enables input actions, resets progression state to initial values, retrieves references to aircraft controllers and airflow visualization system, and finds PlayableDirectors for all timeline assets. Activates all aerodynamic visualization features (force arrows, particle system, center of pressure indicator).

Definition at line 79 of file Sec1StallActivity.cs.

7.34.3.5 StartActivity()

```
void Sec1StallActivity.StartActivity ()
```

Start the activity when triggered by ModuleActivityScheduler.

Called by the activity scheduler to begin the training sequence. Unlocks input for the first step.

Implements IActivityController.

Definition at line 124 of file Sec1StallActivity.cs.

7.34.3.6 Update()

```
void Sec1StallActivity.Update ()  [private]
```

Main update loop.

Routes execution to appropriate handler based on current training step. Notifies activity scheduler upon completion.

Definition at line 147 of file Sec1StallActivity.cs.

7.34.4 Member Data Documentation

7.34.4.1 aoaManipulator

```
AxisRotationController Sec1StallActivity.aoaManipulator  [private]
```

Controller for aircraft pitch manipulation.

Definition at line 68 of file Sec1StallActivity.cs.

7.34.4.2 controllerOrigin

```
Quaternion Sec1StallActivity.controllerOrigin = Quaternion.identity  [private]
```

Initial controller orientation captured at step start.

Definition at line 43 of file Sec1StallActivity.cs.

7.34.4.3 currentPitch

```
float Sec1StallActivity.currentPitch = 0f  [private]
```

Current aircraft pitch angle in degrees, updated each frame.

Definition at line 65 of file Sec1StallActivity.cs.

7.34.4.4 currentPitchTarget

```
int Sec1StallActivity.currentPitchTarget = 0  [private]
```

Index of current pitch target in pitchTargets array.

Definition at line 62 of file Sec1StallActivity.cs.

7.34.4.5 currentStep

```
TurnStep Sec1StallActivity.currentStep  [private]
```

Current training step in the activity sequence.

Definition at line 63 of file Sec1StallActivity.cs.

7.34.4.6 deadZone

```
float Sec1StallActivity.deadZone = 15f
```

Controller deadzone threshold in degrees to filter noise.

Definition at line 41 of file Sec1StallActivity.cs.

7.34.4.7 hasControllerOrigin

```
bool Sec1StallActivity.hasControllerOrigin = false  [private]
```

Flag indicating if controller origin has been captured this step.

Definition at line 44 of file Sec1StallActivity.cs.

7.34.4.8 inputLocked

```
bool Sec1StallActivity.inputLocked = false  [private]
```

Flag to disable user input during transitions and timelines.

Definition at line 64 of file Sec1StallActivity.cs.

7.34.4.9 mouseRollSpeed

```
float Sec1StallActivity.mouseRollSpeed = 10f
```

Speed multiplier for mouse-based pitch control.

Definition at line 48 of file Sec1StallActivity.cs.

7.34.4.10 pitchSensitivity

```
float Sec1StallActivity.pitchSensitivity = 1f
```

Pitch sensitivity multiplier for controller input.

Definition at line 40 of file Sec1StallActivity.cs.

7.34.4.11 pitchTargets

```
float [] Sec1StallActivity.pitchTargets = { 3f, 7f, 10f, 14f, 16f, 18f }  [private]
```

Progressive pitch angle targets in degrees leading to stall.

Definition at line 61 of file Sec1StallActivity.cs.

7.34.4.12 pitchTimelineAssets

```
TimelineAsset [] Sec1StallActivity.pitchTimelineAssets
```

Timeline assets for each pitch milestone (3, 7, 10, 14, 16, 18).

Definition at line 34 of file Sec1StallActivity.cs.

7.34.4.13 pitchTimelines

```
PlayableDirector [] Sec1StallActivity.pitchTimelines  [private]
```

PlayableDirectors corresponding to pitchTimelineAssets.

Definition at line 67 of file Sec1StallActivity.cs.

7.34.4.14 proceduralAirflow

```
ProceduralAirflow Sec1StallActivity.proceduralAirflow  [private]
```

Airflow visualization system for force arrows, particles, and COP.

Definition at line 69 of file Sec1StallActivity.cs.

7.34.4.15 rotationInput

```
InputActionReference Sec1StallActivity.rotationInput
```

VR controller rotation input action for aircraft pitch control.

Definition at line 37 of file Sec1StallActivity.cs.

7.34.4.16 stepAdvanceTolerance

```
float Sec1StallActivity.stepAdvanceTolerance = 3f
```

Tolerance in degrees for achieving target pitch before advancing.

Definition at line 42 of file Sec1StallActivity.cs.

7.34.4.17 useMouseClicks

```
bool Sec1StallActivity.useMouseClicks = false
```

Enable mouse button control for testing without VR.

Definition at line 47 of file Sec1StallActivity.cs.

7.34.5 Property Documentation

7.34.5.1 mas

```
ModuleActivityScheduler Sec1StallActivity.mas  [get], [private]
```

Reference to singleton activity scheduler.

Definition at line 31 of file Sec1StallActivity.cs.

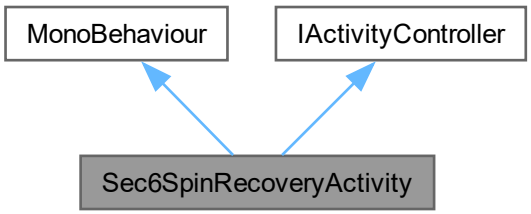
The documentation for this class was generated from the following file:

- Assets/ModuleExclusiveAssets/Module2/Section1/Sec1StallActivity.cs

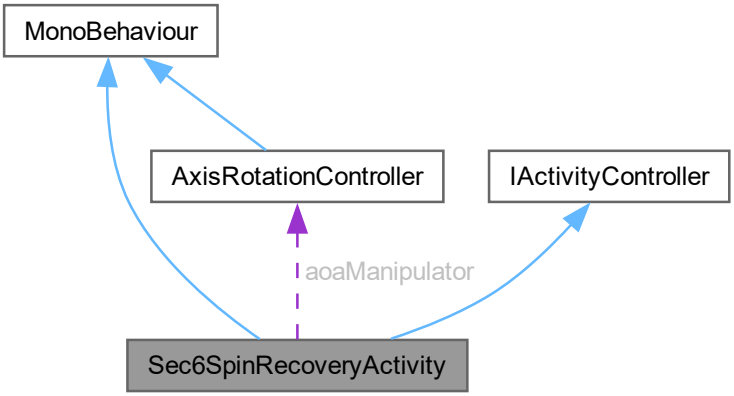
7.35 Sec6SpinRecoveryActivity Class Reference

Section 6 spin recovery training activity controller.

Inheritance diagram for Sec6SpinRecoveryActivity:



Collaboration diagram for Sec6SpinRecoveryActivity:



Public Member Functions

- void OnEnable ()

  Initialize activity on component enable.
- void OnDisable ()

  Cleanup activity on component disable.
- void StartActivity ()

  Start the activity when triggered by ModuleActivityScheduler.
- void EnableCurrentStep ()

  Unlock input for the current training step.

Public Attributes

- GameObject throttleCanvasPrefab

  Prefab for throttle display UI canvas.
- TimelineAsset[ ] pareStepTimelineAssets

  Timeline for each PARE step completion (4 timelines).
- InputActionReference rotationInput

  Controller rotation for pitch/bank control.
- InputActionReference throttleInput

  VR controller input for throttle control (Y-axis).
- float controlSensitivity = 1f

  Roll/pitch sensitivity multiplier for controller input.
- float deadZone = 15f

  Controller deadzone threshold in degrees to filter noise.
- bool spinDirectionRight = true

  Spin direction: true = right spin, false = left spin.
- float spinRotationSpeed = 180f

  Yaw rotation speed during spin in degrees per second.
- float initialSpinBank = 60f

  Initial bank angle for spin in degrees.
- float initialSpinAOA = 25f

  Initial angle of attack for spin in degrees (stalled condition).
- bool useMouseInput = false

  Use mouse for testing without VR.
- float mouseControlSpeed = 10000f

  Speed multiplier for mouse control.

Properties

- ModuleActivityScheduler mas `[get]`

  Reference to singleton activity scheduler.

Private Types

- enum TurnStep {
  powerToIdle , aileronsNeutral , rudderOpposite , elevatorForward ,
  levelFlight }

  PARE recovery sequence states.

Private Member Functions

- void InitializeSpinState ()

  Initialize aircraft into spin state.
- void Update ()

  Main update loop.
- void HandlePowerToIdle ()

  Handle power reduction to idle step (PARE step 1).
- void HandleAileronsNeutral ()

  Handle ailerons neutral step (PARE step 2).

- void HandleRudderOpposite ()

  Handle rudder opposite spin direction step (PARE step 3).
- void HandleElevatorForward ()

  Handle elevator forward to break stall step (PARE step 4).
- void SpinPlane ()

  Continuously rotate aircraft in yaw to simulate spinning.
- void updateThrottleDisplay ()

  Update throttle display canvas with current value.

Private Attributes

- GameObject throttleCanvasInstance

  Instantiated throttle display canvas.
- TMP_Text throttleText

  Text component for displaying throttle percentage.
- Quaternion controllerOrigin = Quaternion.identity

  Controller starting orientation captured at step start.
- bool hasControllerOrigin = false

  Flag indicating if controller origin has been captured.
- AxisRotationController aoaManipulator

  Reference to aircraft rotation controller.
- TurnStep currentStep

  Current step in PARE recovery sequence.
- PlayableDirector[ ] pareStepTimelines

  Directors for PARE step confirmation timelines.
- bool inputLocked = false

  Flag to disable user input during transitions.
- float stepStartTime = 0f

  Time when current step started (for minimum step duration).
- float minimumStepTime = 0.5f

  Minimum time in seconds before step can complete (prevents instant completion).
- bool isSpinning = true

  Flag indicating if aircraft is currently in spinning state.
- float spinDirection = 1f

  Spin direction multiplier: 1 for right spin, -1 for left spin.
- float currentThrottle = 1f

  Current throttle input value from controller.
- float currentBank = 0f

  Current bank angle in degrees, updated each frame.
- float currentPitch = 0f

  Current pitch angle in degrees, updated each frame.
- float totalThrottle = 100

  Current throttle percentage (0-100) displayed to user.
- float throttleIncrement = 20

  Throttle change rate in percent per second.

7.35.1 Detailed Description

Section 6 spin recovery training activity controller.

Manages a PARE (Power-Ailerons-Rudder-Elevator) spin recovery training exercise where users practice the four-step procedure to recover from an aircraft spin. The aircraft begins in an active spin state (rotating continuously in yaw) and users must execute the correct recovery sequence to stop the spin and return to level flight. Implements the IActivityController interface for integration with the ModuleActivityScheduler system.

PARE Recovery Sequence:

1. Power to Idle - Reduce throttle to below 10%
2. Ailerons Neutral - Automated demonstration (passive step)
3. Rudder Opposite - Apply opposite rudder to stop spin rotation
4. Elevator Forward - Push nose down to break the stall
5. Level Flight - Recovery complete

Key features:

- Continuous yaw rotation simulation during spin
- Configurable spin direction (left or right)
- Throttle control with visual display
- Real-time feedback on recovery inputs
- Automatic spin cessation when correct inputs applied

Author

Connor Freebairn | frecd002@mymail.unisa.edu.au

Definition at line 32 of file Sec6SpinRecoveryActivity.cs.

7.35.2 Member Enumeration Documentation

7.35.2.1 TurnStep

```
enum Sec6SpinRecoveryActivity.TurnStep  [private]
```

PARE recovery sequence states.

Defines the five steps of the PARE spin recovery procedure.

Enumerator

powerToIdle	User reduces throttle to idle position.

aileronsNeutral	Automated demonstration of neutral ailerons.
rudderOpposite	User applies rudder opposite to spin direction.
elevatorForward	User pushes elevator forward to break stall.
levelFlight	Recovery complete, return to level flight.

Definition at line 72 of file Sec6SpinRecoveryActivity.cs.

7.35.3 Member Function Documentation

7.35.3.1 EnableCurrentStep()

```
void Sec6SpinRecoveryActivity.EnableCurrentStep ()
```

Unlock input for the current training step.

Resets input lock and controller origin flag to allow user control, and records the step start time for minimum duration enforcement.

Definition at line 183 of file Sec6SpinRecoveryActivity.cs.

7.35.3.2 HandleAileronsNeutral()

```
void Sec6SpinRecoveryActivity.HandleAileronsNeutral ()  [private]
```

Handle ailerons neutral step (PARE step 2).

Automated/passive demonstration step with no user input required. Immediately locks input and plays timeline, then advances to rudder opposite step.

Definition at line 285 of file Sec6SpinRecoveryActivity.cs.

7.35.3.3 HandleElevatorForward()

```
void Sec6SpinRecoveryActivity.HandleElevatorForward ()  [private]
```

Handle elevator forward to break stall step (PARE step 4).

User must push controller forward to pitch nose down below -10, breaking the stall condition. When correct pitch achieved and minimum time elapsed, locks input, lerps AOA to 3 (recovered flight), plays confirmation timeline, and advances to level flight completion. Allows controller or debug mouse input while waiting for correct input.

Definition at line 367 of file Sec6SpinRecoveryActivity.cs.

7.35.3.4 HandlePowerToIdle()

```
void Sec6SpinRecoveryActivity.HandlePowerToIdle ()  [private]
```

Handle power reduction to idle step (PARE step 1).

User must reduce throttle below 10% to complete this step. Continuously updates totalThrottle based on controller input and displays current value on canvas. When throttle drops below threshold, locks input, plays confirmation timeline, advances to next step, and destroys throttle display canvas.

Definition at line 253 of file Sec6SpinRecoveryActivity.cs.

7.35.3.5 HandleRudderOpposite()

```
void Sec6SpinRecoveryActivity.HandleRudderOpposite ()  [private]
```

Handle rudder opposite spin direction step (PARE step 3).

User must apply rudder in direction opposite to spin by rolling controller:

- Right spin requires left rudder (negative bank $<$ -10)
- Left spin requires right rudder (positive bank $>$ 10)

When correct rudder is applied and minimum time elapsed, stops spin rotation, locks input, lerps bank to neutral, plays confirmation timeline, and advances to elevator forward step. Allows controller or debug mouse input while waiting for correct input.

Definition at line 308 of file Sec6SpinRecoveryActivity.cs.

7.35.3.6 InitializeSpinState()

```
void Sec6SpinRecoveryActivity.InitializeSpinState ()  [private]
```

Initialize aircraft into spin state.

Sets the aircraft to initial spin conditions with high AOA (stalled) and appropriate bank angle based on spin direction. Uses lerp for smooth transition into spin state.

Definition at line 153 of file Sec6SpinRecoveryActivity.cs.

7.35.3.7 OnDisable()

```
void Sec6SpinRecoveryActivity.OnDisable ()
```

Cleanup activity on component disable.

Disables input actions for rotation and throttle control.

Definition at line 134 of file Sec6SpinRecoveryActivity.cs.

7.35.3.8 OnEnable()

```
void Sec6SpinRecoveryActivity.OnEnable ()
```

Initialize activity on component enable.

Enables input actions, resets progression state to initial PARE step, retrieves references to aircraft controller, finds PlayableDirectors for timeline assets, and initializes the aircraft into spin state with appropriate bank and AOA.

Definition at line 107 of file Sec6SpinRecoveryActivity.cs.

7.35.3.9 SpinPlane()

```
void Sec6SpinRecoveryActivity.SpinPlane ()  [private]
```

Continuously rotate aircraft in yaw to simulate spinning.

Called each frame while isSpinning flag is true. Applies continuous yaw rotation at spinRotationSpeed in the direction specified by spinDirection multiplier. Stops when isSpinning is set to false in HandleRudderOpposite.

Definition at line 409 of file Sec6SpinRecoveryActivity.cs.

7.35.3.10 StartActivity()

```
void Sec6SpinRecoveryActivity.StartActivity ()
```

Start the activity when triggered by ModuleActivityScheduler.

Called by the activity scheduler to begin the PARE recovery sequence. Unlocks input for the first step.

Implements IActivityController.

Definition at line 172 of file Sec6SpinRecoveryActivity.cs.

7.35.3.11 Update()

```
void Sec6SpinRecoveryActivity.Update ()  [private]
```

Main update loop.

Continuously rotates aircraft during spin, captures controller origin, reads current aircraft state, and routes execution to appropriate PARE step handler.

Definition at line 197 of file Sec6SpinRecoveryActivity.cs.

7.35.3.12 updateThrottleDisplay()

```
void Sec6SpinRecoveryActivity.updateThrottleDisplay () [private]
```

Update throttle display canvas with current value.

Instantiates throttle canvas on first call and positions it in world space. Updates the TMP_Text component with current totalThrottle percentage value.

Note

Uses hardcoded position values that should be replaced with more robust system

Definition at line 427 of file Sec6SpinRecoveryActivity.cs.

7.35.4 Member Data Documentation

7.35.4.1 aoaManipulator

```
AxisRotationController Sec6SpinRecoveryActivity.aoaManipulator [private]
```

Reference to aircraft rotation controller.

Definition at line 63 of file Sec6SpinRecoveryActivity.cs.

7.35.4.2 controllerOrigin

```
Quaternion Sec6SpinRecoveryActivity.controllerOrigin = Quaternion.identity [private]
```

Controller starting orientation captured at step start.

Definition at line 60 of file Sec6SpinRecoveryActivity.cs.

7.35.4.3 controlSensitivity

```
float Sec6SpinRecoveryActivity.controlSensitivity = 1f
```

Roll/pitch sensitivity multiplier for controller input.

Definition at line 47 of file Sec6SpinRecoveryActivity.cs.

7.35.4.4 currentBank

```
float Sec6SpinRecoveryActivity.currentBank = 0f [private]
```

Current bank angle in degrees, updated each frame.

Definition at line 93 of file Sec6SpinRecoveryActivity.cs.

7.35.4.5 currentPitch

```
float Sec6SpinRecoveryActivity.currentPitch = 0f  [private]
```

Current pitch angle in degrees, updated each frame.

Definition at line 94 of file Sec6SpinRecoveryActivity.cs.

7.35.4.6 currentStep

```
TurnStep Sec6SpinRecoveryActivity.currentStep  [private]
```

Current step in PARE recovery sequence.

Definition at line 65 of file Sec6SpinRecoveryActivity.cs.

7.35.4.7 currentThrottle

```
float Sec6SpinRecoveryActivity.currentThrottle = 1f  [private]
```

Current throttle input value from controller.

Definition at line 92 of file Sec6SpinRecoveryActivity.cs.

7.35.4.8 deadZone

```
float Sec6SpinRecoveryActivity.deadZone = 15f
```

Controller deadzone threshold in degrees to filter noise.

Definition at line 48 of file Sec6SpinRecoveryActivity.cs.

7.35.4.9 hasControllerOrigin

```
bool Sec6SpinRecoveryActivity.hasControllerOrigin = false  [private]
```

Flag indicating if controller origin has been captured.

Definition at line 61 of file Sec6SpinRecoveryActivity.cs.

7.35.4.10 initialSpinAOA

```
float Sec6SpinRecoveryActivity.initialSpinAOA = 25f
```

Initial angle of attack for spin in degrees (stalled condition).

Definition at line 54 of file Sec6SpinRecoveryActivity.cs.

7.35.4.11 initialSpinBank

```
float Sec6SpinRecoveryActivity.initialSpinBank = 60f
```

Initial bank angle for spin in degrees.

Definition at line 53 of file Sec6SpinRecoveryActivity.cs.

7.35.4.12 inputLocked

```
bool Sec6SpinRecoveryActivity.inputLocked = false  [private]
```

Flag to disable user input during transitions.

Definition at line 83 of file Sec6SpinRecoveryActivity.cs.

7.35.4.13 isSpinning

```
bool Sec6SpinRecoveryActivity.isSpinning = true  [private]
```

Flag indicating if aircraft is currently in spinning state.

Definition at line 88 of file Sec6SpinRecoveryActivity.cs.

7.35.4.14 minimumStepTime

```
float Sec6SpinRecoveryActivity.minimumStepTime = 0.5f  [private]
```

Minimum time in seconds before step can complete (prevents instant completion).

Definition at line 85 of file Sec6SpinRecoveryActivity.cs.

7.35.4.15 mouseControlSpeed

```
float Sec6SpinRecoveryActivity.mouseControlSpeed = 10000f
```

Speed multiplier for mouse control.

Definition at line 58 of file Sec6SpinRecoveryActivity.cs.

7.35.4.16 pareStepTimelineAssets

```
TimelineAsset [] Sec6SpinRecoveryActivity.pareStepTimelineAssets
```

Timeline for each PARE step completion (4 timelines).

Definition at line 40 of file Sec6SpinRecoveryActivity.cs.

7.35.4.17 pareStepTimelines

```
PlayableDirector [] Sec6SpinRecoveryActivity.pareStepTimelines  [private]
```

Directors for PARE step confirmation timelines.

Definition at line 81 of file Sec6SpinRecoveryActivity.cs.

7.35.4.18 rotationInput

```
InputActionReference Sec6SpinRecoveryActivity.rotationInput
```

Controller rotation for pitch/bank control.

Definition at line 43 of file Sec6SpinRecoveryActivity.cs.

7.35.4.19 spinDirection

```
float Sec6SpinRecoveryActivity.spinDirection = 1f  [private]
```

Spin direction multiplier: 1 for right spin, -1 for left spin.

Definition at line 89 of file Sec6SpinRecoveryActivity.cs.

7.35.4.20 spinDirectionRight

```
bool Sec6SpinRecoveryActivity.spinDirectionRight = true
```

Spin direction: true = right spin, false = left spin.

Definition at line 51 of file Sec6SpinRecoveryActivity.cs.

7.35.4.21 spinRotationSpeed

```
float Sec6SpinRecoveryActivity.spinRotationSpeed = 180f
```

Yaw rotation speed during spin in degrees per second.

Definition at line 52 of file Sec6SpinRecoveryActivity.cs.

7.35.4.22 stepStartTime

```
float Sec6SpinRecoveryActivity.stepStartTime = 0f  [private]
```

Time when current step started (for minimum step duration).

Definition at line 84 of file Sec6SpinRecoveryActivity.cs.

7.35.4.23 throttleCanvasInstance

```
GameObject Sec6SpinRecoveryActivity.throttleCanvasInstance  [private]
```

Instantiated throttle display canvas.

Definition at line 36 of file Sec6SpinRecoveryActivity.cs.

7.35.4.24 throttleCanvasPrefab

```
GameObject Sec6SpinRecoveryActivity.throttleCanvasPrefab
```

Prefab for throttle display UI canvas.

Definition at line 35 of file Sec6SpinRecoveryActivity.cs.

7.35.4.25 throttleIncrement

```
float Sec6SpinRecoveryActivity.throttleIncrement = 20  [private]
```

Throttle change rate in percent per second.

Definition at line 98 of file Sec6SpinRecoveryActivity.cs.

7.35.4.26 throttleInput

```
InputActionReference Sec6SpinRecoveryActivity.throttleInput
```

VR controller input for throttle control (Y-axis).

Definition at line 44 of file Sec6SpinRecoveryActivity.cs.

7.35.4.27 throttleText

```
TMP_Text Sec6SpinRecoveryActivity.throttleText  [private]
```

Text component for displaying throttle percentage.

Definition at line 37 of file Sec6SpinRecoveryActivity.cs.

7.35.4.28 totalThrottle

```
float Sec6SpinRecoveryActivity.totalThrottle = 100  [private]
```

Current throttle percentage (0-100) displayed to user.

Definition at line 97 of file Sec6SpinRecoveryActivity.cs.

7.35.4.29 useMouseInput

```
bool Sec6SpinRecoveryActivity.useMouseInput = false
```

Use mouse for testing without VR.

Definition at line 57 of file Sec6SpinRecoveryActivity.cs.

7.35.5 Property Documentation

7.35.5.1 mas

```
ModuleActivityScheduler Sec6SpinRecoveryActivity.mas  [get], [private]
```

Reference to singleton activity scheduler.

Definition at line 34 of file Sec6SpinRecoveryActivity.cs.

The documentation for this class was generated from the following file:

- Assets/ModuleExclusiveAssets/Module2/Section6/Sec6SpinRecoveryActivity.cs

7.36 ModuleManager.Section Class Reference

Sections are combinations of timelines and activities that form a Module.

Public Attributes

- string name
- List< PlayableDirector > timelines

  A collection of timelines that make up a Section.
- List< ModuleActivities > activities

  A collection of activities that make up a Section.

7.36.1 Detailed Description

Sections are combinations of timelines and activities that form a Module.

Definition at line 29 of file ModuleManager.cs.

7.36.2 Member Data Documentation

7.36.2.1 activities

```
List<ModuleActivities> ModuleManager.Section.activities
```

A collection of activities that make up a Section.

Note that for an activity to be paired with a timeline, it MUST be in the same index slot in this List as the relevant timeline in the timelines List (e.g. for an activity to play after the timeline in the [3] index of the timelines List, the activity must also be in the [3] slot of this List).

Definition at line 33 of file ModuleManager.cs.

7.36.2.2 name

```
string ModuleManager.Section.name
```

Definition at line 31 of file ModuleManager.cs.

7.36.2.3 timelines

```
List<PlayableDirector> ModuleManager.Section.timelines
```

A collection of timelines that make up a Section.

Each timeline contains animations, subtitles and narration, and signals.

Definition at line 32 of file ModuleManager.cs.

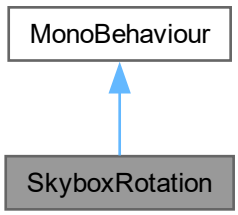
The documentation for this class was generated from the following file:

- Assets/Scripts/ModuleStructure/ModuleManager.cs

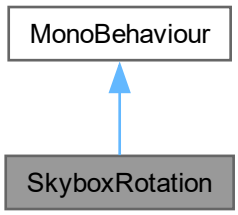
7.37 SkyboxRotation Class Reference

Ambient class that slowly and gradualy rotates the skybox in the world to give the impression of moving clouds.

Inheritance diagram for SkyboxRotation:



Collaboration diagram for SkyboxRotation:



Public Attributes

- float rotationSpeed = 0.1f

  Speed at which the clouds should move.

Private Member Functions

- void Update ()

7.37.1 Detailed Description

Ambient class that slowly and gradualy rotates the skybox in the world to give the impression of moving clouds.

Author

Caleb Martin | marcy066@mymail.unisa.edu.au

Definition at line 9 of file SkyboxRotation.cs.

7.37.2 Member Function Documentation

7.37.2.1 Update()

```
void SkyboxRotation.Update ()  [private]
```

Definition at line 13 of file SkyboxRotation.cs.

7.37.3 Member Data Documentation

7.37.3.1 rotationSpeed

```
float SkyboxRotation.rotationSpeed = 0.1f
```

Speed at which the clouds should move.

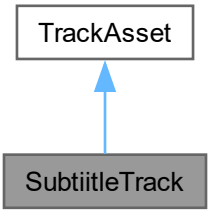
Definition at line 11 of file SkyboxRotation.cs.

The documentation for this class was generated from the following file:

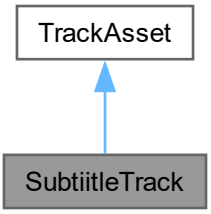
- Assets/Scripts/MiscBehaviour/SkyboxRotation.cs

7.38 SubtiitleTrack Class Reference

Inheritance diagram for SubtiitleTrack:



Collaboration diagram for SubtiitleTrack:



7.38.1 Detailed Description

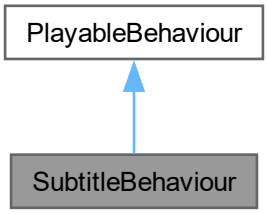
Definition at line 10 of file SubtitleTrack.cs.

The documentation for this class was generated from the following file:

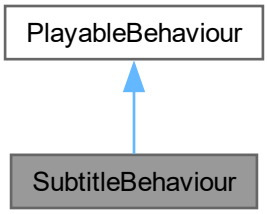
- Assets/Scripts/CustomTimelineTracks/SubtitleTimelineTrack/SubtitleTrack.cs

7.39 SubtitleBehaviour Class Reference

Inheritance diagram for SubtitleBehaviour:



Collaboration diagram for SubtitleBehaviour:



Public Member Functions

- override void ProcessFrame (Playable playable, FrameData info, object playerData)

Public Attributes

- string subtitleText
- Color subtitleColor

7.39.1 Detailed Description

Definition at line 5 of file SubtitleBehaviour.cs.

7.39.2 Member Function Documentation

7.39.2.1 ProcessFrame()

```
override void SubtitleBehaviour.ProcessFrame (
            Playable playable,
            FrameData info,
            object playerData)
```

Definition at line 13 of file SubtitleBehaviour.cs.

7.39.3 Member Data Documentation

7.39.3.1 subtitleColor

```
Color SubtitleBehaviour.subtitleColor
```

Definition at line 9 of file SubtitleBehaviour.cs.

7.39.3.2 subtitleText

```
string SubtitleBehaviour.subtitleText
```

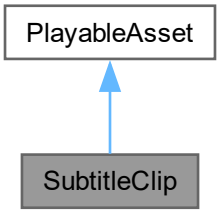
Definition at line 8 of file SubtitleBehaviour.cs.

The documentation for this class was generated from the following file:

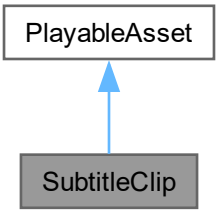
- Assets/Scripts/CustomTimelineTracks/SubtitleTimelineTrack/SubtitleBehaviour.cs

7.40 SubtitleClip Class Reference

Inheritance diagram for SubtitleClip:



Collaboration diagram for SubtitleClip:



Public Member Functions

- override Playable CreatePlayable (PlayableGraph graph, GameObject owner)

Public Attributes

- Color subtitleColor
- string subtitleText

7.40.1 Detailed Description

Definition at line 5 of file SubtitleClip.cs.

7.40.2 Member Function Documentation

7.40.2.1 CreatePlayable()

```
override Playable SubtitleClip.CreatePlayable (
            PlayableGraph graph,
            GameObject owner)
```

Definition at line 10 of file SubtitleClip.cs.

7.40.3 Member Data Documentation

7.40.3.1 subtitleColor

```
Color SubtitleClip.subtitleColor
```

Definition at line 7 of file SubtitleClip.cs.

7.40.3.2 subtitleText

```
string SubtitleClip.subtitleText
```

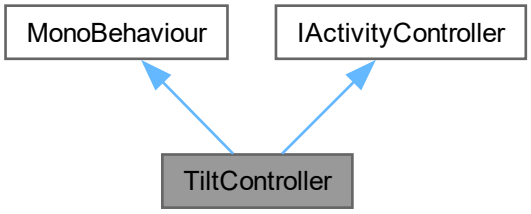
Definition at line 8 of file SubtitleClip.cs.

The documentation for this class was generated from the following file:

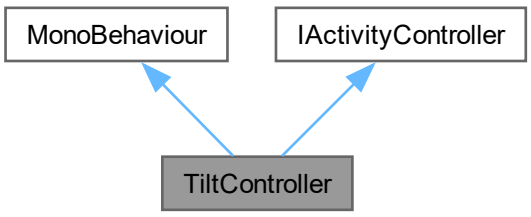
- Assets/Scripts/CustomTimelineTracks/SubtitleTimelineTrack/SubtitleClip.cs

7.41 TiltController Class Reference

Inheritance diagram for TiltController:



Collaboration diagram for TiltController:



Public Types

- enum RotationAxis { Pitch , Yaw , Roll }

Public Member Functions

- void StartActivity ()

  Each child activity can have a differing implementation of what must be carried out/what inputs must be switched on when the activity starts.

Public Attributes

- InputActionReference inputAction
- float rotationThreshold = 25f
- RotationAxis monitoredAxis

Properties

- ModuleActivityScheduler mas `[get]`

Private Member Functions

- void OnEnable ()
- void OnDisable ()
- IEnumerator CaptureInitialRotationDelayed ()
- void OnRotationChanged (InputAction.CallbackContext ctx)
- float NormalizeAngle (float angle)

Private Attributes

- Quaternion initialRotation
- bool activityEnabled = false
- bool hasTriggered = false

7.41.1 Detailed Description

Definition at line 5 of file TiltController.cs.

7.41.2 Member Enumeration Documentation

7.41.2.1 RotationAxis

`enum TiltController.RotationAxis`

Enumerator

Pitch	
Yaw	
Roll	

Definition at line 13 of file TiltController.cs.

7.41.3 Member Function Documentation

7.41.3.1 CaptureInitialRotationDelayed()

```
IEnumerator TiltController.CaptureInitialRotationDelayed ()  [private]
```

Definition at line 50 of file TiltController.cs.

7.41.3.2 NormalizeAngle()

```
float TiltController.NormalizeAngle (
            float angle)  [private]
```

Definition at line 105 of file TiltController.cs.

7.41.3.3 OnDisable()

```
void TiltController.OnDisable ()  [private]
```

Definition at line 32 of file TiltController.cs.

7.41.3.4 OnEnable()

```
void TiltController.OnEnable ()  [private]
```

Definition at line 22 of file TiltController.cs.

7.41.3.5 OnRotationChanged()

```
void TiltController.OnRotationChanged (
            InputAction.CallbackContext ctx)  [private]
```

Definition at line 61 of file TiltController.cs.

7.41.3.6 StartActivity()

```
void TiltController.StartActivity ()
```

Each child activity can have a differing implementation of what must be carried out/what inputs must be switched on when the activity starts.

Override this method for new activity scripts.

Implements IActivityController.

Definition at line 42 of file TiltController.cs.

7.41.4 Member Data Documentation

7.41.4.1 activityEnabled

```
bool TiltController.activityEnabled = false  [private]
```

Definition at line 18 of file TiltController.cs.

7.41.4.2 hasTriggered

```
bool TiltController.hasTriggered = false  [private]
```

Definition at line 19 of file TiltController.cs.

7.41.4.3 initialRotation

```
Quaternion TiltController.initialRotation  [private]
```

Definition at line 16 of file TiltController.cs.

7.41.4.4 inputAction

```
InputActionReference TiltController.inputAction
```

Definition at line 9 of file TiltController.cs.

7.41.4.5 monitoredAxis

```
RotationAxis TiltController.monitoredAxis
```

Definition at line 14 of file TiltController.cs.

7.41.4.6 rotationThreshold

```
float TiltController.rotationThreshold = 25f
```

Definition at line 10 of file TiltController.cs.

7.41.5 Property Documentation

7.41.5.1 mas

```
ModuleActivityScheduler TiltController.mas  [get], [private]
```

Definition at line 7 of file TiltController.cs.

The documentation for this class was generated from the following file:

- Assets/ModuleExclusiveAssets/Module2/Section7/TiltController.cs

Chapter 8

File Documentation

8.1 Assets/Editor/GhostMeshGenerator.cs File Reference

Classes

- class GhostMeshGenerator

8.2 GhostMeshGenerator.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002 using UnityEditor;
00003 using System.IO;
00004
00005 public class GhostMeshGenerator
00006 {
00007     [MenuItem("Tools/Generate mesh from selected meshes")]
00008     static void GenerateShostMesh()
00009     {
00010         GameObject target = Selection.activeGameObject;
00011         MeshFilter[] meshFilters = target.GetComponentsInChildren<MeshFilter>();
00012
00013         CombineInstance[] combine = new CombineInstance[meshFilters.Length];
00014
00015         for (int i = 0; i < meshFilters.Length; i++)
00016         {
00017             combine[i].mesh = meshFilters[i].sharedMesh;
00018             combine[i].transform = meshFilters[i].transform.localToWorldMatrix;
00019         }
00020
00021         Mesh ghostMesh = new Mesh();
00022         ghostMesh.name = target.name + "_GhostMesh";
00023         ghostMesh.indexFormat = UnityEngine.Rendering.IndexFormat.UInt32;
00024         ghostMesh.CombineMeshes(combine, true, true);
00025
00026         string path = "Assets/GhostMeshes";
00027         if (!Directory.Exists(path))
00028         {
00029             Directory.CreateDirectory(path);
00030         }
00031
00032         string assetPath = path + "/" + ghostMesh.name + ".asset";
00033         AssetDatabase.CreateAsset(ghostMesh, assetPath);
00034         AssetDatabase.SaveAssets();
00035     }
00036 }
```

8.3 Assets/ModuleExclusiveAssets/Module2/Section1/Sec1StallActivity.cs File Reference

Classes

- class Sec1StallActivity

  Section 1 stall training activity controller.

8.4 Sec1StallActivity.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002 using UnityEngine.InputSystem;
00003 using UnityEngine.Playables;
00004 using UnityEngine.Timeline;
00005
00028 public class Sec1StallActivity : MonoBehaviour, IActivityController
00029 {
00030
00031     private ModuleActivityScheduler mas => ModuleActivityScheduler.Instance;
00032
00033     [Header("timelines")]
00034     public TimelineAsset[] pitchTimelineAssets;
00035
00036     [Header("Input action references")]
00037     public InputActionReference rotationInput;
00038
00039     [Header("Control Settings")]
00040     public float pitchSensitivity = 1f;
00041     public float deadZone = 15f;
00042     public float stepAdvanceTolerance = 3f;
00043     private Quaternion controllerOrigin = Quaternion.identity;
00044     private bool hasControllerOrigin = false;
00045
00046     [Header("Editor Debug settings")]
00047     public bool useMouseClicks = false;
00048     public float mouseRollSpeed = 10f;
00049
00055     private enum TurnStep
00056     {
00057         Pitch,
00058         Complete
00059     }
00060
00061     private float[] pitchTargets = { 3f, 7f, 10f, 14f, 16f, 18f };
00062     private int currentPitchTarget = 0;
00063     private TurnStep currentStep;
00064     private bool inputLocked = false;
00065     private float currentPitch = 0f;
00066
00067     private PlayableDirector[] pitchTimelines;
00068     private AxisRotationController aoaManipulator;
00069     private ProceduralAirflow proceduralAirflow;
00070
00079     public void OnEnable()
00080     {
00081         if (rotationInput != null && rotationInput.action != null)
00082         {
00083             rotationInput.action.Enable();
00084         }
00085         // reset progression
00086         currentStep = TurnStep.Pitch;
00087         currentPitchTarget = 0;
00088         inputLocked = false;
00089         currentPitch = 0f;
00090         // Manipulators
00091         aoaManipulator = ActivityHelper.getAxisRotationController(0f, 0f);
00092         proceduralAirflow = ActivityHelper.getProceduralAirflow();
00093         // Call helper to find directors for timeline assets
00094         pitchTimelines = ActivityHelper.FindPlayableDirector(pitchTimelineAssets);
00095         proceduralAirflow.enableForceArrows = true;
00096         proceduralAirflow.enableParticleSystem = true;
00097         proceduralAirflow.enableCentreOfPressure = true;
00098         Debug.Log("Sec1StallActivity OnEnable completed");
00099     }
```

```
00100
00107     public void OnDisable()
00108     {
00109         if (rotationInput != null && rotationInput.action != null)
00110         {
00111             rotationInput.action.Disable();
00112         }
00113         proceduralAirflow.enableForceArrows = false;
00114         proceduralAirflow.enableParticleSystem = false;
00115         proceduralAirflow.enableCentreOfPressure = false;
00116     }
00117
00124     public void StartActivity()
00125     {
00126         EnableCurrentStep();
00127     }
00128
00135     public void EnableCurrentStep()
00136     {
00137         inputLocked = false;
00138         hasControllerOrigin = false;
00139     }
00140
00147     void Update()
00148     {
00149         switch ((TurnStep)currentStep)
00150         {
00151             case TurnStep.Pitch:
00152                 HandlePitching();
00153                 break;
00154             case TurnStep.Complete:
00155                 mas.OnExternalStepCompleted();
00156                 break;
00157         }
00158     }
00159
00160
00177     public void HandlePitching()
00178     {
00179         if (inputLocked) return;
00180
00181         // Read current rotation from the input
00182         Quaternion currentRotation = rotationInput.action.ReadValue<Quaternion>();
00183
00184         if (!hasControllerOrigin)
00185         {
00186             controllerOrigin = currentRotation;
00187             hasControllerOrigin = true;
00188             Debug.Log("Captured controller origin: " + controllerOrigin.eulerAngles);
00189         }
00190
00191
00192         currentPitch = ActivityHelper.getNormalisedPlanePitch(aoaManipulator);
00193
00194         if (ActivityHelper.checkExactRotationTargetAchieved(currentPitch,
      pitchTargets[currentPitchTarget], stepAdvanceTolerance))
00195         {
00196             inputLocked = true;
00197             hasControllerOrigin = false;
00198             aoaManipulator.LerpAOA(pitchTargets[currentPitchTarget]);
00199             StartCoroutine(ActivityHelper.PlayTimeLine(pitchTimelines[currentPitchTarget], () =>
00200             {
00201                 EnableCurrentStep();
00202                 currentPitchTarget++;
00203                 Debug.Log("currentPitch target: " + currentPitchTarget + " pitchTarget Length: " +
      pitchTargets.Length);
00204                 if (currentPitchTarget >= pitchTargets.Length)
00205                 {
00206                     // Advance to completion
00207                     currentStep = TurnStep.Complete;
00208                     Debug.Log("currentPitchTarget: " + currentPitchTarget + " pitchTarget length: " +
      pitchTargets.Length + " currentStep " + currentStep);
00209                 }
00210                 mas.OnExternalStepCompleted();
00211             }));
00212         }
00213         else
00214         {
00215             if (useMouseClicks)
00216             {
00217                 ActivityHelper.DebugMouseControlPitch(mouseRollSpeed, aoaManipulator);
00218             }
00219             else
00220             {
00221                 ActivityHelper.controllerPitchControl(pitchSensitivity, aoaManipulator, rotationInput,
      deadZone, controllerOrigin.eulerAngles.x);
00222             }
```

```
00223         }
00224     }
00225 }
```

8.5 Assets/ModuleExclusiveAssets/Module2/Section2/HoldToLower↩ Nose.cs File Reference

Classes

- class HoldToLowerNose

8.6 HoldToLowerNose.cs

Go to the documentation of this file.

```
00001 using System.Collections;
00002 //using Unity.Tutorials.Core.Editor;
00003 using UnityEngine;
00004 using UnityEngine.InputSystem;
00005 using UnityEngine.XR.Interaction.Toolkit;
00006
00010
00011
00012 public class HoldToLowerNose : MonoBehaviour, IActivityController
00013 {
00014     private ModuleActivityScheduler mas => ModuleActivityScheduler.Instance; // Reference to MAS
      singleton
00015
00016     public InputActionReference rotateAction; // Input action for rotating the left controller
00017
00018     public float minRotationThreshold = 0.1f; // Default, can be changed in inspector
00019     public float maxRotationThreshold = 90f;
00020
00021     //public float minZPositionThreshold = 0.1f; // Default, can be changed in inspector
00022     //public float maxZPositionThreshold = 1.0f;
00023
00024     Quaternion initialControllerRotation;
00025     Quaternion controllerRotation;
00026     Vector3 initialControllerRot;
00027     Vector3 controllerRot;
00028
00029     //public ActionBasedController controller;
00030
00031     private bool waitPeriodOver = false;
00032
00033
00034     // Controller Position.
00035     private bool activityEnabled = false; // Start as disabled
00036
00037     // Subscribe to event listener and enable input action
00038     private void OnEnable()
00039     {
00040         if (rotateAction != null)
00041         {
00042             //rotateAction.action.performed += OnRotatePerformed;
00043             rotateAction.action.performed += OnPositionChange;
00044             rotateAction.action.Enable();
00045         }
00046     }
00047
00048     // Unsubscribe to event listener and disable input action
00049     private void OnDisable()
00050     {
00051         if (rotateAction != null)
00052         {
00053             //rotateAction.action.performed -= OnRotatePerformed;
00054             rotateAction.action.performed -= OnPositionChange;
00055             rotateAction.action.Disable();
00056         }
00057     }
00058
00059     //
00060     public void StartActivity()
```

```
00061     {
00062         activityEnabled = true;
00063         //      rotateAction.action.Enable();
00064     }
00065 
00066 
00067     IEnumerator WaitPeriod()
00068     {
00069         yield return new WaitForSeconds(3);
00070         waitPeriodOver = true;
00071     }
00072 
00073 
00074 
00084 
00085     // Check if user has moved the controller forward.
00086     // i.e: PUSH THE CONTROL COLUMN FORWARD TO LOWER AOA.
00087 
00088     private void OnPositionChange(InputAction.CallbackContext ctx)
00089     {
00090 
00091         //if (initialControllerRotation == null)
00092         //{
00093         //    initialControllerRotation = ctx.ReadValue<Quaternion>(); // Store initial position.
00094 
00095         //    initialControllerRot = initialControllerRotation.eulerAngles;
00096         //}
00097 
00098         controllerRotation = ctx.ReadValue<Quaternion>();
00099         controllerRot = controllerRotation.eulerAngles;
00100 
00101         // Get the X rotation.
00102         float xRotation = Mathf.DeltaAngle(0f, controllerRotation.normalized.x); // fixed changed this
      to x
00103 
00104         if (!activityEnabled) return;
00105 
00106 
00107         // 0.1f, -30f
00108         if (xRotation >= minRotationThreshold && xRotation <= maxRotationThreshold)
00109         {
00110             //controller.SendHapticImpulse(1, 5f);
00111 
00112             //StartCoroutine(WaitPeriod());
00113             //if (initialControllerRot.magnitude > controllerRot.magnitude) // remove this for testing
00114 
00115             StartCoroutine(WaitPeriod());
00116 
00117             AxisRotationController aoaManip =
      GameObject.FindAnyObjectByType<AxisRotationController>();
00118             aoaManip.LerpAOA(-10); // Lower nose by 10 degrees.
00119 
00120 
00121 
00122         }
00123     }
00124 
00125 
00126     public void Update()
00127     {
00128 
00129         if (waitPeriodOver == true)
00130         {
00131             waitPeriodOver = false; // Run the code only once.
00132 
00133             mas.OnExternalStepCompleted();
00134 
00135             // This is a one-action activity, so disable after completed input
00136             activityEnabled = false;
00137 
00138             // Clean-up
00139             Destroy(gameObject); // there is only one step in the activity associated with this
      script.
00140 
00141         }
00142 
00143     }
00144 }
```

8.7 Assets/ModuleExclusiveAssets/Module2/Section2/NEW/IncreaseThrottle.cs File Reference

Classes

- class IncreaseThrottle

8.8 IncreaseThrottle.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002 using UnityEngine.InputSystem;
00003
00004 public class IncreaseThrottle : MonoBehaviour, IActivityController
00005 {
00006     private ModuleActivityScheduler mas => ModuleActivityScheduler.Instance; // Reference to MAS
      singleton
00007
00008     public InputActionReference inputAction; // Input action for control pad on left controller
00009     public float throttleThreshold = 0.1f; // Default, can be changed in inspector
00010     private bool activityEnabled = false; // Start as disabled
00011
00012     private bool hasTriggered = false; // Prevent multi-triggering
00013
00014     // Subscribe to event listener and enable input action
00015     private void OnEnable()
00016     {
00017         if (inputAction != null)
00018         {
00019             inputAction.action.performed += OnPadMoved;
00020             inputAction.action.canceled += OnPadReleased;
00021             inputAction.action.Enable();
00022         }
00023     }
00024
00025     // Unsubscribe to event listener and disable input action
00026     private void OnDisable()
00027     {
00028         if (inputAction != null)
00029         {
00030             inputAction.action.performed -= OnPadMoved;
00031             inputAction.action.canceled -= OnPadReleased;
00032             inputAction.action.Disable();
00033         }
00034     }
00035
00036     //
00037     public void StartActivity()
00038     {
00039         activityEnabled = true;
00040         hasTriggered = false;
00041     }
00042
00043     private void OnPadMoved(InputAction.CallbackContext ctx)
00044     {
00045         if (!activityEnabled || hasTriggered) return;
00046
00047         Debug.Log("Left control pad moved!");
00048
00049         Vector2 axis = ctx.ReadValue<Vector2>(); // Get the axis to be read from the controller
00050
00051
00052         // Detect if control pad is pulled up/north beyond threshold
00053         if (axis.y > throttleThreshold)
00054         {
00055
00056             // Put this here for testing that script works!
00057             //AOAManipulator aoaManip = GameObject.FindAnyObjectByType<AOAManipulator>();
00058             //aoaManip.LerpAOA(-30);
00059
00060             // Step is considered completed
00061             hasTriggered = true;
00062             mas.OnExternalStepCompleted();
00063
00064             activityEnabled = false;
00065             Destroy(gameObject);
```

```
00066         }
00067         else {
00068
00069             AxisRotationController aoaManip =
      GameObject.FindAnyObjectByType<AxisRotationController>();
00070             aoaManip.LerpAOA(-180); // Rotate to different angles to test
00071         }
00072     }
00073
00074     private void OnPadReleased(InputAction.CallbackContext ctx)
00075     {
00076         // Reset trigger when the player releases the stick
00077         Vector2 axis = ctx.ReadValue<Vector2>();
00078         if (Mathf.Abs(axis.y) < 0.1f)
00079         {
00080             hasTriggered = false;
00081         }
00082     }
00083 }
```

8.9 Assets/ModuleExclusiveAssets/Module2/Section3/ClimbingTurn↩ Activity.cs File Reference

Classes

- class ClimbingTurnActivity

  Climbing turn training activity controller.

8.10 ClimbingTurnActivity.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002 using UnityEngine.InputSystem;
00003 using UnityEngine.Playables;
00004 using UnityEngine.Timeline;
00005
00022 public class ClimbingTurnActivity : MonoBehaviour, IActivityController
00023 {
00024     private ModuleActivityScheduler mas => ModuleActivityScheduler.Instance;
00025
00026     [Header("timeline assets")]
00027     public TimelineAsset postSlipStream;
00028     public TimelineAsset postBank;
00029     public TimelineAsset postStall;
00030
00031     [Header("Input action references")]
00032     public InputActionReference rotationInput;
00033
00034     [Header("Control Settings")]
00035     public float bankSensitivity = 45f;
00036     public float deadZone = 0.1f;
00037     public float stepAdvanceTolerance = 3f;
00038
00039     [Header("Editor Debug settings")]
00040     public bool useMouseClicks = true;
00041     public float mouseRollSpeed = 10f;
00042
00048     private enum TurnStep
00049     {
00050         SlipStream,
00051         Bank,
00052         Stall,
00053         Complete
00054     }
00055
00056     private bool rollInputLocked = false;
00057     private bool pitchInputLocked = false;
00058     private TurnStep currentStep;
00059     private float currentBank = 0f;
00060
00061     private Quaternion controllerOrigin = Quaternion.identity;
00062     private bool hasControllerOrigin = false;
```

```
00063
00064     private bool hasNotifiedMAS = false;
00065
00066     // Manipulators
00067     private AxisRotationController aoaManipulator;
00068     private BankGhostTrailBehaviour bankGhostTrail;
00069
00070     // Timeline PlayableDirectors
00071     private PlayableDirector postSlipStreamDirector;
00072     private PlayableDirector postBankDirector;
00073     private PlayableDirector postStallDirector;
00074
00082     public void OnEnable()
00083     {
00084         if (rotationInput != null && rotationInput.action != null)
00085         {
00086             rotationInput.action.Enable();
00087         }
00088
00089         // reset progression
00090         currentStep = TurnStep.SlipStream;
00091         rollInputLocked = false;
00092         pitchInputLocked = false;
00093         hasControllerOrigin = false;
00094         currentBank = 0f;
00095
00096         aoaManipulator = ActivityHelper.getAxisRotationController(10f, 0f);
00097         bankGhostTrail = ActivityHelper.getBankGhostTrail(true);
00098
00099         // Call helper to find directors for timeline assets
00100         postSlipStreamDirector = ActivityHelper.FindPlayableDirector(postSlipStream);
00101         postBankDirector = ActivityHelper.FindPlayableDirector(postBank);
00102         postStallDirector = ActivityHelper.FindPlayableDirector(postStall);
00103     }
00104
00110     public void OnDisable()
00111     {
00112         if (rotationInput != null && rotationInput.action != null)
00113         {
00114             rotationInput.action.Disable();
00115         }
00116         bankGhostTrail.enable = false;
00117     }
00118
00125     public void StartActivity()
00126     {
00127         EnableCurrentStep();
00128     }
00129
00136     public void EnableCurrentStep()
00137     {
00138         rollInputLocked = false;
00139         pitchInputLocked = false;
00140         hasControllerOrigin = false;
00141     }
00142
00149     void Update()
00150     {
00151         switch ((TurnStep)currentStep)
00152         {
00153             case TurnStep.SlipStream:
00154                 HandleSlipStream();
00155                 break;
00156             case TurnStep.Bank:
00157                 HandleBanking();
00158                 break;
00159             case TurnStep.Stall:
00160                 HandleStall();
00161                 break;
00162             case TurnStep.Complete:
00163                 if (!hasNotifiedMAS) // failsafe for preventing multiple triggers
00164                 {
00165                     Debug.Log("Climbing Turn Activity Complete - notifying MAS");
00166                     mas.OnExternalStepCompleted();
00167                     hasNotifiedMAS = true;
00168                 }
00169                 break;
00170         }
00171     }
00172
00182     public void HandleSlipStream()
00183     {
00184         //TODO Slipstream Animation/effect
00185         if (rollInputLocked) return;
00186             rollInputLocked = true;
00187             StartCoroutine(ActivityHelper.PlayTimeLine(postSlipStreamDirector, () =>
00188             {
```

```
00189
00190                 currentStep = TurnStep.Bank;
00191                 rollInputLocked = false;
00192             }));
00193     }
00194
00204     public void HandleBanking()
00205     {
00206         if (rollInputLocked) return;
00207
00208         Quaternion currentRotation = rotationInput.action.ReadValue<Quaternion>();
00209
00210         if (!hasControllerOrigin)
00211         {
00212             controllerOrigin = currentRotation;
00213             hasControllerOrigin = true;
00214             Debug.Log("Captured controller origin (Bank): " + controllerOrigin.eulerAngles);
00215         }
00216
00217         currentBank = ActivityHelper.getNormalisedPlaneRoll(aoaManipulator);
00218
00219         if (ActivityHelper.checkExactRotationTargetAchieved(currentBank, 15f, stepAdvanceTolerance))
00220         {
00221             hasControllerOrigin = false;
00222             rollInputLocked = true;
00223             aoaManipulator.LerpBank(15f);
00224             StartCoroutine(ActivityHelper.PlayTimeLine(postBankDirector, () =>
00225             {
00226                 pitchInputLocked = false;
00227                 currentStep = TurnStep.Stall;
00228                 // TODO uncomment after testing
00229                 mas.OnExternalStepCompleted();
00230             }));
00231         }
00232         else
00233         {
00234             if (useMouseClicks)
00235             {
00236                 ActivityHelper.DebugMouseControlRoll(mouseRollSpeed, aoaManipulator);
00237             }
00238             else
00239             {
00240                 ActivityHelper.controllerRollControl(bankSensitivity, aoaManipulator, rotationInput,
    deadZone, controllerOrigin.eulerAngles.z);
00241             }
00242         }
00243     }
00244
00255     public void HandleStall()
00256     {
00257         if (pitchInputLocked) return;
00258
00259         Quaternion currentRotation = rotationInput.action.ReadValue<Quaternion>();
00260
00261         if (!hasControllerOrigin)
00262         {
00263             controllerOrigin = currentRotation;
00264             hasControllerOrigin = true;
00265             Debug.Log("Captured controller origin (Stall): " + controllerOrigin.eulerAngles);
00266         }
00267
00268         float currentPitch = ActivityHelper.getNormalisedPlanePitch(aoaManipulator);
00269
00270         if (ActivityHelper.checkRotationTargetAchieved(currentPitch, 25f, stepAdvanceTolerance))
00271         {
00272             pitchInputLocked = true;
00273             hasControllerOrigin = false;
00274             aoaManipulator.LerpAOA(25f);
00275             StartCoroutine(ActivityHelper.PlayTimeLine(postStallDirector, () =>
00276             {
00277                 // Advance to next stage
00278                 currentStep = TurnStep.Complete;
00279                 // TODO uncomment after testing
00280                 mas.OnExternalStepCompleted();
00281             }));
00282         }
00283         else
00284         {
00285             if (useMouseClicks)
00286             {
00287                 ActivityHelper.DebugMouseControlPitch(mouseRollSpeed, aoaManipulator);
00288             }
00289             else
00290             {
00291                 ActivityHelper.controllerPitchControl(bankSensitivity, aoaManipulator, rotationInput,
    deadZone, controllerOrigin.eulerAngles.x);
00292             }
```

```
00293         }
00294     }
00295 }
```

8.11 Assets/ModuleExclusiveAssets/Module2/Section3/DescendingTurnActivity.cs File Reference

Classes

- class DescendingTurnActivity

  Descending turn training activity controller.

8.12 DescendingTurnActivity.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002 using UnityEngine.InputSystem;
00003 using UnityEngine.Playables;
00004 using UnityEngine.Timeline;
00005
00022 public class DescendingTurnActivity : MonoBehaviour, IActivityController
00023 {
00024     private ModuleActivityScheduler mas => ModuleActivityScheduler.Instance;
00025
00026     [Header("timeline assets")]
00027     public TimelineAsset postBankAndPitch;
00028     public TimelineAsset postPitchUp;
00029     public TimelineAsset postStall;
00030
00031     [Header("Input action references")]
00032     public InputActionReference rotationInput;
00033
00034     [Header("Control Settings")]
00035     public float bankSensitivity = 45f;
00036     public float deadZone = 0.1f;
00037     public float stepAdvanceTolerance = 3f;
00038
00039     private Quaternion controllerOrigin = Quaternion.identity;
00040     private bool hasControllerOrigin = false;
00041
00042     private bool hasNotifiedMAS = false;
00043
00044     [Header("Editor Debug settings")]
00045     public bool useMouseClicks = true;
00046     public float mouseRollSpeed = 10f;
00047
00053     private enum TurnStep
00054     {
00055         InitialSetup,
00056         BankAndPitch,
00057         PitchUp,
00058         Stall,
00059         Complete,
00060         PlayingTimeline
00061     }
00062
00063     private bool rollInputLocked = false;
00064     private bool pitchInputLocked = false;
00065     private TurnStep currentStep;
00066     private float currentBank = 0f;
00067
00068     // Manipulators
00069     private AxisRotationController aoaManipulator;
00070     private BankGhostTrailBehaviour bankGhostTrail;
00071
00072     // Timeline PlayableDirectors
00073     private PlayableDirector postBankAndPitchDirector;
00074     private PlayableDirector postPitchUpDirector;
00075     private PlayableDirector postStallDirector;
00076
00084     public void OnEnable()
00085     {
```

```
00086         if (rotationInput != null && rotationInput.action != null)
00087         {
00088             rotationInput.action.Enable();
00089         }
00090 
00091         // reset progression
00092         currentStep = TurnStep.InitialSetup;
00093         rollInputLocked = false;
00094         pitchInputLocked = false;
00095         hasControllerOrigin = false;
00096 
00097         currentBank = 0f;
00098 
00099         aoaManipulator = ActivityHelper.getAxisRotationController(-10f, 0f);
00100         bankGhostTrail = ActivityHelper.getBankGhostTrail(true);
00101 
00102         // Call helper to find directors for timeline assets
00103         postBankAndPitchDirector = ActivityHelper.FindPlayableDirector(postBankAndPitch);
00104         postPitchUpDirector = ActivityHelper.FindPlayableDirector(postPitchUp);
00105         postStallDirector = ActivityHelper.FindPlayableDirector(postStall);
00106     }
00107 
00113     public void OnDisable()
00114     {
00115         if (rotationInput != null && rotationInput.action != null)
00116         {
00117             rotationInput.action.Disable();
00118         }
00119         bankGhostTrail.enable = false;
00120     }
00121 
00128     public void StartActivity()
00129     {
00130         EnableCurrentStep();
00131     }
00132 
00139     public void EnableCurrentStep()
00140     {
00141         rollInputLocked = false;
00142         pitchInputLocked = false;
00143         hasControllerOrigin = false;
00144     }
00145 
00152     void Update()
00153     {
00154         Debug.Log("Current Step: " + currentStep);
00155         switch ((TurnStep)currentStep)
00156         {
00157             case TurnStep.InitialSetup:
00158                 CheckSetupComplete();
00159                 break;
00160             case TurnStep.BankAndPitch:
00161                 HandleBankAndPitch();
00162                 break;
00163             case TurnStep.PitchUp:
00164                 HandlePitch();
00165                 break;
00166             case TurnStep.Stall:
00167                 HandleStall();
00168                 break;
00169             case TurnStep.Complete:
00170                 if (!hasNotifiedMAS) // failsafe for preventing multiple triggers
00171                 {
00172                     Debug.Log("Descending Turn Activity Complete - notifying MAS");
00173                     mas.OnExternalStepCompleted();
00174                     hasNotifiedMAS = true;
00175                 }
00176                 break;
00177         }
00178     }
00179 
00187     public void CheckSetupComplete()
00188     {
00189         if (!aoaManipulator.IsLerpingPitch() && !aoaManipulator.IsLerpingRoll())
00190         {
00191             currentStep = TurnStep.BankAndPitch;
00192             EnableCurrentStep();
00193         }
00194     }
00195 
00208     public void HandleBankAndPitch()
00209     {
00210         Quaternion currentRotation = rotationInput.action.ReadValue<Quaternion>();
00211         if (!hasControllerOrigin)
00212         {
00213             controllerOrigin = currentRotation;
00214             hasControllerOrigin = true;
```

```
00215         }
00216
00217         Quaternion relativeRotation = Quaternion.Inverse(controllerOrigin) * currentRotation;
00218         float relativeZ = Mathf.DeltaAngle(0f, relativeRotation.eulerAngles.z);
00219         float relativeX = Mathf.DeltaAngle(0f, relativeRotation.eulerAngles.x);
00220
00221         float planeRoll = ActivityHelper.getNormalisedPlaneRoll(aoaManipulator);
00222         float planePitch = ActivityHelper.getNormalisedPlanePitch(aoaManipulator);
00223
00224         bool rollAchieved = ActivityHelper.checkExactRotationTargetAchieved(planeRoll, 30f,
      stepAdvanceTolerance);
00225         bool pitchAchieved = ActivityHelper.checkExactRotationTargetAchieved(planePitch, -5f,
      stepAdvanceTolerance);
00226
00227         Debug.Log("rollAchieved: " + rollAchieved + " pitchAchieved: " + pitchAchieved);
00228         if (rollAchieved)
00229         {
00230             rollInputLocked = true;
00231             aoaManipulator.LerpBank(30f);
00232         }
00233         if (pitchAchieved)
00234         {
00235             // Here, lerp causes issue. fix tomorrow
00236             pitchInputLocked = true;
00237             aoaManipulator.LerpAOA(-5f);
00238         }
00239
00240         if (rollInputLocked && pitchInputLocked)
00241         {
00242             currentStep = TurnStep.PlayingTimeline;
00243             hasControllerOrigin = false;
00244             Debug.Log("Both roll and pitch achieved, starting coroutine...");
00245             StartCoroutine(ActivityHelper.PlayTimeLine(postBankAndPitchDirector, () =>
00246             {
00247                 Debug.Log("Timeline complete, advancing to PitchUp");
00248                 currentStep = TurnStep.PitchUp;
00249                 rollInputLocked = false;
00250                 pitchInputLocked = false;
00251                 EnableCurrentStep();
00252                 mas.OnExternalStepCompleted();
00253             }));
00254         }
00255
00256         if (!rollInputLocked)
00257         {
00258             if (useMouseClicks)
00259             {
00260                 ActivityHelper.DebugMouseControlRollLeft(mouseRollSpeed, aoaManipulator);
00261             }
00262             else
00263             {
00264                 ActivityHelper.controllerRollControl(bankSensitivity, aoaManipulator, rotationInput,
      deadZone, controllerOrigin.eulerAngles.z);
00265             }
00266         }
00267         if (!pitchInputLocked)
00268         {
00269             if (useMouseClicks)
00270             {
00271                 ActivityHelper.DebugMouseControlPitchUp(mouseRollSpeed, aoaManipulator);
00272             }
00273             else
00274             {
00275                 ActivityHelper.controllerPitchControl(bankSensitivity, aoaManipulator, rotationInput,
      deadZone, controllerOrigin.eulerAngles.x);
00276             }
00277         }
00278     }
00279
00288     public void HandlePitch()
00289     {
00290         if (pitchInputLocked) return;
00291
00292         Quaternion currentRotation = rotationInput.action.ReadValue<Quaternion>();
00293         if (!hasControllerOrigin)
00294         {
00295             controllerOrigin = currentRotation;
00296             hasControllerOrigin = true;
00297         }
00298
00299         float currentPitch = ActivityHelper.getNormalisedPlanePitch(aoaManipulator);
00300
00301         if (ActivityHelper.checkExactRotationTargetAchieved(currentPitch, 5f, stepAdvanceTolerance))
00302         {
00303             Debug.Log($"Pitch target achieved: {currentPitch}");
00304             pitchInputLocked = true;
00305             hasControllerOrigin = false;
```

```
00306             aoaManipulator.LerpAOA(5f, 0.5f);
00307             StartCoroutine(ActivityHelper.PlayTimeLine(postPitchUpDirector, () =>
00308             {
00309                 pitchInputLocked = false;
00310                 currentStep = TurnStep.Stall;
00311                 EnableCurrentStep();
00312                 mas.OnExternalStepCompleted();
00313             }));
00314         }
00315         else
00316         {
00317             if (useMouseClicks)
00318             {
00319                 ActivityHelper.DebugMouseControlPitch(mouseRollSpeed, aoaManipulator);
00320             }
00321             else
00322             {
00323                 ActivityHelper.controllerPitchControl(bankSensitivity, aoaManipulator, rotationInput,
      deadZone, controllerOrigin.eulerAngles.x);
00324             }
00325         }
00326     }
00327
00335     public void HandleStall()
00336     {
00337         if (pitchInputLocked) return;
00338         {
00339             pitchInputLocked = true;
00340             StartCoroutine(ActivityHelper.PlayTimeLine(postStallDirector, () =>
00341             {
00342                 EnableCurrentStep();
00343                 pitchInputLocked = false;
00344                 currentStep = TurnStep.Complete;
00345             }));
00346         }
00347     }
00348 }
```

8.13 Assets/ModuleExclusiveAssets/Module2/Section3/LevelTurn↩ Activity.cs File Reference

Classes

- class LevelTurnActivity

  Level turn training activity controller.

8.14 LevelTurnActivity.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002 using UnityEngine.InputSystem;
00003 using UnityEngine.Playables;
00004 using UnityEngine.Timeline;
00005
00021 public class LevelTurnActivity : MonoBehaviour, IActivityController
00022 {
00023    private ModuleActivityScheduler mas => ModuleActivityScheduler.Instance;
00024
00025     [Header("timelines")]
00026     public TimelineAsset[] bankTimelineAssets;
00027     public TimelineAsset stallTimelineAsset;
00028
00029     [Header("Input action references")]
00030     public InputActionReference rotationInput;
00031
00032     [Header("Control Settings")]
00033     public float bankSensitivity = 45f;
00034     public float deadZone = 0.1f;
00035     public float stepAdvanceTolerance = 3f;
00036
00037     private Quaternion controllerOrigin = Quaternion.identity;
00038     private bool hasControllerOrigin = false;
```

```
00039
00040     [Header("Editor Debug settings")]
00041     public bool useMouseClicks = false;
00042     public float mouseRollSpeed = 10000f;
00043
00049     private enum TurnStep
00050     {
00051         Bank,
00052         stallSetup,
00053         Stall,
00054         Complete
00055     }
00056
00057     private float[] bankTargets = { 15f, 30f, 45f, 60f };
00058     private int currentBankTarget = 0;
00059     private TurnStep currentStep;
00060     private bool inputLocked = false;
00061     private float currentBank = 0f;
00062
00063     private PlayableDirector[] bankTimelines;
00064     private PlayableDirector stallTimeline;
00065
00066     private AxisRotationController aoaManipulator;
00067     private BankGhostTrailBehaviour bankGhostTrail;
00068
00076     public void OnEnable()
00077     {
00078         if (rotationInput != null && rotationInput.action != null)
00079         {
00080             rotationInput.action.Enable();
00081         }
00082         // reset progression
00083         currentStep = TurnStep.Bank;
00084         currentBankTarget = 0;
00085         inputLocked = false;
00086         currentBank = 0f;
00087         // Manipulators
00088         aoaManipulator = ActivityHelper.getAxisRotationController(0f, 0f);
00089         bankGhostTrail = ActivityHelper.getBankGhostTrail(true);
00090         // Call helper to find directors for timeline assets
00091         bankTimelines = ActivityHelper.FindPlayableDirector(bankTimelineAssets);
00092         stallTimeline = ActivityHelper.FindPlayableDirector(stallTimelineAsset);
00093     }
00094
00100     public void OnDisable()
00101     {
00102         if (rotationInput != null && rotationInput.action != null)
00103         {
00104             rotationInput.action.Disable();
00105         }
00106         bankGhostTrail.enable = false;
00107     }
00108
00115     public void StartActivity()
00116     {
00117         EnableCurrentStep();
00118     }
00119
00126     public void EnableCurrentStep()
00127     {
00128         inputLocked = false;
00129         hasControllerOrigin = false;
00130     }
00131
00137     void Update()
00138     {
00139         switch ((TurnStep)currentStep)
00140         {
00141             case TurnStep.Bank:
00142                 HandleBanking();
00143                 break;
00144             case TurnStep.stallSetup:
00145                 StallSetup();
00146                 break;
00147             case TurnStep.Stall:
00148                 HandleStall();
00149                 break;
00150             case TurnStep.Complete:
00151                 // Do nothing, wait for MAS to end activity
00152                 break;
00153         }
00154     }
00155
00164     public void HandleBanking()
00165     {
00166         if (inputLocked) return;
00167
```

```
00168         Quaternion currentRotation = rotationInput.action.ReadValue<Quaternion>();
00169
00170         if (!hasControllerOrigin)
00171         {
00172             controllerOrigin = currentRotation;
00173             hasControllerOrigin = true;
00174             Debug.Log("Captured controller origin: " + controllerOrigin.eulerAngles);
00175         }
00176
00177         currentBank = ActivityHelper.getNormalisedPlaneRoll(aoaManipulator);
00178
00179         if (ActivityHelper.checkRotationTargetAchieved(currentBank, bankTargets[currentBankTarget],
      stepAdvanceTolerance))
00180         {
00181             inputLocked = true;
00182             hasControllerOrigin = false;
00183             aoaManipulator.LerpBank(bankTargets[currentBankTarget] * Mathf.Sign(currentBank));
00184             StartCoroutine(ActivityHelper.PlayTimeLine(bankTimelines[currentBankTarget], () =>
00185             {
00186                 EnableCurrentStep();
00187                 currentBankTarget++;
00188                 // TODO uncomment after testing
00189                 mas.OnExternalStepCompleted();
00190                 Debug.Log("currentbank target: " + currentBankTarget + " bankTarget Length: " +
      bankTargets.Length);
00191                 if (currentBankTarget >= bankTargets.Length)
00192                 {
00193                     // Advance to next stage
00194                     currentStep = TurnStep.stallSetup;
00195                     Debug.Log("currentbankTarget: " + currentBankTarget + " cbankTarget length: " +
      bankTargets.Length + " currentStep " + currentStep);
00196                 }
00197             }));
00198         }
00199         else
00200         {
00201             if (useMouseClicks)
00202             {
00203                 ActivityHelper.DebugMouseControlRoll(mouseRollSpeed, aoaManipulator);
00204             }
00205             else
00206             {
00207                 ActivityHelper.controllerRollControl(bankSensitivity, aoaManipulator, rotationInput,
      deadZone, controllerOrigin.eulerAngles.z);
00208             }
00209         }
00210     }
00211
00219     public void StallSetup()
00220     {
00221         inputLocked = true;
00222         // set bank to 60 for initial state of stall timeline
00223         currentBank = ActivityHelper.getNormalisedPlaneRoll(aoaManipulator);
00224         if (currentBank < 0)
00225         {
00226             aoaManipulator.LerpBank(60f, 5f);
00227         }
00228         else
00229         {
00230             aoaManipulator.LerpBank(60f);
00231         }
00232
00233         if (ActivityHelper.checkExactRotationTargetAchieved(currentBank, 60f, stepAdvanceTolerance))
00234         {
00235             aoaManipulator.LerpBank(60f, .5f);
00236             inputLocked = false;
00237             currentStep = TurnStep.Stall;
00238         }
00239     }
00240
00249     public void HandleStall()
00250     {
00251         if (inputLocked) return;
00252
00253         // Read current rotation from the input
00254         Quaternion currentRotation = rotationInput.action.ReadValue<Quaternion>();
00255
00256         if (!hasControllerOrigin)
00257         {
00258             controllerOrigin = currentRotation;
00259             hasControllerOrigin = true;
00260             Debug.Log("Captured controller origin: " + controllerOrigin.eulerAngles);
00261         }
00262
00263         float currentPitch = ActivityHelper.getNormalisedPlanePitch(aoaManipulator);
00264
00265         if (ActivityHelper.checkRotationTargetAchieved(currentPitch, 10f, stepAdvanceTolerance))
```

```
00266         {
00267             inputLocked = true;
00268             hasControllerOrigin = false;
00269             aoaManipulator.LerpAOA(10f);
00270             StartCoroutine(ActivityHelper.PlayTimeLine(stallTimeline, () =>
00271             {
00272                 EnableCurrentStep();
00273                 if (currentBankTarget >= bankTargets.Length)
00274                 {
00275                     // Advance to next stage
00276                     currentStep = TurnStep.Stall;
00277                 }
00278                 // TODO uncomment after testing
00279                 mas.OnExternalStepCompleted();
00280             }));
00281         }
00282         else
00283         {
00284             if (useMouseClicks)
00285             {
00286                 ActivityHelper.DebugMouseControlPitch(mouseRollSpeed, aoaManipulator);
00287             }
00288             else
00289             {
00290                 ActivityHelper.controllerPitchControl(bankSensitivity, aoaManipulator, rotationInput,
      deadZone, controllerOrigin.eulerAngles.x);
00291             }
00292         }
00293     }
00294 }
```

8.15 Assets/ModuleExclusiveAssets/Module2/Section6/Sec6SpinRecoveryActivity.cs File Reference

Classes

- class Sec6SpinRecoveryActivity

  Section 6 spin recovery training activity controller.

8.16 Sec6SpinRecoveryActivity.cs

Go to the documentation of this file.

```
00001 using TMPro;
00002 using UnityEngine;
00003 using UnityEngine.InputSystem;
00004 using UnityEngine.Playables;
00005 using UnityEngine.Timeline;
00006
00032 public class Sec6SpinRecoveryActivity : MonoBehaviour, IActivityController
00033 {
00034     private ModuleActivityScheduler mas => ModuleActivityScheduler.Instance;
00035     public GameObject throttleCanvasPrefab;
00036     private GameObject throttleCanvasInstance;
00037     private TMP_Text throttleText;
00038
00039     [Header("Timelines")]
00040     public TimelineAsset[] pareStepTimelineAssets;
00041
00042     [Header("Input Action References")]
00043     public InputActionReference rotationInput;
00044     public InputActionReference throttleInput;
00045
00046     [Header("Control Settings")]
00047     public float controlSensitivity = 1f;
00048     public float deadZone = 15f;
00049
00050     [Header("Spin Settings")]
00051     public bool spinDirectionRight = true;
00052     public float spinRotationSpeed = 180f;
00053     public float initialSpinBank = 60f;
00054     public float initialSpinAOA = 25f;
00055
```

```
00056     [Header("Editor Debug Settings")]
00057     public bool useMouseInput = false;
00058     public float mouseControlSpeed = 10000f;
00059
00060     private Quaternion controllerOrigin = Quaternion.identity;
00061     private bool hasControllerOrigin = false;
00062
00063     private AxisRotationController aoaManipulator;
00064
00065     private TurnStep currentStep;
00066
00072     private enum TurnStep
00073     {
00074         powerToIdle,
00075         aileronsNeutral,
00076         rudderOpposite,
00077         elevatorForward,
00078         levelFlight
00079     }
00080
00081     private PlayableDirector[] pareStepTimelines;
00082
00083     private bool inputLocked = false;
00084     private float stepStartTime = 0f;
00085     private float minimumStepTime = 0.5f;
00086
00087     // Spin animation
00088     private bool isSpinning = true;
00089     private float spinDirection = 1f;
00090
00091     // Current aircraft state
00092     private float currentThrottle = 1f;
00093     private float currentBank = 0f;
00094     private float currentPitch = 0f;
00095
00096     // Throttle values
00097     private float totalThrottle = 100;
00098     private float throttleIncrement = 20;
00099
00107     public void OnEnable()
00108     {
00109         if (rotationInput != null && rotationInput.action != null)
00110         {
00111             rotationInput.action.Enable();
00112         }
00113         if (throttleInput != null && throttleInput.action != null)
00114         {
00115             throttleInput.action.Enable();
00116         }
00117         // reset progression
00118         currentStep = TurnStep.powerToIdle;
00119         inputLocked = false;
00120         // Manipulators
00121         aoaManipulator = ActivityHelper.getAxisRotationController(0f, 0f);
00122         // Call helper to find directors for timeline assets
00123         pareStepTimelines = ActivityHelper.FindPlayableDirector(pareStepTimelineAssets);
00124
00125         // Initialize spin state
00126         InitializeSpinState();
00127     }
00128
00134     public void OnDisable()
00135     {
00136         if (rotationInput != null && rotationInput.action != null)
00137         {
00138             rotationInput.action.Disable();
00139         }
00140         if (throttleInput != null && throttleInput.action != null)
00141         {
00142             throttleInput.action.Disable();
00143         }
00144     }
00145
00153     private void InitializeSpinState()
00154     {
00155         if (aoaManipulator != null)
00156         {
00157             // Set nose down AOA for stall
00158             aoaManipulator.LerpAOA(initialSpinAOA, 0.5f);
00159
00160             // Set bank angle for spin
00161             float spinBank = spinDirectionRight ? initialSpinBank : -initialSpinBank;
00162             aoaManipulator.LerpBank(spinBank, 0.5f);
00163         }
00164     }
00165
00172     public void StartActivity()
```

```
00173     {
00174         EnableCurrentStep();
00175     }
00176 
00183     public void EnableCurrentStep()
00184     {
00185         inputLocked = false;
00186         hasControllerOrigin = false;
00187         stepStartTime = Time.time;
00188         Debug.Log($"Enabled step: {currentStep}");
00189     }
00190 
00197     void Update()
00198     {
00199         if (isSpinning)
00200         {
00201             SpinPlane();
00202         }
00203 
00204         if (inputLocked) return;
00205 
00206         // Capture controller origin on first input
00207         if (!hasControllerOrigin && rotationInput != null && rotationInput.action != null)
00208         {
00209             Quaternion currentRotation = rotationInput.action.ReadValue<Quaternion>();
00210             controllerOrigin = currentRotation;
00211             hasControllerOrigin = true;
00212             Debug.Log("Captured controller origin: " + controllerOrigin.eulerAngles);
00213         }
00214 
00215         // Read current aircraft state
00216         currentBank = ActivityHelper.getNormalisedPlaneRoll(aoaManipulator);
00217         currentPitch = ActivityHelper.getNormalisedPlanePitch(aoaManipulator);
00218 
00219         // Read throttle if available
00220         if (throttleInput != null && throttleInput.action != null)
00221         {
00222             currentThrottle = throttleInput.action.ReadValue<Vector2>().y;
00223         }
00224 
00225         switch (currentStep)
00226         {
00227             case TurnStep.powerToIdle:
00228                 HandlePowerToIdle();
00229                 break;
00230             case TurnStep.aileronsNeutral:
00231                 HandleAileronsNeutral();
00232                 break;
00233             case TurnStep.rudderOpposite:
00234                 HandleRudderOpposite();
00235                 break;
00236             case TurnStep.elevatorForward:
00237                 HandleElevatorForward();
00238                 break;
00239             case TurnStep.levelFlight:
00240                 // Wait for mas to end activity
00241                 break;
00242         }
00243     }
00244 
00253     private void HandlePowerToIdle()
00254     {
00255         totalThrottle += Mathf.Sign(currentThrottle) * Time.deltaTime * throttleIncrement;
00256         totalThrottle = Mathf.Clamp(totalThrottle, 0, 100);
00257 
00258         Debug.Log($"Total throttle: {totalThrottle}     currentThrottle: {currentThrottle}");
00259         updateThrottleDisplay();
00260         // Check if throttle is at idle
00261         if (totalThrottle < 10)
00262         {
00263             inputLocked = true;
00264             hasControllerOrigin = false;
00265 
00266             Debug.Log("Power to idle achieved!");
00267 
00268             StartCoroutine(ActivityHelper.PlayTimeLine(pareStepTimelines[0], () =>
00269             {
00270                 currentStep = TurnStep.aileronsNeutral;
00271                 // destroy throttle canvas
00272                 Destroy(throttleCanvasInstance);
00273                 EnableCurrentStep();
00274                 mas.OnExternalStepCompleted();
00275             }));
00276         }
00277     }
00278 
00285     private void HandleAileronsNeutral()
```

```
00286     {
00287         inputLocked = true;
00288         StartCoroutine(ActivityHelper.PlayTimeLine(pareStepTimelines[1], () =>
00289         {
00290             currentStep = TurnStep.rudderOpposite;
00291             EnableCurrentStep();
00292             mas.OnExternalStepCompleted();
00293         }));
00294     }
00295 
00308     private void HandleRudderOpposite()
00309     {
00310         // User needs to roll controller opposite to spin direction
00311         bool rudderCorrect = false;
00312 
00313         if (spinDirectionRight)
00314         {
00315             // Right spin needs left rudder (negative bank)
00316             rudderCorrect = currentBank < -10f;
00317         }
00318         else
00319         {
00320             // Left spin needs right rudder (positive bank)
00321             rudderCorrect = currentBank > 10f;
00322         }
00323 
00324         if (rudderCorrect && (Time.time - stepStartTime) > minimumStepTime)
00325         {
00326             inputLocked = true;
00327             hasControllerOrigin = false;
00328 
00329             // Stop the spin!
00330             isSpinning = false;
00331 
00332             Debug.Log("Rudder opposite applied - stopping spin rotation!");
00333 
00334             // Lock at current bank to show rudder applied
00335             aoaManipulator.LerpBank(0, 5f);
00336 
00337             StartCoroutine(ActivityHelper.PlayTimeLine(pareStepTimelines[2], () =>
00338             {
00339                 currentStep = TurnStep.elevatorForward;
00340                 EnableCurrentStep();
00341                 mas.OnExternalStepCompleted();
00342             }));
00343         }
00344         else
00345         {
00346             // Allow user to control bank/rudder
00347             if (useMouseInput)
00348             {
00349                 ActivityHelper.DebugMouseControlRoll(mouseControlSpeed, aoaManipulator);
00350             }
00351             else
00352             {
00353                 ActivityHelper.controllerRollControl(controlSensitivity, aoaManipulator,
    rotationInput, deadZone, controllerOrigin.eulerAngles.z);
00354             }
00355         }
00356     }
00357 
00367     private void HandleElevatorForward()
00368     {
00369         // User needs to pitch controller forward (nose down)
00370         if (currentPitch < -10f && (Time.time - stepStartTime) > minimumStepTime)
00371         {
00372             inputLocked = true;
00373             hasControllerOrigin = false;
00374 
00375             Debug.Log("Elevator forward - breaking the stall!");
00376 
00377             // Break the stall by reducing AOA
00378             aoaManipulator.LerpAOA(3f, 5f);
00379 
00380             StartCoroutine(ActivityHelper.PlayTimeLine(pareStepTimelines[3], () =>
00381             {
00382                 currentStep = TurnStep.levelFlight;
00383                 EnableCurrentStep();
00384                 mas.OnExternalStepCompleted();
00385             }));
00386         }
00387         else
00388         {
00389             // Allow user to control pitch
00390             if (useMouseInput)
00391             {
00392                 ActivityHelper.DebugMouseControlPitch(mouseControlSpeed, aoaManipulator);
```

```
00393             }
00394             else
00395             {
00396                 ActivityHelper.controllerPitchControl(controlSensitivity, aoaManipulator,
00397                     rotationInput, deadZone, controllerOrigin.eulerAngles.x);
00398             }
00399         }
00400     }
00401
00409     private void SpinPlane()
00410     {
00411         // Continuously rotate the aircraft in yaw to simulate spinning
00412         if (aoaManipulator != null)
00413         {
00414             float yawIncrement = spinRotationSpeed * spinDirection;
00415             aoaManipulator.IncrementYaw(yawIncrement * Time.deltaTime);
00416         }
00417     }
00418
00427     private void updateThrottleDisplay()
00428     {
00429         if (throttleCanvasInstance == null)
00430         {
00431             throttleCanvasInstance = Instantiate(throttleCanvasPrefab);
00432             // This is bad, magic numbers are bad. please replace with a more robust system
00433             throttleCanvasInstance.transform.position = new Vector3(798, 1450, 1897);
00434             throttleCanvasInstance.transform.rotation = Quaternion.Euler(0, -154, 0);
00435
00436             // Find the child object named "ThrottleNumber" and get its TMP_Text
00437             Transform textTransform = throttleCanvasInstance.transform.Find("ThrottleNumber");
00438             if (textTransform != null)
00439             {
00440                 throttleText = textTransform.GetComponent<TMP_Text>();
00441             }
00442             else
00443             {
00444                 Debug.LogError("ThrottleNumber object not found in throttleCanvasInstance!");
00445             }
00446         }
00447
00448         // Update the displayed throttle value
00449         if (throttleText != null)
00450         {
00451             throttleText.text = totalThrottle.ToString();
00452         }
00453
00454     }
00455 }
```

8.17 Assets/ModuleExclusiveAssets/Module2/Section7/TiltController.cs File Reference

Classes

- class TiltController

8.18 TiltController.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002 using UnityEngine.InputSystem;
00003 using System.Collections;
00004
00005 public class TiltController : MonoBehaviour, IActivityController
00006 {
00007     private ModuleActivityScheduler mas => ModuleActivityScheduler.Instance; // Reference to MAS
      singleton
00008
00009     public InputActionReference inputAction; // Input action for detecting rotation of controller
00010     public float rotationThreshold = 25f; // Default, can be changed in inspector
00011
00012     // Inspector setting which axis to monitor input for
```

```
00013     public enum RotationAxis { Pitch, Yaw, Roll }
00014     public RotationAxis monitoredAxis;
00015
00016     private Quaternion initialRotation;
00017
00018     private bool activityEnabled = false; // Start as disabled
00019     private bool hasTriggered = false; // Prevent multi-triggering
00020
00021     // Subscribe to event listener and enable input action
00022     private void OnEnable()
00023     {
00024         if (inputAction != null)
00025         {
00026             inputAction.action.performed += OnRotationChanged;
00027             inputAction.action.Enable();
00028         }
00029     }
00030
00031     // Unsubscribe to event listener and disable input action
00032     private void OnDisable()
00033     {
00034         if (inputAction != null)
00035         {
00036             inputAction.action.performed -= OnRotationChanged;
00037             inputAction.action.Disable();
00038         }
00039     }
00040
00041     //
00042     public void StartActivity()
00043     {
00044         activityEnabled = true;
00045         hasTriggered = false;
00046         StartCoroutine(CaptureInitialRotationDelayed());
00047     }
00048
00049     // Capture a neutral position to calculate rotation change from (prevent instant triggering,
      depending on which direction the user has the controller sat in when the activity begins)
00050     private IEnumerator CaptureInitialRotationDelayed()
00051     {
00052         yield return new WaitForSeconds(0.1f); // short delay
00053         if (inputAction != null)
00054         {
00055             initialRotation = inputAction.action.ReadValue<Quaternion>();
00056             Debug.Log($"[TiltController] Captured neutral: {initialRotation.eulerAngles}");
00057         }
00058     }
00059
00060     // Called whenever rotation input is received (likely every frame let's be real here)
00061     private void OnRotationChanged(InputAction.CallbackContext ctx)
00062     {
00063         if (!activityEnabled || hasTriggered) return;
00064
00065         // Get the current angle of rotation from the controller
00066         Quaternion currentRotation = ctx.ReadValue<Quaternion>();
00067
00068         // Obtain the difference between the current rotation and the initial neutral rotation
00069         Vector3 deltaEuler = (Quaternion.Inverse(initialRotation) * currentRotation).eulerAngles;
00070
00071         // Normalize from 0 to 360deg to -180 to 180deg using helper script for predictable rotation
      comparison
00072         deltaEuler.x = NormalizeAngle(deltaEuler.x);
00073         deltaEuler.y = NormalizeAngle(deltaEuler.y);
00074         deltaEuler.z = NormalizeAngle(deltaEuler.z);
00075
00076         // Pick the axis were monitoring
00077         float valueToCheck = 0f;
00078         switch (monitoredAxis)
00079         {
00080             case RotationAxis.Pitch:
00081                 valueToCheck = deltaEuler.x;
00082                 break;
00083             case RotationAxis.Yaw:
00084                 valueToCheck = deltaEuler.y;
00085                 break;
00086             case RotationAxis.Roll:
00087                 valueToCheck = deltaEuler.z;
00088                 break;
00089         }
00090
00091         // The comparison! If rotation exceeds the threshold of the selected axis (in the correct
      polarity/direction +/-), we are in business!
00092         if ((rotationThreshold > 0 && valueToCheck >= rotationThreshold) || (rotationThreshold < 0 &&
      valueToCheck <= rotationThreshold))
00093         {
00094             hasTriggered = true;
00095             mas.OnExternalStepCompleted();
```

```
00096
00097             activityEnabled = false;
00098             Destroy(gameObject);
00099         }
00100
00101         Debug.Log($"dEuler = {deltaEuler} | valueToCheck({monitoredAxis}) = {valueToCheck}");
00102     }
00103
00104     // Normalize angle to workable range if applicable
00105     private float NormalizeAngle(float angle)
00106     {
00107         if (angle > 180f) angle -= 360f;
00108         return angle;
00109     }
00110 }
```

8.19 Assets/Scripts/ActivityStructure/ActivityHelper.cs File Reference

Classes

- class ActivityHelper

  Static helper class for activity management and control.

8.20 ActivityHelper.cs

Go to the documentation of this file.

```
00001 using System.Collections;
00002 using NUnit.Framework.Interfaces;
00003 using Unity.Burst.CompilerServices;
00004 using UnityEngine;
00005 using UnityEngine.InputSystem;
00006 using UnityEngine.Playables;
00007 using UnityEngine.Timeline;
00008
00021 public static class ActivityHelper
00022 {
00033     public static PlayableDirector FindPlayableDirector(TimelineAsset asset)
00034     {
00035         // Check for null asset
00036         if (asset == null)
00037         {
00038             Debug.LogWarning("TimeLineAsset is NULL: Cannot find PlayableDirector");
00039             return null;
00040         }
00041
00042         // Get all PlayableDirectors in the scene
00043         PlayableDirector[] allDirectors = Object.FindObjectsOfType<PlayableDirector>();
00044         // Search for the Director that uses the specified asset
00045         for (int i = 0; i < allDirectors.Length; i++)
00046         {
00047             // Get the next Director
00048             PlayableDirector director = allDirectors[i];
00049             if (director.playableAsset == asset)
00050             {
00051                 return director;
00052             }
00053         }
00054
00055         Debug.LogWarning("No PlayableDirector found for timeline asset: " + asset.name);
00056         return null;
00057     }
00058
00069     public static PlayableDirector[] FindPlayableDirector(TimelineAsset[] assets)
00070     {
00071         // Check for null or empty array
00072         if (assets == null || assets.Length == 0)
00073         {
00074             Debug.LogWarning("TimeLineAsset array is NULL or empty: Cannot find PlayableDirectors");
00075             return null;
00076         }
00077
00078         // Get all PlayableDirectors in the scene
00079         PlayableDirector[] allDirectors = Object.FindObjectsOfType<PlayableDirector>();
```

```
00080         // Create an array to hold found Directors
00081         PlayableDirector[] foundDirectors = new PlayableDirector[assets.Length];
00082
00083         // Search for Directors for each asset
00084         for (int i = 0; i < assets.Length; i++)
00085         {
00086             // Get the next asset
00087             TimelineAsset asset = assets[i];
00088             foundDirectors[i] = null;
00089
00090             // Check for null asset
00091             if (asset != null)
00092             {
00093                 // Search for the Director that uses this asset
00094                 for (int j = 0; j < allDirectors.Length; j++)
00095                 {
00096                     PlayableDirector director = allDirectors[j];
00097                     if (director.playableAsset == asset)
00098                     {
00099                         foundDirectors[i] = director;
00100                         break;
00101                     }
00102                 }
00103                 // If still null
00104                 if (foundDirectors[i] == null)
00105                 {
00106                     Debug.LogWarning("No PlayableDirector found for timeline asset: " + asset.name);
00107                 }
00108             }
00109             else
00110             {
00111                 Debug.LogWarning("TimelineAsset at index " + i + " is NULL: Cannot find
      PlayableDirector");
00112             }
00113         }
00114         return foundDirectors;
00115     }
00116
00125     public static ProceduralAirflow getProceduralAirflow()
00126     {
00127         try
00128         {
00129             return GameObject.FindObjectOfType<ProceduralAirflow>();
00130         }
00131         catch
00132         {
00133             Debug.LogError("ProceduralAirflow not found in scene!");
00134             return null;
00135         }
00136     }
00137
00146     public static AxisRotationController getAxisRotationController()
00147     {
00148         try
00149         {
00150             return GameObject.FindObjectOfType<AxisRotationController>();
00151         }
00152         catch
00153         {
00154             Debug.LogError("AOAManipulator not found in scene!");
00155             return null;
00156         }
00157     }
00158
00169     public static AxisRotationController getAxisRotationController(float aoa = 0, float bank = 0)
00170     {
00171         try
00172         {
00173             AxisRotationController manip = GameObject.FindObjectOfType<AxisRotationController>();
00174             manip.LerpAOA(aoa);
00175             manip.LerpBank(bank);
00176             return manip;
00177         }
00178         catch
00179         {
00180             Debug.LogError("AOAManipulator not found in scene!");
00181             return null;
00182         }
00183     }
00184
00193     public static BankGhostTrailBehaviour getBankGhostTrail(bool enable = true)
00194     {
00195         try
00196         {
00197             BankGhostTrailBehaviour trail = GameObject.FindObjectOfType<BankGhostTrailBehaviour>();
00198             trail.enable = enable;
00199             return trail;
```

```
00200         }
00201         catch
00202         {
00203             Debug.LogError("BankGhostTrailBehaviour not found in scene!");
00204             return null;
00205         }
00206     }
00207
00219     public static float normalizedControllerZRotation(InputActionReference input, float initial = 0)
00220     {
00221         Quaternion controllerRotation = input.action.ReadValue<Quaternion>();
00222
00223         // Create initial rotation quaternion
00224         Quaternion initialRotation = Quaternion.Euler(0, 0, initial);
00225
00226         // Calculate relative rotation
00227         Quaternion relativeRotation = Quaternion.Inverse(initialRotation) * controllerRotation;
00228
00229         // Extract Z rotation from relative quaternion
00230         Vector3 relativeEuler = relativeRotation.eulerAngles;
00231         float zRotation = Mathf.DeltaAngle(0, relativeEuler.z);
00232
00233         //Debug.Log($"controller rotation: {controllerRotation.eulerAngles}  initial z rotation:
    {initial}  normalised z rotation {zRotation}");
00234         return zRotation;
00235     }
00236
00248     public static float normalizedControllerXRotation(InputActionReference input, float initial = 0)
00249     {
00250         Quaternion controllerRotation = input.action.ReadValue<Quaternion>();
00251
00252         // Create initial rotation quaternion
00253         Quaternion initialRotation = Quaternion.Euler(initial, 0, 0);
00254
00255         // Calculate relative rotation
00256         Quaternion relativeRotation = Quaternion.Inverse(initialRotation) * controllerRotation;
00257
00258         // Extract X rotation from relative quaternion
00259         Vector3 relativeEuler = relativeRotation.eulerAngles;
00260         float xRotation = Mathf.DeltaAngle(0, relativeEuler.x);
00261
00262         //Debug.Log($"controller rotation: {controllerRotation.eulerAngles}  initial x rotation:
    {initial}  normalised x rotation {xRotation}");
00263         return xRotation;
00264     }
00265
00275     public static void DebugMouseControlRoll(float speed, AxisRotationController manip)
00276     {
00277         if (Mouse.current.leftButton.isPressed)
00278         {
00279             manip.IncrementBank(-speed * Time.deltaTime);  // Negative left
00280         }
00281         if (Mouse.current.rightButton.isPressed)
00282         {
00283             manip.IncrementBank(speed * Time.deltaTime);  // Positive right
00284         }
00285     }
00286
00296     public static void DebugMouseControlPitch(float speed, AxisRotationController manip)
00297     {
00298         if (Mouse.current.leftButton.isPressed)
00299         {
00300             manip.IncrementAOA(-speed * Time.deltaTime);  // Negative left
00301         }
00302         if (Mouse.current.rightButton.isPressed)
00303         {
00304             manip.IncrementAOA(speed * Time.deltaTime);  // Positive right
00305         }
00306     }
00307
00316     public static void DebugMouseControlRollLeft(float speed, AxisRotationController manip)
00317     {
00318         if (Mouse.current.rightButton.isPressed)
00319         {
00320             //Debug.Log("Using Mouse Clicks for Roll" + speed + " " + Time.deltaTime);
00321             manip.IncrementBank(speed * Time.deltaTime);
00322         }
00323     }
00324
00333     public static void DebugMouseControlPitchUp(float speed, AxisRotationController manip)
00334     {
00335         if (Mouse.current.leftButton.isPressed)
00336         {
00337             manip.IncrementAOA(speed * Time.deltaTime);
00338         }
00339     }
00340
```

```
00354     public static void controllerRollControl(float speed, AxisRotationController manip,
      InputActionReference input, float deadZone, float initial = 0, Vector2 vector = default)
00355     {
00356         if (vector == default)
00357         {
00358             vector.x = -70f;
00359             vector.y = 70f;
00360         }
00361 
00362         float planeX = Mathf.DeltaAngle(0, manip.getRotation().eulerAngles.x);
00363         planeX = clampRange(planeX, vector);
00364         manip.setRoll(planeX);
00365 
00366         float zRotation = normalizedControllerZRotation(input, initial);
00367         if (Mathf.Abs(zRotation) > deadZone)
00368         {
00369             zRotation = getRotationSpeed(zRotation, speed, deadZone);
00370             manip.IncrementBank(zRotation * Time.deltaTime);
00371         }
00372     }
00373 
00388     public static void controllerPitchControl(float speed, AxisRotationController manip,
      InputActionReference input, float deadZone, float initial = 0, Vector2 vector = default)
00389     {
00390         if (vector == default)
00391         {
00392             vector.x = -40f;
00393             vector.y = 40f;
00394         }
00395 
00396         float planeZ = Mathf.DeltaAngle(0, manip.getRotation().eulerAngles.z);
00397         planeZ = clampRange(planeZ, vector);
00398         manip.setPitch(planeZ);
00399 
00400         float xRotation = normalizedControllerXRotation(input, initial);
00401         if (Mathf.Abs(xRotation) > deadZone)
00402         {
00403             // reverse pitch control as pulling back increases AOA
00404             xRotation = getRotationSpeed(xRotation, speed, deadZone);
00405             manip.IncrementAOA(-1 * xRotation * Time.deltaTime);
00406         }
00407     }
00408 
00416     public static float clampRange(float rotation, Vector2 range)
00417     {
00418         float result = Mathf.Clamp(rotation, range.x, range.y);
00419         return result;
00420     }
00421 
00433     public static float getRotationSpeed(float rotation, float speed, float deadZone)
00434     {
00435         // Make roll speed proportional to how far past the deadzone the input is
00436         float rotSpeed = (Mathf.Abs(rotation) - deadZone) * speed;
00437         rotSpeed = Mathf.Sign(rotation) * rotSpeed;
00438         return rotSpeed;
00439     }
00440 
00452     public static bool checkRotationTargetAchieved(float currentRotation, float targetRotation, float
      tolerance)
00453     {
00454         if (Mathf.Abs(Mathf.Abs(currentRotation) - targetRotation) < tolerance)
00455         {
00456             return true;
00457         }
00458         return false;
00459     }
00460 
00472     public static bool checkExactRotationTargetAchieved(float currentRotation, float targetRotation,
      float tolerance)
00473     {
00474         //Debug.Log("Exact Rotation: " + (currentRotation - targetRotation));
00475         if (Mathf.Abs(currentRotation - targetRotation) < tolerance)
00476         {
00477             return true;
00478         }
00479         return false;
00480     }
00481 
00491     public static float getNormalisedPlaneRoll(AxisRotationController manip)
00492     {
00493         float xRot = manip.getRotation().eulerAngles.x;
00494         return Mathf.DeltaAngle(0f, xRot);
00495     }
00496 
00506     public static float getNormalisedPlanePitch(AxisRotationController manip)
00507     {
00508         float zRot = manip.getRotation().eulerAngles.z;
```

```
00509         return Mathf.DeltaAngle(0f, zRot);
00510     }
00511
00524     public static IEnumerator PlayTimeLine(PlayableDirector director, System.Action onComplete, float
      fallBackWait = 1f)
00525     {
00526         if (director != null && director.playableAsset != null)
00527         {
00528             director.Play();
00529
00530             bool isComplete = false;
00531             // Subscribe to the stopped event
00532             // Lambda called when timeline ends setting isComplete to true
00533             director.stopped += (d) => { isComplete = true; };
00534
00535             // Wait until timeline is complete
00536             while (!isComplete)
00537             {
00538                 yield return null;
00539             }
00540             // Call lambda fuction passed in to signal activity complete
00541             onComplete.Invoke();
00542         }
00543         else
00544         {
00545             Debug.Log("Yeilding as fallback for unassigned PlayableDirector (Timeline)");
00546             yield return new WaitForSeconds(fallBackWait);
00547             onComplete.Invoke();
00548         }
00549     }
00550 }
```

8.21 Assets/Scripts/ActivityStructure/IActivityController.cs File Reference

Classes

- interface IActivityController

  Interface class.

8.22 IActivityController.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002
00008
00009 public interface IActivityController
00010 {
00011     void StartActivity();
00012 }
```

8.23 Assets/Scripts/CommonActivityBehaviour/AccelerateScript.cs File Reference

Classes

- class AccelerateScript

8.24 AccelerateScript.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002 using UnityEngine.InputSystem;
00003 using System.Collections;
00004
00005
00006 // I COPIED INCREASETHROTTLE.CS AND ADDDED COROUTINE TIMER!
00007 public class AccelerateScript : MonoBehaviour, IActivityController
00008 {
00009     private ModuleActivityScheduler mas => ModuleActivityScheduler.Instance; // Reference to MAS
      singleton
00010
00011     public InputActionReference inputAction; // Input action for control pad on left controller
00012     public float throttleThreshold = 0.1f; // Default, can be changed in inspector
00013     private bool activityEnabled = false; // Start as disabled
00014
00015     private bool hasTriggered = false; // Prevent multi-triggering
00016
00017     // Subscribe to event listener and enable input action
00018     private void OnEnable()
00019     {
00020         if (inputAction != null)
00021         {
00022             inputAction.action.performed += OnPadMoved;
00023             inputAction.action.canceled += OnPadReleased;
00024             inputAction.action.Enable();
00025         }
00026     }
00027
00028
00029     IEnumerator WaitCoroutine() {
00030         yield return new WaitForSeconds(5);
00031     }
00032
00033     // Unsubscribe to event listener and disable input action
00034     private void OnDisable()
00035     {
00036         if (inputAction != null)
00037         {
00038             inputAction.action.performed -= OnPadMoved;
00039             inputAction.action.canceled -= OnPadReleased;
00040             inputAction.action.Disable();
00041         }
00042     }
00043
00044     //
00045     public void StartActivity()
00046     {
00047         activityEnabled = true;
00048         hasTriggered = false;
00049     }
00050
00051     private void OnPadMoved(InputAction.CallbackContext ctx)
00052     {
00053         if (!activityEnabled || hasTriggered) return;
00054
00055         Debug.Log("Left control pad moved!");
00056
00057         Vector2 axis = ctx.ReadValue<Vector2>(); // Get the axis to be read from the controller
00058
00059
00060         // If axis.y > throttleThreshold for timerSeconds
00061
00062         // Detect if control pad is pulled up/north beyond threshold
00063         if (axis.y > throttleThreshold)
00064         {
00065             StartCoroutine(WaitCoroutine()); // Wait for 5 seconds within range.
00066
00067             // Step is considered completed
00068             hasTriggered = true;
00069             mas.OnExternalStepCompleted();
00070
00071             activityEnabled = false;
00072             Destroy(gameObject);
00073         }
00074     }
00075
00076     private void OnPadReleased(InputAction.CallbackContext ctx)
00077     {
00078         // Reset trigger when the player releases the stick
00079         Vector2 axis = ctx.ReadValue<Vector2>();
00080         if (Mathf.Abs(axis.y) < 0.1f)
00081         {
```

```
00082             hasTriggered = false;
00083         }
00084     }
00085
00086
00087     // I added an accesor method.
00088     public bool getHasTriggered() {
00089         return hasTriggered;
00090     }
00091 }
```

8.25 Assets/Scripts/CommonActivityBehaviour/ControllerBackward.cs File Reference

Classes

- class ControllerBackward

8.26 ControllerBackward.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002 using UnityEngine.InputSystem;
00003 using UnityEngine.XR.Interaction.Toolkit;
00004 using UnityEngine.XR.Interaction.Toolkit.Interactables;
00005
00009 public class ControllerBackward : MonoBehaviour, IActivityController
00010 {
00011     private ModuleActivityScheduler mas => ModuleActivityScheduler.Instance; // Reference to MAS
      singleton
00012
00013     public InputActionReference rotateAction; // Input action for rotating the left controller
00014
00015     [Range(0,1)]
00016     public float intensity = 1f;
00017     public float duration = 5f;
00018
00019     public float minZPositionThreshold = 0f; // Default, can be changed in inspector
00020     public float maxZPositionThreshold = 90f; // 1.0f
00021
00022     private bool activityEnabled = false; // Start as disabled
00023
00024
00025     public void Start()
00026     {
00027         XRBaseInteractable interactable = GetComponent<XRBaseInteractable>();
00028         interactable.activated.AddListener(TriggerHaptic);
00029     }
00030
00031     // Subscribe to event listener and enable input action
00032     private void OnEnable()
00033     {
00034         if (rotateAction != null)
00035         {
00036             //rotateAction.action.performed += OnRotatePerformed;
00037             rotateAction.action.performed += OnPositionChange;
00038             rotateAction.action.Enable();
00039         }
00040     }
00041
00042     // Unsubscribe to event listener and disable input action
00043     private void OnDisable()
00044     {
00045         if (rotateAction != null)
00046         {
00047             //rotateAction.action.performed -= OnRotatePerformed;
00048             rotateAction.action.performed -= OnPositionChange;
00049             rotateAction.action.Disable();
00050         }
00051     }
00052
00053     //
```

```
00054     public void StartActivity()
00055     {
00056         activityEnabled = true;
00057         //      rotateAction.action.Enable();
00058     }
00059 
00060     // Wrapper for TriggerHaptic method.
00061     public void TriggerHaptic(BaseInteractionEventArgs e) {
00062 
00063         if (e.interactorObject is UnityEngine.XR.Interaction.Toolkit.Interactors.XRBaseInputInteractor
      controllerInteractor) {
00064             TriggerHaptic(controllerInteractor.xrController);
00065         }
00066     }
00067 
00068     public void TriggerHaptic(XRBaseController controller) {
00069         if (intensity > 0) {
00070             controller.SendHapticImpulse(intensity, duration);
00071         }
00072     }
00073 
00074 
00075     private void OnPositionChange(InputAction.CallbackContext ctx)
00076     {
00077 
00078         Quaternion controllerPosition = ctx.ReadValue<Quaternion>();
00079         float zPosition = Mathf.DeltaAngle(0f, controllerPosition.normalized.z);
00080         //float position = ctx.ReadValue<float>();// Get the value from the controller
00081 
00082         if (!activityEnabled) return;
00083 
00084         // Same logic as ControllerForward.cs but using negative values this time.
00085         if (zPosition >= -minZPositionThreshold && zPosition <= -maxZPositionThreshold)
00086         {
00087 
00088             // Send a haptic to the user so the user knows they are within range, also could be a good
      idea to add an angle meter for the DA-40 rotation.
00089 
00090 
00091             AxisRotationController aoaManip =
      GameObject.FindAnyObjectByType<AxisRotationController>();
00092             aoaManip.LerpAOA(-10); // Lower AOA by 10 degrees.
00093             //currentIndex++;
00094 
00095             mas.OnExternalStepCompleted();
00096 
00097             // This is a one-action activity, so disable after completed input
00098             activityEnabled = false;
00099 
00100             // Clean-up
00101             Destroy(gameObject); // there is only one step in the activity associated with this
      script.
00102         }
00103     }
00104 }
```

8.27 Assets/Scripts/CommonActivityBehaviour/DetectRotation.cs File Reference

Classes

- class DetectRotation

8.28 DetectRotation.cs

Go to the documentation of this file.

```
00001 using System;
00002 using UnityEngine;
00003 using UnityEngine.InputSystem;
00004 
00005 public class DetectRotation : MonoBehaviour
00006 {
```

```
00007
00008
00009     public GameObject rightController;
00010     public GameObject DA_40;
00011
00012     public InputActionReference locomotionToggle;
00013
00014     // Toggle for d4_40 rotation on and off during activities or module.
00015     public Boolean rotationToggle = false;
00016
00017
00018
00019     // Test
00020
00021     // InputActionReference
00022     //public void OnEnable()
00023     //{
00024
00025     //}
00026
00027     //
00028
00029
00030     // Update is called once per frame
00031     void Update()
00032     {
00033
00034         var rightControllerPosition = rightController.transform.position;
00035         var rightControllerRotation = rightController.transform.rotation;
00036         var DA_40_r = DA_40.transform.rotation; // Set to quaternion type..
00037         //Debug.Log("rPos: " + rightControllerPosition + " rRot: " + rightControllerRotation);
00038
00039         if (rightControllerRotation != null) {
00040
00041             // Moving left
00042             if (rightControllerRotation.z > 0.090) {
00043                 //Debug.Log("moving left");
00044
00045                 if (rotationToggle == true)
00046                 {
00047                     DA_40.transform.rotation = rightControllerRotation; // on axis...?
00048                 }
00049             }
00050
00051             // Median.
00052             if (rightControllerRotation.z <= 0.80 || rightControllerRotation.z >= 0.059) {
00053                 //Debug.Log("In the median range.");
00054
00055                 if (rotationToggle == true)
00056                 {
00057                     DA_40.transform.rotation = rightControllerRotation;
00058                 }
00059             }
00060
00061             // Moving right.
00062             if (rightControllerRotation.z < 0.060) {
00063                 //Debug.Log("moving right");
00064
00065                 if (rotationToggle == true)
00066                 {
00067                     DA_40.transform.rotation = rightControllerRotation;
00068                 }
00069             }
00070
00071         }
00072
00073     }
00074
00075 }
```

8.29 Assets/Scripts/CommonActivityBehaviour/RotateLeftController.cs File Reference

Classes

- class RotateLeftController

8.30 RotateLeftController.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002 using UnityEngine.InputSystem;
00003
00007 public class RotateLeftController : MonoBehaviour, IActivityController
00008 {
00009     private ModuleActivityScheduler mas => ModuleActivityScheduler.Instance; // Reference to MAS
      singleton
00010
00011     public InputActionReference rotateAction; // Input action for rotating the left controller
00012     public float rotationThreshold = 0.1f; // Default, can be changed in inspector
00013     private bool activityEnabled = false; // Start as disabled
00014
00015     // Subscribe to event listener and enable input action
00016     private void OnEnable()
00017     {
00018         if (rotateAction != null)
00019         {
00020             rotateAction.action.performed += OnRotatePerformed;
00021             rotateAction.action.Enable();
00022         }
00023     }
00024
00025     // Unsubscribe to event listener and disable input action
00026     private void OnDisable()
00027     {
00028         if (rotateAction != null)
00029         {
00030             rotateAction.action.performed -= OnRotatePerformed;
00031             rotateAction.action.Disable();
00032         }
00033     }
00034
00035     //
00036     public void StartActivity()
00037     {
00038         activityEnabled = true;
00039 //      rotateAction.action.Enable();
00040     }
00041
00042
00043
00044
00055     private void OnRotatePerformed(InputAction.CallbackContext ctx)
00056     {
00057         if (!activityEnabled) return; // Only process if the activity is...y'know, active...
00058
00059         float rotationAmount = ctx.ReadValue<float>();// Get the value from the controller
00060
00061         if (rotationAmount >= rotationThreshold)
00062         {
00063
00064             AxisRotationController aoaManip =
      GameObject.FindAnyObjectByType<AxisRotationController>();
00065             aoaManip.LerpAOA(-10); // Lower AOA by 10 degrees. // this value may be needed to change
      if you want to reuse the script for multiple rotation angles.
00066
00067             mas.OnExternalStepCompleted();
00068
00069             // This is a one-action activity, so disable after completed input
00070             activityEnabled = false;
00071
00072             // Clean-up
00073             Destroy(gameObject);
00074         }
00075     }
00076
00077 }
```

8.31 Assets/Scripts/CustomTimelineTracks/MaterialTimelineTrack/↩ MaterialBehaviour.cs File Reference

Classes

- class MaterialBehaviour

8.32 MaterialBehaviour.cs

Go to the documentation of this file.

```
00001 using TMPro;
00002 using UnityEngine;
00003 using UnityEngine.Playables;
00004
00005 public class MaterialBehaviour : PlayableBehaviour
00006 {
00007
00008     public Material currentMaterial;
00009     public Texture targetTexture; // new texture.
00010     //public Material targetMaterial;
00011     public override void ProcessFrame(Playable playable, FrameData info, object playerData)
00012     {
00013
00014         Material mat = playerData as Material;
00015         //Material tMat = playerData as Material;
00016         Texture tex = playerData as Texture;
00017
00018         // Update the material in the timeline.
00019         mat = currentMaterial;
00020         tex = targetTexture;
00021
00022         //Debug.Log("tMat" + tMat.name);
00023
00024         // Reassign the texture of a material! This will alter the material itself!
00025         mat.mainTexture = tex;
00026
00028         //mat.color = tMat.color;
00029
00030     }
00031 }
```

8.33 Assets/Scripts/CustomTimelineTracks/MaterialTimelineTrack/↩ MaterialClip.cs File Reference

Classes

- class MaterialClip

8.34 MaterialClip.cs

Go to the documentation of this file.

```
00001 using UnityEditor;
00002 using UnityEngine;
00003 using UnityEngine.Playables;
00004
00005 public class MaterialClip : PlayableAsset
00006 {
00007
00008     // MaterialClip is used in timeline and can edit the materials.
00009     public Material currentMaterial;
00010     public Texture targetTexture;
00011     //public Material targetMaterial;
00012
00013     public override Playable CreatePlayable(PlayableGraph graph, GameObject owner)
00014     {
00015
00016         // update this to material behaviour:
00017         var playable = ScriptPlayable<MaterialBehaviour>.Create(graph);
00018         MaterialBehaviour materialBehaviour = playable.GetBehaviour();
00019
00020
00021         // Assign MaterialBehaviour variables.
00022         materialBehaviour.currentMaterial = currentMaterial;
00023         materialBehaviour.targetTexture = targetTexture;
00024         //materialBehaviour.targetMaterial = targetMaterial;
00025
00026         return playable; // return vars to playable.
00027
00028     }
00029 }
```

8.35 Assets/Scripts/CustomTimelineTracks/MaterialTimelineTrack/↩ MaterialTrack.cs File Reference

Classes

- class MaterialTrack

8.36 MaterialTrack.cs

Go to the documentation of this file.

```
00001 using TMPro;
00002 using UnityEngine;
00003 using UnityEngine.Timeline;
00004
00005
00006
00007
00008 [TrackBindingType(typeof(SkinnedMeshRenderer))] // Put the SkinnedMeshRenderer of the plane in here!
      // can change the type to just MeshRenderer if you want.
00009 [TrackClipType(typeof(MaterialClip))] // use materialClip script.
00010
00011 // this is the track to change material of an object.
00012 // use trackasset type.
00013 public class MaterialTrack : TrackAsset
00014 {
00015     // Start is called once before the first execution of Update after the MonoBehaviour is created
00016     void Start()
00017     {
00018
00019     }
00020
00021     // Update is called once per frame
00022     void Update()
00023     {
00024
00025     }
00026 }
```

8.37 Assets/Scripts/CustomTimelineTracks/SubtitleTimelineTrack/↩ SubtitleBehaviour.cs File Reference

Classes

- class SubtitleBehaviour

8.38 SubtitleBehaviour.cs

Go to the documentation of this file.

```
00001 using TMPro;
00002 using UnityEngine;
00003 using UnityEngine.Playables;
00004
00005 public class SubtitleBehaviour : PlayableBehaviour
00006 {
00007
00008     public string subtitleText;
00009     public Color subtitleColor;
00010
00011     // Update this later to make the text color serializble.
00012
00013     public override void ProcessFrame(Playable playable, FrameData info, object playerData)
00014     {
00015         TextMeshProUGUI text = playerData as TextMeshProUGUI;
00016         text.text = subtitleText;
00017         //text.color = new Color(0, 0, 0, info.weight); // The subtitle text is instantiated as black.
00018
00019     }
00020
00021 }
```

8.39 Assets/Scripts/CustomTimelineTracks/SubtitleTimelineTrack/↩ SubtitleClip.cs File Reference

Classes

- class SubtitleClip

8.40 SubtitleClip.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002 using UnityEngine.Playables;
00003
00004
00005 public class SubtitleClip : PlayableAsset
00006 {
00007     public Color subtitleColor;
00008     public string subtitleText;
00009
00010     public override Playable CreatePlayable(PlayableGraph graph, GameObject owner)
00011     {
00012         var playable = ScriptPlayable<SubtitleBehaviour>.Create(graph);
00013
00014         SubtitleBehaviour subtitleBehaviour = playable.GetBehaviour();
00015         subtitleBehaviour.subtitleText = subtitleText;
00016
00017         // GET THE PLAYER DATA SO I CAN GET THE TEXT.COLOR!
00018         //subtitleBehaviour.text.color =
00019         subtitleBehaviour.subtitleColor = subtitleColor;
00020
00021
00022         return playable;
00023
00024     }
00025 }
```

8.41 Assets/Scripts/CustomTimelineTracks/SubtitleTimelineTrack/↩ SubtitleTrack.cs File Reference

Classes

- class SubtiitleTrack

8.42 SubtitleTrack.cs

Go to the documentation of this file.

```
00001 using System.Collections;
00002 using System.Collections.Generic;
00003 using UnityEngine;
00004 using UnityEngine.Timeline;
00005 using TMPro;
00006
00007
00008 [TrackBindingType(typeof(TextMeshProUGUI))]
00009 [TrackClipType(typeof(SubtitleClip))]
00010 public class SubtiitleTrack : TrackAsset
00011 {
00012
00013 }
```

8.43 Assets/Scripts/DA_40_SpecialFeatures/BankGhostTrailBehaviour.cs File Reference

Classes

- class BankGhostTrailBehaviour

8.44 BankGhostTrailBehaviour.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002
00003 public class BankGhostTrailBehaviour : MonoBehaviour
00004 {
00005     [Header("enable")]
00006     public bool enable = true;
00007     [Header("references")]
00008     public ParticleSystem ghostTrail;
00009     [Header("Ghost Trail Motion")]
00010     public float velocity = 50f;
00011     private ParticleSystem.Particle[] ghostParticles;
00012
00018     void Update()
00019     {
00020         if (enable)
00021         {
00022             UpdateGhostTrailMotion();
00023         }
00024     }
00025
00044     void UpdateGhostTrailMotion()
00045     {
00046         if (ghostTrail == null) return;
00047
00048         if (ghostParticles == null || ghostParticles.Length < ghostTrail.main.maxParticles)
00049         {
00050             ghostParticles = new ParticleSystem.Particle[ghostTrail.main.maxParticles];
00051         }
00052
00053         int count = ghostTrail.GetParticles(ghostParticles);
00054         //Debug.Log("Particle count: " + count);
00055         if (count == 0) return;
00056
00057         // Get bank angle
00058         float bankAngle = transform.rotation.eulerAngles.x;
00059         // convert from unsigned 0-360 to range -180 and 180
00060         float signedBank = Mathf.DeltaAngle(0f, bankAngle);
00061
00062         // Check if in level flight
00063         if (Mathf.Abs(signedBank) < 1f)
00064         {
00065             // return, particle system handles level flight by default
00066             return;
00067         }
00068
00069         /* Formula for calculating turning radius
00070          * R = V^2 / (g * tan(BankAngle)
00071          * R = Radius m
00072          * V = velocity m/s
00073          * g = gravity m/s
00074          * bankAngle = degrees
00075         */
00076
00077         float g = 9.81f;
00078         // convert bank angle from degrees to radians
00079         float bankRadians = Mathf.Abs(signedBank) * Mathf.Deg2Rad;
00080         // Mathf.Tan expects radians not degrees
00081         float tanResult = Mathf.Tan(bankRadians);
00082
00083         // Calculate radius
00084         float radius = (velocity * velocity) / (g * tanResult);
00085         // turn direction -1 = right 1 = left
00086         float turnDir = Mathf.Sign(signedBank);
00087
00088         // The planes forward axis is +x therefore transform.forward returns the direction the planes
      +z axis is facing
```

```
00089         Vector3 sidewaysDir = transform.forward;
00090         // Isolate horizontal x and z directions
00091         sidewaysDir.y = 0;
00092         //Normalize horizontal dir
00093         sidewaysDir.Normalize();
00094         // change direction to left or right
00095         sidewaysDir *= turnDir;
00096 
00097         // calculate turn centre
00098         Vector3 turnCentre = transform.position + sidewaysDir * radius;
00099 
00100         float verticalComponent = -transform.right.y;
00101 
00102         // Update Each Ghost Particle
00103         for (int i = 0; i < count; i++)
00104         {
00105             // particle position
00106             Vector3 pos = ghostParticles[i].position;
00107             // get vector from current pos to centre
00108             Vector3 toCentre = turnCentre - pos;
00109             // remove y conponent
00110             toCentre.y = 0f;
00111             // normalize
00112             Vector3 dirToCentre = toCentre.normalized;
00113 
00114             // calculate the "tangent" using the cross product of the diirection to centre and
      vector3.down
00115             // returns the a vector perpendicular to both inputs
00116             Vector3 tangentDir = Vector3.Cross(Vector3.down, dirToCentre).normalized;
00117             // flip the tangent direction based on the turn side.
00118             tangentDir *= turnDir;
00119 
00120             // Add vertical component
00121             tangentDir.y = verticalComponent;
00122 
00123             // set the new velocity along the tangent
00124             ghostParticles[i].velocity = tangentDir * velocity;
00125 
00126             //Debug.Log("Particles velocity" + tangentDir*velocity);
00127         }
00128         // Pass the updated particles back to the ParticleSystem
00129         ghostTrail.SetParticles(ghostParticles, count);
00130     }
00131 
00137     void OnDrawGizmosSelected()
00138     {
00139         if (ghostTrail == null) return;
00140 
00141         // Get bank angle
00142         float bankAngle = transform.rotation.eulerAngles.x;
00143 
00144         if (bankAngle < 5f) return;
00145 
00146         // convert from unsigned 0-360 to range -180 and 180
00147         float signedBank = Mathf.DeltaAngle(0f, bankAngle);
00148         float g = 9.81f;
00149         // convert bank angle from degrees to radians
00150         float bankRadians = Mathf.Abs(signedBank) * Mathf.Deg2Rad;
00151         // Mathf.Tan expects radians not degrees
00152         float tanResult = Mathf.Tan(bankRadians);
00153         // Calculate radius
00154         float radius = (velocity * velocity) / (g * tanResult);
00155         // turn direction -1 = right 1 = left
00156         float turnDir = Mathf.Sign(signedBank);
00157         // The planes forward axis is +x therefore transform.forward returns the direction the planes
      +z axis is facing
00158         Vector3 sidewaysDir = transform.forward;
00159         // Isolate horizontal x and z directions
00160         sidewaysDir.y = 0;
00161         //Normalize horizontal dir
00162         sidewaysDir.Normalize();
00163         // change direction to left or right
00164         sidewaysDir *= turnDir;
00165         // calculate turn centre
00166         Vector3 turnCentre = transform.position + sidewaysDir * radius;
00167 
00168         Gizmos.color = Color.cyan;
00169         Gizmos.DrawWireSphere(turnCentre, radius);
00170     }
00171 }
```

8.45 Assets/Scripts/DA_40_SpecialFeatures/DA_40.cs File Reference

Classes

- class DA_40

  Handles mostly cosmetic elements of the DA-40 aircraft itself, including default appearance, sounds, component access and toggling, and propeller rotation.

8.46 DA_40.cs

Go to the documentation of this file.

```
00001 using System.Collections.Generic;
00002 using UnityEngine;
00003 using TMPro;
00004
00010
00011 public class DA_40 : MonoBehaviour
00012 {
00013     [Header("Plane Settings")]
00014     [SerializeField] private Color GlowColour;
00015     [SerializeField] private List<GameObject> InitialHighlightedComponents = new List<GameObject>();
00016     [SerializeField] private bool CockpitClosed;
00017     [SerializeField] private bool PorteClosed;
00018     [SerializeField] private bool PropellerEnabled;
00019     [SerializeField] private bool GhostTrailEnabled;
00020     [SerializeField] private ParticleSystem GhostTrail;
00021     [SerializeField] private bool AxialPanelsEnabled;
00022     [SerializeField] private float PropellerRotationSpeed = 1000f;
00023     [SerializeField] private bool IsPassthroughEnabled;
00024     [SerializeField] private List<GameObject> AxialScrollingPlanes = new List<GameObject>(); //**<
      DEPRECATED: A collection of planes (not the aircraft, but two triangles arranged in a panel in 3D
      space) that dynamically represent the aircraft's dynamic translation in the world. Replaced by the
      ghost trail system (see ParticleSystem GhostTrail above). */
00025     [SerializeField] private float AxialScrollSpeed;
00026     [SerializeField] private TextMeshProUGUI subtitles;
00027
00028     [Header("Sounds")]
00029     [SerializeField] private AudioSource FlightLoop;
00030     [SerializeField] private AudioSource PropellerStart;
00031     [SerializeField] private AudioSource StallWarning;
00032
00033     [Header("Dashboard")]
00034     [SerializeField] private GameObject Dashboard;
00035
00036     [Header("Cockpit")]
00037     [SerializeField] private GameObject Canopy;
00038     [SerializeField] private GameObject Porte;
00039     [SerializeField] private GameObject Vitres;
00040     [SerializeField] private GameObject Interior;
00041     [SerializeField] private GameObject Seats;
00042     [SerializeField] private GameObject Yoke_R;
00043     [SerializeField] private GameObject Yoke_L;
00044
00045     [Header("Fuselage")]
00046     [SerializeField] private GameObject FuselageFrame;
00047     [SerializeField] private GameObject Step_R;
00048     [SerializeField] private GameObject Step_L;
00049     [SerializeField] private GameObject Antennes;
00050     [SerializeField] private GameObject PropellerHub;
00051     [SerializeField] private GameObject Helice;
00052     [SerializeField] private GameObject Exhaust;
00053
00054     [Header("Empennage")]
00055     [SerializeField] private GameObject EmpennageFrame;
00056     [SerializeField] private GameObject Rudder;
00057     [SerializeField] private GameObject Stabiliser;
00058     [SerializeField] private GameObject Elevator;
00059     [SerializeField] private GameObject TailGuard;
00060     [SerializeField] private GameObject TailLight;
00061
00062     [Header("Right Wing")]
00063     [SerializeField] private GameObject WingFrame_R;
00064     [SerializeField] private GameObject Aileron_R;
00065     [SerializeField] private GameObject LandingFlap_R;
00066
00067     [Header("Left Wing")]
```

```
00068     [SerializeField] private GameObject WingFrame_L;
00069     [SerializeField] private GameObject Aileron_L;
00070     [SerializeField] private GameObject LandingFlap_L;
00071     [SerializeField] private GameObject LandingLightBulb;
00072
00073     [Header("Wing Components")]
00074     [SerializeField] private GameObject InnerGuards;
00075     [SerializeField] private GameObject Struts;
00076
00077     [Header("Landing Gear")]
00078     [SerializeField] private GameObject LandingStrutBrace_F;
00079     [SerializeField] private GameObject LandingStrut_F;
00080     [SerializeField] private GameObject OuterGuard_F;
00081     [SerializeField] private GameObject InnerGuard_F;
00082     [SerializeField] private GameObject Wheel_F;
00083     [SerializeField] private GameObject LandingStrut_R;
00084     [SerializeField] private GameObject OuterGuard_R;
00085     [SerializeField] private GameObject InnerGuard_R;
00086     [SerializeField] private GameObject Wheel_R;
00087     [SerializeField] private GameObject LandingStrut_L;
00088     [SerializeField] private GameObject OuterGuard_L;
00089     [SerializeField] private GameObject InnerGuard_L;
00090     [SerializeField] private GameObject Wheel_L;
00091
00092     private bool lastTrailState;
00093
00094     private float propellerAcceleration = 500f;
00095     private float currentPropellerSpeed = 0f;
00096
00097
00098     void Start()
00099     {
00100         // Initially set the selected components to glow at runtime
00101         PlaneHighlighter.Highlight(InitialHighlightedComponents, GlowColour);
00102         // UnhighlightPlaneComponent(new List<GameObject> { Canopy, Porte });
00103
00104         if (CockpitClosed)
00105         {
00106             Canopy.transform.localPosition = new Vector3(0.586243f, -1.451507f, -4.410744e-06f);
00107             Canopy.transform.localRotation = Quaternion.Euler(0f, 0f, 35.039f);
00108         }
00109
00110         if (PorteClosed)
00111         {
00112             Porte.transform.localPosition = new Vector3(-0.05631721f, 0.2968897f, -0.3203334f);
00113             Porte.transform.localRotation = Quaternion.Euler(55.165f, 3.138f, -4.324f);
00114         }
00115
00116         if (!AxialPanelsEnabled)
00117         {
00118             foreach (var axis in AxialScrollingPlanes)
00119             {
00120                 axis.SetActive(false);
00121             }
00122         }
00123
00124         // Initial setup of ghost trail
00125         if (GhostTrailEnabled)
00126             GhostTrail.Play();
00127         else
00128             GhostTrail.Stop(true, ParticleSystemStopBehavior.StopEmitting);
00129
00130         lastTrailState = GhostTrailEnabled;
00131
00132         // Clear subtitles
00133         subtitles.text = "";
00134     }
00135
00136
00137     void Update()
00138     {
00139         // Linear increase and decrease of propeller speed when toggled on and off respectively
00140         if (PropellerEnabled && Helice != null) // Rotate the propeller if it is toggled on
00141         {
00142             currentPropellerSpeed = Mathf.MoveTowards(currentPropellerSpeed, PropellerRotationSpeed,
      propellerAcceleration * Time.deltaTime);
00143
00144             Helice.transform.Rotate(Vector3.right, -currentPropellerSpeed * Time.deltaTime);
00145         }
00146         else
00147         {
00148             currentPropellerSpeed = Mathf.MoveTowards(currentPropellerSpeed, 0f, propellerAcceleration
      * Time.deltaTime);
00149             Helice.transform.Rotate(Vector3.right, -currentPropellerSpeed * Time.deltaTime);
00150         }
00151
00152         if (GhostTrailEnabled != lastTrailState) // Every frame, turn on the ghost trail if enabled by
```

```
    checking prev state
00153         {
00154             if (GhostTrailEnabled)
00155                 GhostTrail.Play();
00156             else
00157                 GhostTrail.Stop(true, ParticleSystemStopBehavior.StopEmitting);
00158 
00159             lastTrailState = GhostTrailEnabled;
00160         }
00161 
00162         // This is all deprecated!!! Doesn't need to be here anymore, but it is disabled in practice
    already.
00163         for (int i = 0; i < AxialScrollingPlanes.Count; i++) // For every axial plane (should only be
    3)
00164         {
00165             var axis = AxialScrollingPlanes[i];
00166             if (axis == null) continue;
00167 
00168             var renderers = axis.GetComponentsInChildren<Renderer>(true);
00169             Vector2 offset = Vector2.zero; // Offset will be what the texture uses to scroll
00170 
00171             switch (i) // Each plane will read different position values to determine their axis'
    offset
00172             {
00173                 case 0: // X plane offset - "back", relies on vehicle's Y and Z positions
00174                     offset = new Vector2(transform.position.y, transform.position.z) *
    AxialScrollSpeed;
00175                     //Debug.Log("Changing stuff!");
00176                     break;
00177                 case 1: // Y plane offset - "left", relies on vehicle's X and Z positions
00178                     offset = new Vector2(transform.position.x, transform.position.z) *
    AxialScrollSpeed;
00179                     //Debug.Log("Changing stuff!");
00180                     break;
00181                 case 2: // Z plane offset - "up", relies on vehicle's X and Y positions
00182                     offset = new Vector2(transform.position.y, transform.position.x) *
    AxialScrollSpeed;
00183                     //Debug.Log("Changing stuff!");
00184                     break;
00185             }
00186 
00187             foreach (var renderer in renderers) // Apply the offset
00188             {
00189                 renderer.material.SetTextureOffset("_BaseMap", offset);
00190             }
00191         }
00192     }
00193 
00199     public void playSFX(AudioSource sound)
00200     {
00201         sound.Play();
00202     }
00203 
00209     public void stopSFX(AudioSource sound)
00210     {
00211         sound.Stop();
00212     }
00213 
00219     public void TogglePropeller(bool newState)
00220     {
00221         PropellerEnabled = newState;
00222     }
00223 }
```

8.47 Assets/Scripts/DA_40_SpecialFeatures/PlaneComponent↩ Highlighter.cs File Reference

Classes

- class PlaneHighlighter

  Handles highlighting of specific components of the aircraft.

8.48 PlaneComponentHighlighter.cs

Go to the documentation of this file.

```
00001 using System.Collections.Generic;
00002 using UnityEngine;
00003
00009
00010 public static class PlaneHighlighter
00011 {
00018     public static void Highlight(IEnumerable<GameObject> components, Color glowColour)
00019     {
00020         foreach (var obj in components) // Loop through each plane component passed in (parent or
      standalone)
00021         {
00022             Debug.Log($"Highlighting: {obj.name}");
00023             if (obj == null) continue; // Skip plane components that are null (shouldn't happen, but a
      good failsafe)
00024
00025             var renderers = obj.GetComponentsInChildren<Renderer>(true); // Get the render component
      of each child object AND the parent object
00026             foreach (var renderer in renderers) // Loop through each renderer
00027             {
00028                 var mat = renderer.material; // Create a new material instance for JUST the selected
      component so that the original material and other components that may share that material aren't
      affected
00029
00030                 if (mat.HasProperty("_Surface")) // Checking if the material is transparent or opaque,
      if so we want to skip it
00031                 {
00032                     int surface = (int)mat.GetFloat("_Surface");
00033                     if (surface == 1) continue; // Skip transparent material
00034                 }
00035
00036                 mat.EnableKeyword("_EMISSION"); // Toggle emission for the material if not already
      enabled
00037                 mat.SetColor("_EmissionColor", glowColour * 100f); // Set the emission colour to the
      passed in colour, and intensify it
00038             }
00039         }
00040     }
00041
00047     public static void Unhighlight(IEnumerable<GameObject> components)
00048     {
00049         foreach (var obj in components) // Loop through each plane component passed in (parent or
      standalone)
00050         {
00051             Debug.Log($"Un-highlighting: {obj.name}");
00052             if (obj == null) continue; // Skip plane components that are null (shouldn't happen, but a
      good failsafe)
00053
00054             var renderers = obj.GetComponentsInChildren<Renderer>(true); // Get the render component
      of each child object AND the parent object
00055             foreach (var renderer in renderers) // Loop through each renderer and turn off their
      emission and reset their properties
00056             {
00057                 var mat = renderer.material;
00058                 mat.DisableKeyword("_EMISSION");
00059                 mat.SetColor("_EmissionColor", Color.black);
00060             }
00061         }
00062     }
00063 }
```

8.49 Assets/Scripts/DA_40_SpecialFeatures/ProceduralAirflow.cs File Reference

Classes

- class ProceduralAirflow

  Airflow, Force Arrows and COP Behaviour.

8.50 ProceduralAirflow.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
```

```
00002 using System.Collections.Generic;
00003
00017 [ExecuteAlways]
00018 public class ProceduralAirflow : MonoBehaviour
00019 {
00020
00021     // Enable/disable bool
00022     [Header("")]
00023     public bool enableParticleSystem = false;
00024     public bool enableForceArrows = false;
00025     public bool enableCentreOfPressure = false;
00026
00027     // Wing references
00028     [Header("Wing References")]
00029     public Transform leadingEdge;
00030     public Transform trailingEdge;
00031
00032     // Arrow anchor point references
00033     [Header("Force Arrow Anchor Points")]
00034     public Transform liftAnchor;
00035     public Transform weightAnchor;
00036     public Transform thrusAnchor;
00037     public Transform dragAnchor;
00038
00039     // Arrow Settings
00040     [Header("Arrow Settings")]
00041     public float defaultArrowSize = 1f;
00042     public Material forceArrowMaterial;
00043     public GameObject arrowHeadModel;
00044
00045     [Header("COP Variables")]
00046     public GameObject copModel;
00047
00048     // Particle system settings
00049     [Header("Particle System Settings")]
00050     public ParticleSystem ps;
00051     public float emissionRate = 20f;
00052     public float forwardSpawnDistance = 0.5f;
00053     public float particleSize = 0.05f;
00054     public int maxParticles = 1000;
00055
00056     // Flow speed bounds
00057     [Header("Flow Speed Bounds")]
00058     public float topMaxSpeed = 2f;
00059     public float topMinSpeed = 1.5f;
00060     public float bottomMaxSpeed = 1f;
00061     public float bottomMinSpeed = 0.5f;
00062
00063     // Simulation settings
00064     [Header("Simulation Settings")]
00065     public float airSeperationStrength = 0.5f;
00066     public float trailDownForcePercent = 0.2f;
00067     public bool useFixedDeltaInEditor = true;
00068     public float editorDelta = 1f / 60f;
00069
00070     // Internal data
00071     private ParticleSystem.Particle[] buffer;
00072     private List<Vector3> velocities = new List<Vector3>();
00073     private List<bool> sideFlags = new List<bool>();
00074     float emissionsNextFrame = 0f;
00075     float rotation = 0f;
00076     // Arrows
00077     private GameObject liftArrow;
00078     private GameObject weightArrow;
00079     private GameObject thrustArrow;
00080     private GameObject dragArrow;
00081     // AOA stages
00082     private static float[] aoaStages = {3f, 7f, 10f, 14f, 16f, 18f};
00083     // multipliers for default arrow size
00084     // used to lerp arro sizes based on AOA
00085     private static float[] liftValues = { 1.0f, 1.2f, 1.5f, 1.7f, 1.6f, 0.8f };
00086     private static float[] dragValues = { 1.0f, 1.2f, 1.5f, 1.7f, 2.0f, 2.5f };
00087     private static float[] thrustValues = { 1.0f, 1.2f, 1.5f, 1.6f, 1.0f, 0.6f };
00088
00089     // cop percentages from leading edge
00090     private static float[] copValues = { 0.25f, 0.24f, 0.22f, 0.20f, 0.20f, 0.45f };
00091
00092     // COP indicator
00093     private GameObject copGameObject;
00094     // Initialization
00095
00101     void Awake()
00102     {
00103         InitializeParticleSystem();
00104         buffer = new ParticleSystem.Particle[maxParticles];
00105     }
00106
```

```
00114     void InitializeParticleSystem()
00115     {
00116         if (ps == null)
00117         {
00118             ps = gameObject.AddComponent<ParticleSystem>();
00119 
00120             var main = ps.main;
00121             main.maxParticles = maxParticles;
00122             // Particle lifetime will be manually controlled and a extended lifetime is to allow
      removal in edgecases
00123             main.startLifetime = 15f;
00124             main.startSize = particleSize;
00125             main.startSpeed = 0f; // we control manually
00126             main.simulationSpace = ParticleSystemSimulationSpace.World;
00127             main.playOnAwake = true;
00128 
00129             var emission = ps.emission;
00130             emission.enabled = false; // manual emission
00131 
00132             var shape = ps.shape;
00133             shape.enabled = false; // spawn at exact positions
00134 
00135             //TODO add material creation
00136 
00137             Debug.Log("ParticleSystem auto created.");
00138         }
00139 
00140         // clear previous lists
00141         velocities.Clear();
00142         sideFlags.Clear();
00143     }
00144 
00150     void Update()
00151     {
00152         if (leadingEdge == null || trailingEdge == null || ps == null) return;
00153 
00154         if (buffer == null || buffer.Length != maxParticles)
00155             buffer = new ParticleSystem.Particle[maxParticles];
00156 
00157         float dt = Application.isPlaying ? Time.deltaTime : (useFixedDeltaInEditor ? editorDelta :
      Time.deltaTime);
00158 
00159         if (enableParticleSystem)
00160         {
00161             // Emit new particles
00162             EmitParticles(dt);
00163 
00164             // Update existing particles
00165             UpdateParticles(dt);
00166         }
00167 
00168         //Debug.Log("Are force Arrows Enabled? " + enableForceArrows);
00169         if (enableForceArrows)
00170         {
00171             // Fix inconsistent toggle behaviour
00172             // Add a check for if arrows are enabled but not instantiated
00173             bool missingArrow = liftArrow == null || weightArrow == null || thrustArrow == null ||
      dragArrow == null;
00174             // Update force arrows
00175             float newRotation = transform.rotation.eulerAngles.z;
00176             if (missingArrow || rotation != newRotation)
00177             {
00178                 //Debug.Log("Is this working");
00179                 rotation = newRotation;
00180                 UpdateForceArrows();
00181             }
00182         }
00183         else
00184         {
00185             DestroyPreviousArrows();
00186         }
00187         //Debug.Log($"lift arrow: {liftArrow == null} weight arrow: {weightArrow == null} thrust
      arrow: {thrustArrow == null} drag arrow: {dragArrow == null}");
00188 
00189         if (enableCentreOfPressure)
00190         {
00191             updateCentreOfPressure();
00192         }
00193         else
00194         {
00195             destroyCentreOfPressure();
00196         }
00197 
00198         // Ensure particle system is playing
00199         if (!ps.isPlaying) ps.Play();
00200 
00201         // Editor simulation
```

```
00202         if (!Application.isPlaying)
00203             ps.Simulate(dt, true, false, true);
00204     }
00205 
00213     void EmitParticles(float dt)
00214     {
00215         // calculate particles number of particles to spawn
00216         emissionsNextFrame += emissionRate * dt;
00217 
00218         // get integer value of particle spawn requirements and decrement counter
00219         int particlesToEmit = Mathf.FloorToInt(emissionsNextFrame);
00220         emissionsNextFrame -= particlesToEmit;
00221 
00222         // Return if no particles to spawn
00223         if (particlesToEmit <= 0) return;
00224 
00225         // get normalised vector of the chord
00226         Vector3 chordDir = (trailingEdge.position - leadingEdge.position).normalized;
00227 
00228         for (int i = 0; i < particlesToEmit; i++)
00229         {
00230             // Set the spawn position of the particle
00231             var emitParams = new ParticleSystem.EmitParams();
00232             Vector3 spawnPos = leadingEdge.position - chordDir * forwardSpawnDistance;
00233             emitParams.position = spawnPos;
00234 
00235             emitParams.startLifetime = 5f;
00236             emitParams.startSize = particleSize;
00237 
00238             Vector3 toLeadingEdge = (leadingEdge.position - spawnPos).normalized;
00239             emitParams.velocity = toLeadingEdge * Random.Range(topMinSpeed, topMaxSpeed);
00240 
00241             // randomly assign particle as above or below the chord
00242             // set random velocity between top and min speeds for update loop conditions
00243             bool above = Random.value > 0.2f;
00244             Vector3 velocity;
00245             if (above)
00246             {
00247                 Vector3 airflowDir = chordDir;
00248                 airflowDir.y *= airSeperationStrength;
00249                 airflowDir = airflowDir.normalized;
00250                 velocity = airflowDir * Random.Range(topMinSpeed, topMaxSpeed);
00251                 sideFlags.Add(true);
00252             }
00253             else
00254             {
00255                 velocity = chordDir * Random.Range(bottomMinSpeed, bottomMaxSpeed);
00256                 sideFlags.Add(false);
00257             }
00258 
00259             // Store the per particle velocities
00260             velocities.Add(velocity);
00261 
00262             // add particles to local buffer and particle system
00263             ps.Emit(emitParams, 1);
00264         }
00265 
00266     }
00267 
00278     void UpdateParticles(float dt)
00279     {
00280         int count = ps.GetParticles(buffer);
00281 
00282         // the local position of leading and trailing edges
00283         Vector3 localLeading = transform.InverseTransformPoint(leadingEdge.position);
00284         Vector3 localTrailing = transform.InverseTransformPoint(trailingEdge.position);
00285 
00286         // calculate chord direction in world space
00287         Vector3 chordDir = (trailingEdge.position - leadingEdge.position).normalized;
00288         // calculate the chord length in local space
00289         float chordLengthLocal = Vector3.Distance(localTrailing, localLeading);
00290 
00291         // determine the chord orientation in local space. sign determines if teh trailing edge is at
    +x or -x in local space
00292         float sign = Mathf.Sign(localTrailing.x - localLeading.x);
00293 
00294         // calculate poiints for particle direction changes
00295         float deflectionStart = chordLengthLocal * .10f;     // 20% of the chord length from leading
    edge
00296         float trailDistance = chordLengthLocal * .50f;       // 50% of the chord length past trailing
    edge
00297 
00298         // Loop over all active particles
00299         for (int i = 0; i < count; i++)
00300         {
00301             // get particle world position and convert to local position
00302             Vector3 particleWorld = buffer[i].position;
```

```
00303             Vector3 particleLocal = transform.InverseTransformPoint(particleWorld);
00304
00305             // Determines a particles position along the chord / displacement from leading edge
00306             // sign ensures distances along the chord are positive
00307             float alongChord = (particleLocal.x - localLeading.x) * sign;
00308
00309             // check is particles are over trail distance
00310             if (alongChord > chordLengthLocal + trailDistance)
00311             {
00312                 // Remove particle by swapping with last and reducing count
00313                 int lastIdx = count - 1;
00314                 if (i != lastIdx)
00315                 {
00316                     buffer[i] = buffer[lastIdx];
00317                     velocities[i] = velocities[lastIdx];
00318                     sideFlags[i] = sideFlags[lastIdx];
00319                 }
00320                 count--;
00321                 velocities.RemoveAt(lastIdx);
00322                 sideFlags.RemoveAt(lastIdx);
00323                 continue;
00324             }
00325
00326             // if particle is at or past deflection point deflect based on side flags
00327             if (alongChord >= -deflectionStart && alongChord <= 0f)
00328             {
00329                 // Deflect relative to planes local up
00330                 Vector3 offsetTargetLocal;
00331                 if (sideFlags[i])
00332                 {
00333                     // Particle is above the wing, deflect upwards
00334                     offsetTargetLocal = new Vector3(
00335                         localLeading.x + deflectionStart * sign,
00336                         localLeading.y + deflectionStart,
00337                         particleLocal.z
00338                     );
00339                 }
00340                 else
00341                 {
00342                     // Particle is below the wing deflect downwards
00343                     offsetTargetLocal = new Vector3(
00344                         localLeading.x + deflectionStart * sign,
00345                         localLeading.y - deflectionStart,
00346                         particleLocal.z
00347                     );
00348                 }
00349
00350                 // convert offset target to world coorinates
00351                 Vector3 offsetTargetWorld = transform.TransformPoint(offsetTargetLocal);
00352                 // get direction in world coordinates to target
00353                 Vector3 targetDir = (offsetTargetWorld - particleWorld).normalized;
00354
00355                 float speed = velocities[i].magnitude;
00356                 buffer[i].velocity = targetDir * speed;
00357             }
00358
00359             // if particle is past the deflection target (leading edge) set velocity to stored
    velocities value
00360             else if (alongChord > deflectionStart && alongChord <= chordLengthLocal)
00361             {
00362                 buffer[i].velocity = velocities[i];
00363             }
00364             // If particle is past trailing edge remove vertical velocity component
00365             else if (alongChord > chordLengthLocal && alongChord <= chordLengthLocal + trailDistance)
00366             {
00367                 float horizontalSpeed = velocities[i].magnitude;
00368                 // Remove y component from chordDir and normalize
00369                 Vector3 chordDirNoY = new Vector3(chordDir.x, chordDir.y * trailDownForcePercent,
    chordDir.z).normalized;
00370                 buffer[i].velocity = chordDirNoY * horizontalSpeed;
00371             }
00372             else
00373             {
00374                 int n = 1;
00375             }
00376
00377             // Apply velocity update to position
00378             buffer[i].position += buffer[i].velocity * dt;
00379         }
00380
00381         // Hand updated particles back to the system for rendering
00382         ps.SetParticles(buffer, count);
00383     }
00384
00385
00391     void UpdateForceArrows()
00392     {
```

```
00393         /*
00394         // TODO fix arrow offset by using bounding box calculation ot centre arrows.
00395         Renderer render = GetComponentInChildren<MeshRenderer>();
00396         Debug.LogError(render + "This is a debug message for force arrows");
00397         */
00398         if (liftArrow == null || weightArrow == null || thrustArrow == null || dragArrow == null)
00399         {
00400             CreateForceArrows();
00401         }
00402 
00403         // get the angle of the plane
00404         float aoa = transform.eulerAngles.z;
00405 
00406         SetArrow(liftArrow, liftAnchor.position, Vector3.up, GetLiftLength(aoa));
00407         SetArrow(weightArrow, weightAnchor.position, Vector3.down, GetWeightLength(aoa));
00408         SetArrow(thrustArrow, thrusAnchor.position, Vector3.right, GetThrustLength(aoa));
00409         SetArrow(dragArrow, dragAnchor.position, Vector3.left, GetDragLength(aoa));
00410 
00411     }
00412 
00423     float GetLiftLength(float aoa)
00424     {
00425         return defaultArrowSize * InterpolateForceValue(aoa, liftValues);
00426     }
00427 
00437     float GetWeightLength(float aoa)
00438     {
00439         return defaultArrowSize;
00440     }
00441 
00452     float GetThrustLength(float aoa)
00453     {
00454         return defaultArrowSize * InterpolateForceValue(aoa, thrustValues);
00455     }
00456 
00467     float GetDragLength(float aoa)
00468     {
00469         return defaultArrowSize * InterpolateForceValue(aoa, dragValues);
00470     }
00471 
00487     private float InterpolateForceValue(float aoa, float[] forceValues)
00488     {
00489         // Handle values below minimum stage
00490         if (aoa <= aoaStages[0]) return forceValues[0];
00491 
00492         // Handle values above maximum stage
00493         if (aoa >= aoaStages[aoaStages.Length - 1])
00494             return forceValues[forceValues.Length - 1];
00495 
00496         // Find the appropriate stage interval and interpolate
00497         for (int i = 0; i < aoaStages.Length - 1; i++)
00498         {
00499             if (aoa <= aoaStages[i + 1])
00500             {
00501                 // Calculate interpolation factor 0 = start, 1 = end, 0.5 = halfway
00502                 float t = (aoa - aoaStages[i]) / (aoaStages[i + 1] - aoaStages[i]);
00503 
00504                 // Linearly interpolate between lower and upper force values
00505                 return Mathf.Lerp(forceValues[i], forceValues[i + 1], t);
00506             }
00507         }
00508         // Fallback to maximum value
00509         return forceValues[forceValues.Length - 1];
00510     }
00511 
00524     void SetArrow(GameObject arrow, Vector3 basePos, Vector3 dir, float length)
00525     {
00526         float width = 0.1f;   // arrow width
00527         float height = 0.1f;  // arrow height
00528         float headSize = .2f; // Arrow head size
00529 
00530         //Debug.Log(arrow.name + " " + length);
00531 
00532         // Scale the arrow
00533         arrow.transform.localScale = new Vector3(width, height, length);
00534 
00535         // Make the arrow point along the force direction
00536         Quaternion lookRotation = Quaternion.LookRotation(dir, Vector3.up);
00537         // Create a quaternion consisting only of the planes y rotation
00538         Quaternion yRotation = Quaternion.Euler(0f, transform.eulerAngles.y, 0f);
00539         // apply the planes y rotation then rotate the arrow to point along force direction
00540         arrow.transform.rotation = yRotation * lookRotation;
00541 
00542         // Set the arrows position so the base is along the anchor point.
00543         arrow.transform.position = basePos + arrow.transform.forward * (length * 0.5f);
00544 
00545         // Get the arrow head
```

```
00546         Transform headTransform = arrow.transform.GetChild(0);
00547         // Set the head position to the shafts forward face
00548         headTransform.localPosition = new Vector3(0, 0, 0.5f);
00549         // compensate for the shafts scale.
00550         // localScale * parentScale = worldScale therefore localScale = worldScale / parentScale
00551         headTransform.localScale = new Vector3(headSize / width, headSize / height, headSize /
      length);
00552     }
00553
00560     void CreateForceArrows()
00561     {
00562         DestroyPreviousArrows(); // remove old arrows
00563
00564         liftArrow = CreateArrow("Lift", Color.green);
00565         weightArrow = CreateArrow("Weight", Color.red);
00566         thrustArrow = CreateArrow("Thrust", Color.blue);
00567         dragArrow = CreateArrow("Drag", Color.yellow);
00568     }
00569
00580     GameObject CreateArrow(string name, Color colour)
00581     {
00582         // Create a new box / "Arrow"
00583         GameObject box = GameObject.CreatePrimitive(PrimitiveType.Cube);
00584         // set the name
00585         box.name = name + "_Arrow";
00586
00587         Renderer rend = box.GetComponent<Renderer>();
00588         Material mat = new Material(forceArrowMaterial);
00589         // Handle material leaks during edit mode
00590         if (Application.isPlaying)
00591         {
00592             mat.color = colour;
00593             rend.material = mat;
00594         }
00595         else
00596             rend.sharedMaterial.color = colour;
00597
00598         // Create empt game object for the head with specified name
00599         GameObject arrowHead = Instantiate(arrowHeadModel);
00600         // Parent to arrow shaft
00601         arrowHead.transform.SetParent(box.transform);
00602         // Get mesh renderer
00603         MeshRenderer rendHead = arrowHead.GetComponent<MeshRenderer>();
00604         // Assign material to arrow head
00605         rendHead.material = mat;
00606
00607         // Position head to front face of parent arrow shaft
00608         arrowHead.transform.localPosition = new Vector3(0, 0, 0.5f);
00609         arrowHead.transform.localRotation = Quaternion.Euler(0, 0, 0);
00610
00611         // Return new arrow game object
00612         return box;
00613     }
00614
00621     void DestroyPreviousArrows()
00622     {
00623         // Names of generated Arrows
00624         string[] arrowNames = { "Lift_Arrow", "Weight_Arrow", "Thrust_Arrow", "Drag_Arrow" };
00625         foreach (string name in arrowNames)
00626         {
00627             GameObject existing = GameObject.Find(name);
00628             if (existing != null)
00629             {
00630                 // Schedule dduplicates for destruction in play mode
00631                 if (Application.isPlaying)
00632                     Destroy(existing);
00633                 // Else in edit mode destroy immediately
00634                 else
00635                     DestroyImmediate(existing);
00636             }
00637         }
00638     }
00639
00647     void updateCentreOfPressure()
00648     {
00649         if (copGameObject == null)
00650         {
00651             copGameObject = createCopIndicator("COP_Indicator", Color.grey);
00652         }
00653
00654         // get the angle of the plane
00655         float aoa = transform.eulerAngles.z;
00656
00657         // Get the percentage along the chord
00658         float percentage = getCopPercentage(aoa);
00659         //Debug.Log("COP percentage: " + percentage);
00660
```

```
00661         // Calculate chord direction and length
00662         Vector3 chordDir = (trailingEdge.position - leadingEdge.position).normalized;
00663         float chordLength = Vector3.Distance(leadingEdge.position, trailingEdge.position);
00664 
00665         // Calculate position along the chord
00666         Vector3 copPos = leadingEdge.position + chordDir * (chordLength * percentage);
00667 
00668         // Offset below the chord
00669         Vector3 offset = -transform.up * 0.3f;
00670         copGameObject.transform.position = copPos + offset;
00671     }
00672 
00683     GameObject createCopIndicator(string name, Color colour)
00684     {
00685         destroyCentreOfPressure();
00686 
00687         GameObject cop = Instantiate(copModel);
00688 
00689         // set the name
00690         cop.name = name;
00691 
00692         //cop.transform.localScale = new Vector3(0.1f, 0.2f,0.1f);
00701         return cop;
00702 
00703     }
00704 
00715     float getCopPercentage(float aoa)
00716     {
00717         // same logic as arrow calculations
00718         if (aoa <= aoaStages[0]) return copValues[0];
00719         if (aoa <= aoaStages[1]) return Mathf.Lerp(copValues[0], copValues[1], (aoa - aoaStages[0]) /
      (aoaStages[1] - aoaStages[0]));
00720         if (aoa <= aoaStages[2]) return Mathf.Lerp(copValues[1], copValues[2], (aoa - aoaStages[1]) /
      (aoaStages[2] - aoaStages[1]));
00721         if (aoa <= aoaStages[3]) return Mathf.Lerp(copValues[2], copValues[3], (aoa - aoaStages[2]) /
      (aoaStages[3] - aoaStages[2]));
00722         if (aoa <= aoaStages[4]) return Mathf.Lerp(copValues[3], copValues[4], (aoa - aoaStages[3]) /
      (aoaStages[4] - aoaStages[3]));
00723         if (aoa <= aoaStages[5]) return Mathf.Lerp(copValues[4], copValues[5], (aoa - aoaStages[4]) /
      (aoaStages[5] - aoaStages[4]));
00724         return copValues[5];
00725     }
00726 
00733     void destroyCentreOfPressure()
00734     {
00735         GameObject existing = GameObject.Find("COP_Indicator");
00736         // Schedule duplicates for destruction in play mode
00737         if (Application.isPlaying)
00738             Destroy(existing);
00739         // Else in edit mode destroy immediately (Since destroy is tied to the end of the next frame
      which doesnt trigger in edit mode)
00740         else
00741             DestroyImmediate(existing);
00742     }
00743 
00750     void OnDrawGizmos()
00751     {
00752         if (leadingEdge != null && trailingEdge != null)
00753         {
00754             Gizmos.color = Color.yellow;
00755             Gizmos.DrawLine(leadingEdge.position, trailingEdge.position);
00756 
00757             // draw small offset lines for top/bottom airflow guidance
00758             Vector3 chordDir = (trailingEdge.position - leadingEdge.position).normalized;
00759             float chordLen = Vector3.Distance(leadingEdge.position, trailingEdge.position);
00760             Vector3 midpoint = leadingEdge.position + chordDir * chordLen * 0.5f;
00761 
00762             Gizmos.color = Color.cyan;
00763             Gizmos.DrawLine(midpoint, midpoint + Vector3.up * 0.2f);
00764 
00765             Gizmos.color = Color.magenta;
00766             Gizmos.DrawLine(midpoint, midpoint + Vector3.down * 0.2f);
00767         }
00768     }
00769 }
```

8.51 Assets/Scripts/Misc/BankGhostTrailDemo.cs File Reference

Classes

- class BankGhostTrailDemo

  A test comment.

8.52 BankGhostTrailDemo.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00007 public class BankGhostTrailDemo : MonoBehaviour
00008 {
00010     public AxisRotationController manip;
00011
00012     float delay = 0f;
00013     float delayTarget = 5f;
00014     bool started = false;
00015     void Start()
00016     {
00017
00018     }
00019
00029     void Update()
00030     {
00031         if (delay >= delayTarget)
00032         {
00033             if (started)
00034             {
00035                 return;
00036             }
00037             else
00038             {
00039                 started = true;
00040                 manip.LerpBank(69, 6);
00041             }
00042         }
00043         else
00044         {
00045             delay += Time.deltaTime;
00046             Debug.Log("Delay Target: " + delayTarget);
00047         }
00048     }
00049 }
```

8.53 Assets/Scripts/MiscBehaviour/AxisRotationController.cs File Reference

Classes

- class AxisRotationController

  Helper class to perform rotation actions.

8.54 AxisRotationController.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002 using System.Collections;
00003
00012 [ExecuteAlways]
00013 public class AxisRotationController : MonoBehaviour
00014 {
00015     private bool isLerpingPitch;
00016     private bool isLerpingRoll;
00017
00018     private float startRotationZ;
00019     private float targetRotationZ;
00020
00021     private float startRotationX;
00022     private float targetRotationX;
00023
00024     private float durationRoll;
00025     private float durationPitch;
00026     private float timeElapsedRoll;
00027     private float timeElapsedPitch;
00028
```

```
00037     public void LerpAOA(float angle, float lerpDuration = 1.5f)
00038     {
00039         startRotationZ = transform.rotation.eulerAngles.z;
00040         targetRotationZ = angle;
00041         durationPitch = lerpDuration;
00042         timeElapsedPitch = 0f;
00043         isLerpingPitch = true;
00044         //Debug.Log($"LerpAOA started: from {startRotationZ} to {targetRotationZ}, duration
      {lerpDuration}");
00045     }
00046
00055     public void LerpBank(float angle, float lerpDuration = 1.5f)
00056     {
00057         startRotationX = transform.rotation.eulerAngles.x;
00058         targetRotationX = angle;
00059         durationRoll = lerpDuration;
00060         timeElapsedRoll = 0f;
00061         isLerpingRoll = true;
00062     }
00063
00072     public void IncrementBank(float delta)
00073     {
00074         transform.rotation = Quaternion.Euler(transform.eulerAngles.x + delta,
      transform.eulerAngles.y, transform.eulerAngles.z);
00075     }
00076
00085     public void IncrementAOA(float delta)
00086     {
00087         transform.rotation = Quaternion.Euler(transform.eulerAngles.x, transform.eulerAngles.y,
      transform.eulerAngles.z + delta);
00088     }
00089
00098     public void IncrementYaw(float delta)
00099     {
00100         transform.rotation = Quaternion.Euler(transform.eulerAngles.x, transform.eulerAngles.y +
      delta, transform.eulerAngles.z);
00101     }
00102
00108     private void Update()
00109     {
00110         if (isLerpingPitch)
00111         {
00112             UpdatePitch();
00113         }
00114
00115         if (isLerpingRoll)
00116         {
00117             UpdateRoll();
00118         }
00119     }
00120
00126     public Quaternion getRotation()
00127     {
00128         return transform.rotation;
00129     }
00130
00136     public void setRoll(float roll)
00137     {
00138         transform.rotation = Quaternion.Euler(roll, transform.eulerAngles.y, transform.eulerAngles.z);
00139     }
00140
00146     public void setPitch(float pitch)
00147     {
00148         transform.rotation = Quaternion.Euler(transform.eulerAngles.x, transform.eulerAngles.y,
      pitch);
00149     }
00150
00156     public bool IsLerpingRoll()
00157     {
00158         return isLerpingRoll;
00159     }
00160
00166     public bool IsLerpingPitch()
00167     {
00168         return isLerpingPitch;
00169     }
00170
00176     private void UpdateRoll()
00177     {
00178         // increment timeElapsed
00179         timeElapsedRoll += Application.isPlaying ? Time.deltaTime : 1f / 60f; // fake deltaTime in
      editor
00180         // Calculate percentage completion
00181         float t = Mathf.Clamp01(timeElapsedRoll / durationRoll);
00182
00183         // Lerp between start and target rotations
00184         Vector3 euler = transform.rotation.eulerAngles;
```

```
00185         euler.x = Mathf.LerpAngle(startRotationX, targetRotationX, t);
00186
00187         transform.rotation = Quaternion.Euler(euler);
00188
00189         if (t >= 1f) isLerpingRoll = false;
00190     }
00191
00197     private void UpdatePitch()
00198     {
00199         // increment timeElapsed
00200         timeElapsedPitch += Application.isPlaying ? Time.deltaTime : 1f / 60f; // fake deltaTime in
      editor
00201         // Calculate percentage completion
00202         float t = Mathf.Clamp01(timeElapsedPitch / durationPitch);
00203
00204         // Lerp between start and target rotations
00205         Vector3 euler = transform.rotation.eulerAngles;
00206         euler.z = Mathf.LerpAngle(startRotationZ, targetRotationZ, t);
00207
00208         transform.rotation = Quaternion.Euler(euler);
00209
00210         if (t >= 1f) isLerpingPitch = false;
00211     }
00212 }
```

8.55 Assets/Scripts/MiscBehaviour/GhostTrailToggle.cs File Reference

Classes

- class GhostTrailToggle

  Handles the toggling of the ghost trail belonging to the DA-40 aircraft on and off.

8.56 GhostTrailToggle.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002 using UnityEngine.InputSystem;
00003
00009
00010 public class GhostTrailToggle : MonoBehaviour
00011 {
00012     public ParticleSystem GhostTrail;
00013
00014     private bool ghostTrailEnabled = false;
00015
00016     public InputActionReference toggleAction;
00017
00022     void OnEnable()
00023     {
00024         toggleAction.action.performed += OnTogglePressed;
00025         toggleAction.action.Enable();
00026     }
00027
00032     void OnDisable()
00033     {
00034         toggleAction.action.performed -= OnTogglePressed;
00035         toggleAction.action.Disable();
00036     }
00037
00043     private void OnTogglePressed(InputAction.CallbackContext ctx)
00044     {
00045         ghostTrailEnabled = !ghostTrailEnabled;
00046
00047         if (ghostTrailEnabled) // Start trail
00048         {
00049             GhostTrail.Play();
00050         }
00051         else // Stop trail
00052         {
00053             GhostTrail.Stop(true, ParticleSystemStopBehavior.StopEmitting);
00054         }
00055     }
00056 }
```

8.57 Assets/Scripts/MiscBehaviour/MovePlaneToPlatform.cs File Reference

Classes

- class PlaneTeleporter

  Helper to handle translating the DA-40 aircraft to and from the viewing platform in a Timeline.

8.58 MovePlaneToPlatform.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002
00008
00009 public class PlaneTeleporter : MonoBehaviour
00010 {
00011     public Vector3 targetPosition;
00012
00017     public void TeleportToTarget()
00018     {
00019         transform.position = targetPosition;
00020
00021         // HARD-CODED: Subject to change in future. Keep for now.
00022         Vector3 localEuler = transform.rotation.eulerAngles;
00023         localEuler.y = 180f;
00024         transform.rotation = Quaternion.Euler(localEuler);
00025     }
00026 }
```

8.59 Assets/Scripts/MiscBehaviour/SkyboxRotation.cs File Reference

Classes

- class SkyboxRotation

  Ambient class that slowly and gradualy rotates the skybox in the world to give the impression of moving clouds.

8.60 SkyboxRotation.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002
00008
00009 public class SkyboxRotation : MonoBehaviour
00010 {
00011     public float rotationSpeed = 0.1f;
00012
00013     void Update()
00014     {
00015         RenderSettings.skybox.SetFloat("_Rotation", Time.time * rotationSpeed); // Rotate clouds
00016     }
00017 }
```

8.61 Assets/Scripts/ModuleStructure/ModuleActivities.cs File Reference

Classes

- class ModuleActivities

8.62 ModuleActivities.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002 using UnityEngine.Playables;
00003
00004 [CreateAssetMenu(fileName = "NewModuleActivity", menuName = "Module Activity")] // Add a menu control
      to make an activity!!! This is so cool i love this
00005
00006 public class ModuleActivities : ScriptableObject
00007 {
00008     public string activityName;
00009     public string[] instructions; // The list of instructions, each corresponding to one of six
      checkboxes
00010
00011     public bool usesCustomScript = false; // If an activity is more involved, we can opt to use a
      custom script to handle it instead. This will act as an Adapter of sorts to whatever goes on in the
      activity
00012     public GameObject customScript; // The actual GameObject that contains the custom script, if
      needed (this will be instanced from a prefab!!)
00013
00014     public bool playerInputRequired = true; // Acts as a check that when toggled, completes the
      activity
00015
00016     public int totalSteps = 1; // Total number of input steps for instructions, max of six, default to
      one
00017     public int currentStep = 0;
00018
00019     // If activity will be handled through a custom script, instantiate the prefab that contains said
      script
00020     public GameObject PrefabSetup(Transform parent)
00021     {
00022         if (usesCustomScript && customScript != null)
00023         {
00024             // Instantiate the new script from a prefab
00025             GameObject instance = GameObject.Instantiate(customScript, parent);
00026             instance.name = $"{activityName}_ControllerInstance";
00027             return instance;
00028         }
00029
00030         // No custom script required, skip prefab setup
00031         return null;
00032     }
00033
00034     // Self-explanatory.
00035     public bool AdvanceStep()
00036     {
00037         currentStep++;
00038
00039         // Check if the whole activity has been completed
00040         if (currentStep >= totalSteps)
00041         {
00042             return true;
00043         }
00044
00045         // Activity is not yet complete, move onto next step
00046         return false;
00047     }
00048 }
```

8.63 Assets/Scripts/ModuleStructure/ModuleActivityScheduler.cs File Reference

Classes

- class ModuleActivityScheduler

  Singleton management class that talks to the ModuleManager to schedule activities ahead of Timeline endings, forming the interactive components of each section that make up a Module.

8.64 ModuleActivityScheduler.cs

Go to the documentation of this file.

```
00001 using System.Collections.Generic;
00002 using UnityEngine;
00003 using TMPro;
00004
00010
00011 public class ModuleActivityScheduler : MonoBehaviour
00012 {
00013     [SerializeField] private GameObject checkboxPanel;
00014     [SerializeField] private CheckboxManager checkboxManager;
00015     [SerializeField] private TextMeshProUGUI activityTitle;
00016
00017     [SerializeField] private ModuleManager moduleManager;
00018
00019     private ModuleActivities currentActivity;
00020     private int currentStepIndex = 0;
00021     private bool activityInProgress = false;
00022     private GameObject activeControllerObject;
00023
00024     public static ModuleActivityScheduler Instance;
00025
00030     private void Awake()
00031     {
00032         if (Instance == null)
00033         {
00034             Instance = this;
00035         }
00036         else
00037         {
00038             Destroy(gameObject);
00039         }
00040     }
00041
00048     public void StartActivity(ModuleActivities newActivity, ModuleManager manager)
00049     {
00050         // Should never happen, but stop a new activity from starting if it's already in progress
00051         if (activityInProgress)
00052         {
00053             return;
00054         }
00055
00056         // Refresh activity attribs to new slate
00057         currentActivity = newActivity;
00058         moduleManager = manager;
00059         currentStepIndex = 0;
00060         activityInProgress = true;
00061
00062         // UI setup
00063         checkboxPanel.SetActive(true);
00064         SetupActivityUI();
00065
00066         // Optionally instantiate controller prefab
00067         if (currentActivity.customScript != null)
00068         {
00069             activeControllerObject = Instantiate(currentActivity.customScript, new Vector3(0, 0, 0),
      Quaternion.identity);
00070
00071             var controller = activeControllerObject.GetComponent<IActivityController>();
00072             if (controller != null)
00073             {
00074                 controller.StartActivity();
00075             }
00076         }
00077
00078         // Display first step! Let's get this party started
00079         DisplayCurrentStep();
00080     }
00081
00086     private void SetupActivityUI()
00087     {
00088         checkboxManager.SetupSteps(currentActivity.instructions);
00089     }
00090
00095     private void DisplayCurrentStep()
00096     {
00097         if (currentStepIndex < currentActivity.instructions.Length)
00098         {
00099             activityTitle.text = currentActivity.activityName;
00100             checkboxManager.HighlightStep(currentStepIndex);
00101         }
00102         else // If there are no more steps, the activity is complete!
00103         {
00104             CompleteActivity();
```

```
00105         }
00106     }
00107
00112     private void OnStepCompleted()
00113     {
00114         Debug.Log($"Step completed: {currentStepIndex}");
00115         // Check that box!
00116         checkboxManager.CompleteStep(currentStepIndex);
00117         currentStepIndex++;
00118
00119         if (currentStepIndex < currentActivity.instructions.Length)
00120         {
00121             Debug.Log("Displaying next step...");
00122             DisplayCurrentStep();
00123         }
00124         else
00125         {
00126             Debug.Log("All steps done, completing activity...");
00127             CompleteActivity();
00128         }
00129     }
00130
00135     public void OnExternalStepCompleted()
00136     {
00137         Debug.Log("MAS: OnExternalStepCompleted() called!");
00138         OnStepCompleted();
00139     }
00140
00145     private void CompleteActivity()
00146     {
00147         Debug.Log($"Completing activity: {currentActivity}");
00148         // Cleaning up
00149         activityInProgress = false;
00150         if (activeControllerObject != null)
00151         {
00152             Destroy(activeControllerObject);
00153         }
00154
00155         checkboxPanel.SetActive(false);
00156         currentActivity = null;
00157
00158         Debug.Log("Calling ModuleManager.OnActivityComplete()");
00159         moduleManager.OnActivityComplete();
00160     }
00161 }
```

8.65 Assets/Scripts/ModuleStructure/ModuleManager.cs File Reference

Classes

- class ModuleManager

  Central driver class to control the flow of logic on which Modules/sections/timelines/activities should play, and in sequence.
- class ModuleManager.Module

  Modules are the collections of sections that form learning content within the application.
- class ModuleManager.Section

  Sections are combinations of timelines and activities that form a Module.

8.66 ModuleManager.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002 using UnityEngine.Playables;
00003 using UnityEngine.Timeline;
00004 using UnityEngine.UI;
00005 using System.Collections.Generic;
00006
00012
00013 public class ModuleManager : MonoBehaviour
00014 {
```

```
00018     [System.Serializable]
00019     public class Module
00020     {
00021         public string name;
00022         public List<Section> sections;
00023     }
00024
00028     [System.Serializable]
00029     public class Section
00030     {
00031         public string name;
00032         public List<PlayableDirector> timelines;
00033         public List<ModuleActivities> activities;
00034     }
00035
00036     [SerializeField] private ModuleActivityScheduler activityScheduler;
00037     public List<Module> modules = new List<Module>();
00038
00039     private int currentModuleIndex = 0;
00040     private int currentSectionIndex = 0;
00041     private int currentTimelineIndex = 0;
00042     private PlayableDirector activeDirector;
00043
00044     private bool waitingForActivity = false;
00045
00046     void Start()
00047     {
00048
00049     }
00050
00055     void PlayCurrentTimeline()
00056     {
00057         var module = modules[currentModuleIndex];
00058         var section = module.sections[currentSectionIndex];
00059
00060         // If there are no timelines in the currently selected section, skip it
00061         if (section.timelines.Count == 0)
00062         {
00063             NextSection();
00064             return;
00065         }
00066
00067         // Reassign current director, and play it
00068         activeDirector = section.timelines[currentTimelineIndex];
00069         activeDirector.stopped += OnTimelineFinished; // This is a subscription to an event! When the
      timeline finishes (the timeline being a PlayableDirector) it will automatically call
      OnTimelineFinished() below
00070         activeDirector.Play();
00071     }
00072
00078     void OnTimelineFinished(PlayableDirector director)
00079     {
00080         if (waitingForActivity) return;
00081
00082         Debug.Log($"Timeline finished: {director}");
00083         Debug.Log("waitingForActivity: " + waitingForActivity); //
00084
00085         director.stopped -= OnTimelineFinished; // This is an unsubscription to an event! This is just
      to ensure that we don't end up with multiple stacking event listeners. It's good practice
00086
00087         var section = modules[currentModuleIndex].sections[currentSectionIndex];
00088
00089         // Check to see if there exists an activity for this timeline
00090         ModuleActivities activityForTimeline = null;
00091
00092         if (section.activities != null && currentTimelineIndex < section.activities.Count)
00093         {
00094             activityForTimeline = section.activities[currentTimelineIndex];
00095             Debug.Log("activityForTimeline: " + activityForTimeline);
00096         }
00097
00098         if (activityForTimeline != null && !waitingForActivity)
00099         {
00100             waitingForActivity = true;
00101             Debug.Log($"Activity has started: {activityForTimeline}");
00102             StartActivity(activityForTimeline);
00103             return;
00104         }
00105
00106         // If we get here, there is no activity and we can skip directly to the next timeline
00107         waitingForActivity = false;
00108         NextTimeline();
00109     }
00110
00116     void StartActivity(ModuleActivities activity)
00117     {
00118         // Skip if no activities are assigned
```

```
00119         if (activity == null)
00120         {
00121             OnActivityComplete();
00122             return;
00123         }
00124 
00125         // Talk to the activity scheduler and send the activity for this section
00126         activityScheduler.StartActivity(activity, this);
00127     }
00128 
00133     public void OnActivityComplete()
00134     {
00135         waitingForActivity = false;
00136         NextTimeline();
00137     }
00138 
00143     void NextTimeline()
00144     {
00145         var section = modules[currentModuleIndex].sections[currentSectionIndex];
00146         currentTimelineIndex++;
00147 
00148         if (currentTimelineIndex >= section.timelines.Count)
00149         {
00150             NextSection();
00151             return;
00152         }
00153 
00154         PlayCurrentTimeline();
00155     }
00156 
00161     void NextSection()
00162     {
00163         currentTimelineIndex = 0;
00164         currentSectionIndex++;
00165 
00166         var module = modules[currentModuleIndex];
00167         if (currentSectionIndex >= module.sections.Count)
00168         {
00169             NextModule();
00170             return;
00171         }
00172 
00173         PlayCurrentTimeline();
00174     }
00175 
00180     void NextModule()
00181     {
00182         currentSectionIndex = 0;
00183         currentTimelineIndex = 0;
00184         currentModuleIndex++;
00185 
00186         if (currentModuleIndex >= modules.Count)
00187         {
00188             // TODO
00189             // Last module completed.
00190             return;
00191         }
00192 
00193         PlayCurrentTimeline();
00194     }
00195 
00201     public void PlayModule(int moduleIndex)
00202     {
00203         if (moduleIndex < 0 || moduleIndex >= modules.Count)
00204         {
00205             return;
00206         }
00207 
00208         currentModuleIndex = moduleIndex;
00209         currentSectionIndex = 0;
00210         currentTimelineIndex = 0;
00211 
00212         PlayCurrentTimeline();
00213     }
00214 }
```

8.67 Assets/Scripts/UI/AirspeedMeterDisplay.cs File Reference

Classes

- class AirspeedMeterDisplay

  A dynamic airspeed meter that is displayed on the UI.

8.68 AirspeedMeterDisplay.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002 using TMPro;
00003
00009
00010 public class AirspeedMeterDisplay : MonoBehaviour
00011 {
00012     [SerializeField] public TextMeshProUGUI AirspeedMeter;
00013     public float knots = 0;
00014     public float targetValue;
00015     private float moveSpeed;
00016     public bool isMoving;
00017
00018     void Start()
00019     {
00020         //setKnots(40f);
00021         moveTowardsKnotsTarget(40f, 10f);
00022     }
00023
00024     void Update()
00025     {
00026         AirspeedMeter.text = knots.ToString("F0") + " knots"; // String formatted as 0 knots (no
      decimals)
00027
00028         // Interpolation only happens if flag says it should
00029         if (isMoving)
00030         {
00031             knots = Mathf.MoveTowards(knots, targetValue, moveSpeed * Time.deltaTime);
00032
00033             // If it's close enough to the target (accounting for rounding errors), we're done here!
00034             if (Mathf.Approximately(knots, targetValue))
00035             {
00036                 isMoving = false;
00037             }
00038         }
00039     }
00040
00046     void setKnots(float newKnots)
00047     {
00048         knots = newKnots;
00049         isMoving = false;
00050     }
00051
00052     // Glorified mutator for targetValue that Update() then makes use of
00059     void moveTowardsKnotsTarget(float target, float speed)
00060     {
00061         targetValue = target;
00062         moveSpeed = Mathf.Abs(speed);
00063         isMoving = true;
00064     }
00065 }
```

8.69 Assets/Scripts/UI/AoADisplay.cs File Reference

Classes

- class AoADisplay

  A dynamic Angle of Attack meter that is displayed on the UI, changing colour dynamically based on how close to critical angle it is.

8.70 AoADisplay.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002 using TMPro;
00003
00009
00010 public class AoADisplay : MonoBehaviour
```

```
00011 {
00012     [SerializeField] public TextMeshProUGUI AoAMeter;
00013     public GameObject aircraft;
00014
00015     void Start()
00016     {
00017
00018     }
00019
00020     void Update()
00021     {
00022         float aoa = aircraft.transform.eulerAngles.z; // Get the Angle of Attack
00023
00024         float t = Mathf.InverseLerp(0f, 15f, aoa);
00025
00026         Color textColour = Color.Lerp(Color.green, Color.red, t);
00027
00028         AoAMeter.color = textColour;
00029         AoAMeter.text = aoa.ToString("F0") + "";
00030     }
00031 }
```

8.71 Assets/Scripts/UI/CheckboxGroupUI.cs File Reference

Classes

- class CheckboxGroupUI

  A group of UI checkboxes used in activity guidance.

8.72 CheckboxGroupUI.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002 using TMPro;
00003 using UnityEngine.UI;
00004
00010
00011 public class CheckboxGroupUI : MonoBehaviour
00012 {
00013     public GameObject groupObject;
00014     public TextMeshProUGUI labelText;
00015     public Image checkImage;
00016
00022     public void SetChecked(bool isChecked)
00023     {
00024         checkImage.enabled = isChecked;
00025     }
00026
00032     public void SetLabel(string newText)
00033     {
00034         labelText.text = newText;
00035     }
00036
00037     // Show/hide entire checkbox item
00038     public void SetVisible(bool visible)
00039     {
00040         groupObject.SetActive(visible);
00041     }
00042 }
```

8.73 Assets/Scripts/UI/CheckboxManager.cs File Reference

Classes

- class CheckboxManager

  A central class to manage checkboxes in tandem with activities to support the user.

8.74 CheckboxManager.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002 using System.Collections.Generic;
00003 using TMPro;
00004 using UnityEngine.UI;
00005
00011
00012 public class CheckboxManager : MonoBehaviour
00013 {
00014     [SerializeField] private List<CheckboxGroupUI> checkboxGroups = new List<CheckboxGroupUI>();
00015
00016     private int totalSteps = 0;
00017
00023     public void SetupSteps(string[] instructions)
00024     {
00025         totalSteps = instructions.Length;
00026
00027         // Hide all checkboxes and descriptions first before changing them and bringing them back
      changed, and with the correct quantity
00028         foreach (var group in checkboxGroups)
00029         {
00030             group.SetVisible(false);
00031             group.SetChecked(false);
00032         }
00033
00034         for (int i = 0; i < totalSteps && i < checkboxGroups.Count; i++)
00035         {
00036             var group = checkboxGroups[i];
00037
00038             group.SetLabel(instructions[i]);
00039             group.SetChecked(false); // Start unchecked!
00040
00041             // Now we can enable the checkbox group
00042             group.SetVisible(true);
00043         }
00044     }
00045
00051     public void HighlightStep(int stepIndex)
00052     {
00053         for (int i = 0; i < checkboxGroups.Count; i++)
00054         {
00055             var group = checkboxGroups[i];
00056
00057             if (i == stepIndex)
00058             {
00059                 group.labelText.color = Color.yellow;
00060             }
00061         }
00062     }
00063
00069     public void CompleteStep(int stepIndex)
00070     {
00071         // Should never happen, but checking for invalid checkbox index
00072         if (stepIndex < 0 || stepIndex >= checkboxGroups.Count)
00073         {
00074             return;
00075         }
00076
00077         var group = checkboxGroups[stepIndex];
00078         group.SetChecked(true);
00079
00080         group.labelText.color = Color.green; // Change step colour to greed when completed
00081         group.labelText.ForceMeshUpdate();
00082     }
00083 }
```

8.75 Assets/Scripts/UI/MenuController.cs File Reference

Classes

- class MenuController

8.76 MenuController.cs

Go to the documentation of this file.

```
00001 //using UnityEditor.VersionControl;
00002 using UnityEngine;
00003 using UnityEngine.InputSystem;
00004
00005 public class MenuController : MonoBehaviour
00006 {
00007
00008     public GameObject menuPanel;
00009     public InputActionReference openMenuAction;
00010     private void Awake()
00011     {
00012         openMenuAction.action.Enable();
00013         openMenuAction.action.performed += ToggleMenu;
00014         InputSystem.onDeviceChange += OnDeviceChange;
00015     }
00016
00017     private void OnDestroy() {
00018
00019         openMenuAction.action.Disable();
00020         openMenuAction.action.performed -= ToggleMenu;
00021         InputSystem.onDeviceChange -= OnDeviceChange;
00022
00023     }
00024     private void ToggleMenu(InputAction.CallbackContext context) {
00025
00026         menuPanel.SetActive(!menuPanel.activeSelf); // the variable menuPanel of menuController has
      not been assigned.
00027     }
00028     private void OnDeviceChange(InputDevice device, InputDeviceChange change) {
00029
00030         switch (change) {
00031
00032         case InputDeviceChange.Disconnected:
00033             openMenuAction.action.Disable();
00034             openMenuAction.action.performed -= ToggleMenu;
00035             break;
00036         case InputDeviceChange.Reconnected:
00037             openMenuAction.action.Enable();
00038             openMenuAction.action.performed += ToggleMenu;
00039             break;
00040         }
00041     }
00042 }
```

8.77 Assets/Scripts/UI/ModuleMenuSelection.cs File Reference

Classes

- class ModuleMenuSelection

  Handles menu selection of each Module alongside a Play button to trigger Modules via the ModuleManager.

8.78 ModuleMenuSelection.cs

Go to the documentation of this file.

```
00001 using UnityEngine;
00002 using UnityEngine.UI;
00003 using UnityEngine.Playables;
00004
00010
00011 public class ModuleMenuSelection : MonoBehaviour
00012 {
00013     [Header("Buttons")]
00014     [SerializeField] public Button playButton;
00015     [SerializeField] public Button module1Button;
00016     [SerializeField] public Button module2Button;
00017     [SerializeField] public Button module3Button;
00018     [SerializeField] public Button backButton;
```

```
00019
00020     [Header("Module Manager")]
00021     [SerializeField] private ModuleManager moduleManager;
00022
00023     private int selectedModule = -1;
00024
00025     void Start()
00026     {
00027         // No modules selected to start with, play button disabled
00028         playButton.interactable = false;
00029
00030         // Buttons will call SelectModule passing in their unique index when clicked
00031         //module1Button.onClick.AddListener(() => SelectModule(0));
00032         module2Button.onClick.AddListener(() => SelectModule(1));
00033         //module3Button.onClick.AddListener(() => SelectModule(2));
00034
00035         // Methods called when play or back buttons are clicked
00036         playButton.onClick.AddListener(OnPlayPressed);
00037         backButton.onClick.AddListener(OnBackPressed);
00038     }
00039
00045     void SelectModule(int moduleIndex)
00046     {
00047         // Update selected module
00048         selectedModule = moduleIndex;
00049
00050         // Allow the play button to be interacted with
00051         playButton.interactable = true;
00052     }
00053
00058     void OnPlayPressed()
00059     {
00060         // No module selected, do nothing
00061         if (selectedModule == -1) return;
00062
00063         // Disable the menu and reset state of menu
00064         playButton.interactable = false;
00065         gameObject.SetActive(false);
00066
00067         // Play the selected module timeline by talking to ModuleManager
00068         moduleManager.PlayModule(selectedModule);
00069         selectedModule = -1;
00070     }
00071
00076     void OnBackPressed()
00077     {
00078         // Reset state of menu
00079         selectedModule = -1;
00080         playButton.interactable = false;
00081     }
00082 }
```